

mealplanning(mp) — 통합 인프라 시방서

Docker(VM) → 온프레미스 Kubernetes → AWS EKS · 3단계 전 구간 · 오브젝트 · 클라우드 자원 통합판

문서 버전	v1.0 (통합본) · 2026-08-25
문서 성격	확정 시방서. 모든 행은 ① 확정된 설계 결정이거나 ② 실측으로 확인된 형상이다. 추정·예정은 실지 않는다.
대상 시스템	월 식비 예산 기반 밀플래닝 서비스 — 애플리케이션 12종 · 데이터 티어(PostgreSQL-Elasticsearch-Redis) · 수집 파이프라인 · AI 5종 + 서버리스 10종
문서 범위	Docker(VM) · 온프레미스 Kubernetes · AWS EKS 세 단계 전부. 계층별 전환 대조(2부) + K8s 오브젝트 사양(3부) + AWS 자원 사양(4부).
판독법	2부 대조 시트는 5열 고정 — [항목 ① Docker ② 온프레임 K8s ③ AWS EKS 전환 근거]. ① 주황 · ② 파랑 · ③ 초록. 3부는 Kind 별 오브젝트 사양이며 '(양 사이트)' 로 표기한 시트는 온프레미스-EKS 양쪽을 함께 수록한다. 그 외는 온프레미스 기준이다. 4부는 AWS 고유 자원이며 상태값은 '적용' 과 '미채택' 둘뿐이다.
현재 위치	②와 ③이 동시에 살아 있다. ③(EKS)은 aws.mealbong.cloud 로 라이브지만 프로덕션 진입점(app.mealbong.cloud)은 아직 ②를 향한다. ②의 이관 후 역할 = DR 대기(Warm Standby) + 크롤 상시 프로덕션 이중역할. ②는 폐기 대상이 아니다(3-25 Cronjob 시트가 그 실물이다).
실측 기준	Kubernetes: kubectl(양 클러스터, 2026-08-25) · AWS: eks/ec2/iam/s3/ecr/sqs/elasticache/elbv2/logs API · 커밋 이력: git log(최초 2026-07-09)
마스킹	AWS 계정 ID 는 <ACCOUNT_ID> 로, 공인 IP-엔드포인트 호스트명은 생략한다. 사설 대역(192.168.0.x · 10.x)은 그대로 적는다.
작성 근거	docs/mp_k8s_infra_status.md · docs/mp_aws_prep_checklist.md(확정 결정 C-1~C-89) · docs/mp_k8s_backup_strategy.md · docs/docker-infra-status.md(SUPERSEDED · 이력 참조)

목 차

부	시트	답는 것	원천
I. 전환 개요	1-1 단계 정의·이력	Docker→K8s→AWS 3단계의 경계 날짜와 규모 지표	3단계 대조(신규)
	1-2 형상 대조 (총괄)	22개 계층을 한 장에 — 이 문서의 요약	3단계 대조(신규)
II. 계층별 3단계 대조	2-01 컴퓨터·노드	물리 호스트·VM/노드 사양·아키텍처(amd64→arm64)	3단계 대조(신규)
	2-02 네트워크·진입	CNI·LB·진입 경로·VPC/서브넷·방화벽	3단계 대조(신규)
	2-03 이미지·레지스트리	베이스 이미지·태그 정책·Harbor→ECR·취약점 게이트	3단계 대조(신규)
	2-04 애플리케이션 워크로드	서비스 12종의 배치 형태 변화	3단계 대조(신규)
	2-05 데이터 티어	PG·ES·Redis·Kafka 의 운영 주체 변화	3단계 대조(신규)
	2-06 스토리지	볼륨·StorageClass·오브젝트 스토리지·회수 정책	3단계 대조(신규)
	2-07 보안·접근통제	netpol·mTLS·RBAC·PodSecurity·감사	3단계 대조(신규)
	2-08 Secret 관리	.env → ESO(K8s) → Secrets Manager	3단계 대조(신규)
	2-09 CI·CD	GH Actions → Jenkins → GitLab · CD=ArgoCD 단일	3단계 대조(신규)
	2-10 배포 전략	수동 재기동 → 롤링·카나리 → Blue-Green	3단계 대조(신규)
	2-11 오토스케일	없음 → HPA·KEDA → +Karpenter	3단계 대조(신규)
	2-12 관측·알림	VM LGTM → 인클러스터 → +CloudWatch·GuardDuty	3단계 대조(신규)
	2-13 백업·DR	백업 없음 → barman→S3 → 크로스사이트 Warm Standby	3단계 대조(신규)

III. K8s 오브젝트	3-00 보안 개요 (심층방어)	계층별 통제 수단 개요	K8s 오브젝트 시방서
	3-01 Node	(양 사이트) 온프레미스 5 + EKS 2 + Karpenter	실측 보완(신규)
	3-02 NameSpace	ns 별 메시-PSS·생성 출처	K8s 오브젝트 시방서
	3-03 PriorityClass	우선순위 티어	K8s 오브젝트 시방서
	3-04 ResourceQuota	ns 자원 상한	K8s 오브젝트 시방서
	3-05 LimitRange	컨테이너 기본값	K8s 오브젝트 시방서
	3-06 HPA	CPU 기반 파드 확장	K8s 오브젝트 시방서
	3-07 PodDisruptionBudget	축출 보호	K8s 오브젝트 시방서
	3-08 Probe	헬스체크 3종	K8s 오브젝트 시방서
	3-09 Deployment-Rollout	배포 전략·Pod 설계	K8s 오브젝트 시방서
	3-10 SVC	(양 사이트) app 17 · data 14 · 게이트웨이 3 (양 사이트)	실측 보완(신규)
	3-11 HTTPRoute (Canary)	L7 라우팅·카나리 가중치	K8s 오브젝트 시방서
	3-12 ConfigMap	설정	K8s 오브젝트 시방서
	3-13 Secret	비밀	K8s 오브젝트 시방서
	3-14 ServiceAccount	워크로드 신원	K8s 오브젝트 시방서
	3-15 Role (RBAC-온프레미스)	온프레미스 사람·워크로드 권한	K8s 오브젝트 시방서
	3-16 NetworkPolicy(온프레미스)	온프레미스 표준 netpol	K8s 오브젝트 시방서
	3-17 CiliumNetworkPolicy	FQDN egress	K8s 오브젝트 시방서
	3-18 Certificate (TLS)	cert-manager 인증서	K8s 오브젝트 시방서
	3-19 PodSecurity	PSA·SecurityContext	K8s 오브젝트 시방서
	3-20 Mesh	PeerAuthentication·사이드카	K8s 오브젝트 시방서
	3-21 PVC	(양 사이트) 온프레미스 21 + EKS 15 (전량 실측)	실측 보완(신규)
	3-22 PV	(양 사이트) 회수 정책·결박 단위	실측 보완(신규)
	3-23 StorageClass	SC 정의 (EKS overlay 포함)	K8s 오브젝트 시방서

	3-24 Job	(양 사이트) CronJob 산출 Job 규격	실측 보완(신규)
	3-25 Cronjob	(양 사이트) 온프레미스 23 / EKS 13 — 차이 10개가 전부 수집	실측 보완(신규)
	3-26 Operator (install)	오퍼레이터 설치 사양	K8s 오브젝트 시방서
	3-27 ArgoCD	GitOps 배선	K8s 오브젝트 시방서
	3-28 Backup-DR (상세)	백업 트랙 상세	K8s 오브젝트 시방서
	3-29 관측 Deployment	관측 워크로드	K8s 오브젝트 시방서
	3-30 관측 StatefulSet	Prometheus-Loki-Tempo	K8s 오브젝트 시방서
	3-31 관측 DaemonSet	Alloy-node-exporter	K8s 오브젝트 시방서
	3-32 관측 Service	관측 서비스	K8s 오브젝트 시방서
	3-33 관측 Storage	관측 볼륨	K8s 오브젝트 시방서
IV. AWS 자원	4-00 AWS 시방 개요	AWS 설계 원칙·상태 표기·실측 기준	AWS 설계 시방서
	4-01 Docker Image (ECR)	ECR 이미지 18종·베이스·아키텍·태그	AWS 설계 시방서
	4-02 역할분담·IAM 그룹	5인 역할과 IAM/K8s 권한	AWS 설계 시방서
	4-03 Amazon EKS	클러스터 형상 전체	AWS 설계 시방서
	4-04 EKS Object	ns별 워크로드·컨테이너·포트	AWS 설계 시방서
	4-05 IAM	사람/머신 사용자·IRSA 13·서비스 롤 6	AWS 설계 시방서
	4-06 VPC	VPC 2·서브넷 7·GW·엔드포인트·라우팅	AWS 설계 시방서
	4-07 NACL (미채택)	미채택 근거와 대체 통제	AWS 설계 시방서
	4-08 Security Group	SG 7종 인/아웃바운드	AWS 설계 시방서
	4-09 RBAC (EKS 접근)	Access Entry·ClusterRole	AWS 설계 시방서
	4-10 NetworkPolicy (EKS)	netpol 26 + CNP 29	AWS 설계 시방서
	4-11 Amazon S3	버킷 11개·암호화·Object Lock·수명주기	AWS 설계 시방서
	4-12 Amazon SQS	크롤 큐 3 + DLQ 3 + Carpenter 큐	AWS 설계 시방서
	4-13 Amazon ECR	리포 18·태그 불변성·스캔·수명주기	AWS 설계 시방서
	4-14 Amazon Bedrock	AI 배치 모델·권한 경계	AWS 설계 시방서
	4-15 Comprehend (미채택)	미채택 사유	AWS 설계 시방서
	4-16 Amazon Security	WAF·GuardDuty·CloudTrail·감사·알림	AWS 설계 시방서
	4-17 DNS·인증서	Route 53 미채택 근거·ACM	AWS 설계 시방서
V. 결정·비용·승인	5-1 미채택 결정 (전체)	검토했으나 안 쓴 것 24건과 근거	3단계 대조(신규)
	5-2 비용 (3단계 성격)	단계별 비용 성격 요약	3단계 대조(신규)
	5-3 비용 산정 (AWS 상세)	고정비·사용량 과금 실측	AWS 설계 시방서

	5-4 검토·승인	작성·검토 이력	3단계 대조(신규)
--	-----------	----------	------------

1-1 단계 정의 · 이력

단계	기간	명칭	그 단계에서 실제로 한 일 (커밋 이력 실측)	경계를 만든 사건
	07.09 ~ 07.14	주제 선정 · 설계	도메인·데이터소스 확정 · 기술 스택 결정 · 아키텍처 설계 · 최초 커밋(07-09 설계 문서 초기화)	07-09 레포 생성
①	07.15 ~ 07.26	Docker (VM) 단계	Proxmox 위 VM 4대 · Docker Compose 로 앱·데이터·관측·CI 배치. FastAPI 7종 + 데이터 티어 4종 + LGTM. Harbor 레지스트리 · GH Actions self-hosted 러너 → Jenkins 전환	07-15 스키마·서비스 정본 (SSOT) 확정
②	07.27 ~ 08.08	온프레미스 Kubernetes	kubeadm 5노드 클러스터 · Cilium · Istio 메시 · CNPG/ECK/Strimzi 로 데이터 티어 인클러스터화 · ArgoCD GitOps · NetworkPolicy·mTLS·RBAC · 부하시험 · 카나리 배포	07-27 CI 완주 → K8s 구축 착수 (P0)
③	08.09 ~ 08.18	AWS EKS 이전	확정 결정 C-1~C-89 · Terraform/Ansible IaC · GitLab CI(arm64→ECR) · EKS·ALB·WAF · 데이터 이관 · Blue-Green · Carpenter · Lambda 10종 서버리스	08-09 AWS 이관을 최우선 목적으로 승격
	08.19 ~	안정화 · 최종 발표 준비	이관 마무리 감사(F-1~F-10) · 관측 정합 · 문서화 · 발표 자료	
규모 지표 (2026-08-24 실측)				
커밋	1,490	전체 커밋 수	07-09 최초 커밋 이후 · 머지 커밋 523회 포함	
PR·이슈	611	고유 번호 수	최대 PR 번호 #690 · 전 PR 리뷰 필수(직접 머지 금지)	
문서	143	docs/ 마크다운 파일	총 43,683줄 · 코드와 같은 레포에서 PR 리뷰를 받는다(Docs as Code)	
확정 결정	89	C-1 ~ C-89	AWS 이관 확정 결정. docs/mp_aws_prep_checklist.md 가 정본	
팀	5	봉수(PL·인프라) · 태현(플랫폼·배포) · 건우(앱·AI) · 정현(관측·FinOps) · 정은(데이터)	AWS IAM 그룹 3종으로 권한 분리 · K8s 는 mp:admin 단일 등급	

②와 ③은 배타적이지 않다. ③ 완료 후에도 ②는 DR 대기 사이트 + 크롤 상시 프로덕션의 이중역할로 계속 운영된다(확정 결정 C-3).

1-2 형상 대조 (총괄)

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
컴퓨터	Proxmox VM 4대 (16 vCPU / 26GB)	kubeadm 노드 5대 (물리 2대 위)	EKS m7g.xlarge × 2~4 (Graviton)	자원 총량은 비슷하나 스케줄링 주체가 사람→K8s→클라우드로 이동
아키텍처	linux/amd64	linux/amd64	linux/arm64 (Graviton)	동일 성능 대비 단가가 낮다. 크롤러만 amd64 로 남겼다(실행 사이트가 온프레미)
오케스트레이션	Docker Compose (호스트별)	Kubernetes 1.34.10	Amazon EKS 1.34	버전을 맞춰 이식 비용을 없앴다
CNI · 네트워크	Docker bridge + vmbr1 host-only	Cilium 1.19.6 (eBPF · VXLAN · WireGuard)	Cilium 1.19.6 (eBPF · ENI 모드)	같은 CNI 를 유지해 NetworkPolicy 자산을 그대로 옮겼다. IPAM 만 바뀐다
외부 진입	호스트 nginx + 포트 노출	MetalLB L2 (.14 공개 / .15 내부) → Istio Gateway	ALB (ACM 종단 · AWS WAF) → Istio Gateway	진입 실체만 교체하고 Gateway API 리소스는 공유
데이터 티어	Docker Compose 단일 인스턴스 4종	오퍼레이터 (CNPG · ECK · Strimzi · Redis)	CNPG · ECK 자체운영 + ElastiCache for Valkey	RDS-OpenSearch-MSK 전부 미채택 — 매니지드에서 사는 것(백업·패일오버)을 이미 오퍼레이터가 준다
메시지 큐	Kafka KRaft 단일 노드	Kafka KRaft 3대 (RF=3)	없음 — S3 이벤트트 + SQS	브로커 3대와 그 운영을 사는 대신 관리형 큐로 대체. 온프레미 Kafka 는 크롤용으로 존치
스토리지	VM 로컬 디스크 (docker volume)	OpenEBS LVM LocalPV (RWO · 노드 결박)	EBS gp3 CSI (기본 SC = Retain)	LocalPV 는 노드를 못 벗어난다. EBS 는 AZ 안에서 재부착된다
오브젝트	없음	MinIO (인클러스터 · 단일 replica)	Amazon S3 (버킷 11개)	MinIO 삭제. 백업·관측 청크·크롤 원본 전부 S3
레지스트리	Harbor (VM → 호스트 C)	Harbor (호스트 C · 클러스터 밖)	Amazon ECR (18 리포 · arm64)	Harbor 는 온프레미 amd64 공급원으로 존치(C-89). 미러가 아니라 자체 공급원
CI	GitHub Actions self-hosted 러너	Jenkins (호스트 C)	GitLab CI (EC2 1대) + Jenkins 병행	사이트별로 자기 아키텍처를 네이티브로 빌드한다. 멀티아치 manifest 미채택
CD	수동 배포 (Ansible · compose up)	ArgoCD (GitOps · config 레포 :sha 핀)	ArgoCD (동일 · Application 44개)	CD 는 한 번 정하고 안 바꿨다. config 레포 오버레이만 onprem/eks 로 갈린다
배포 전략	컨테이너 재기동 (중단 발생)	롤링 7중 + 카나리 2중 (Argo Rollouts)	Blue-Green 2중 (account · recipe)	승격 1초 · 롤백 2초 · 전환 중 5xx 0건 실측
오토스케일	없음	HPA(account) + KEDA scale-to-zero	HPA 2중 + KEDA 3중 + Karpenter	노드 층 오토스케일이 ③에서 처음 생겼다
서비스 메시	없음	Istio 1.30.3 sidecar · mTLS STRICT	Istio 1.30.3 sidecar · mTLS STRICT	동일. ambient 는 각각
비밀	호스트 .env 파일	External Secrets Operator (K8s provider)	ESO → AWS Secrets Manager (IRSA)	ESO 라는 껍데기를 유지해 앱 매니페스트를 안 고쳤다. 백엔드만 교체
관측	VM 1대에 LGTM 전체 (.11)	인클러스터 kube-prometheus-stack + Loki-Tempo-Alloy	동일 + CloudWatch(감사로그) · GuardDuty	AMP/AMG 미채택 — 대시보드와 알람 자산을 그대로 옮기는 쪽이 싸다
백업	없음	barman-cloud → S3 (WAL 연속 + 정기 base)	동일 + 크로스사이트 복제	① 단계에는 백업이 존재하지 않았다
DR	없음	단일 사이트	AWS(Active) ↔ 온프레미(Warm Standby) · RPO 5분	되살릴 수 없는 원본은 PostgreSQL 하나뿐 → DR 을 여기에 집중
IaC	Terraform(Proxmox) + Ansible	동일 + Helm/Kustomize	Terraform(AWS) + Ansible(eks.yml) + Kustomize	도구 구성은 유지하고 프로바이더만 교체
사람 접근	공유 SSH 키 · 무제한	kubeadm admin.conf(cluster-admin) → RBAC 4티어	IAM 그룹 → EKS Access Entry → mp:admin	③에서 처음으로 '취소 가능한' 자격증명이 됐다
가용성	단일 VM = SPOF	노드 5대 · TSC 분산 · PDB	AZ 2개 · 노드 2~4 · PDB	②에서 이미 노드 단위 HA. ③에서 AZ 단위로 확장

이 시트는 요약이다. 각 행의 근거·실측값은 2번 이후 시트에 있다. ① 주황 = Docker · ② 파랑 = 온프레미 K8s · ③ 초록 = AWS EKS

2-01 컴퓨터 · 노드

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
물리 계층				
물리 호스트	호스트 A (192.168.0.12 · i7-10700F · 32GB) 단독	호스트 A + 호스트 B (192.168.0.22 · 32GB) 2대 + 호스트 C (.10 · VirtualBox · 클러스터 밖)	AWS 관리 (물리 계층 없음)	호스트 B 증설이 ② 의 전제였다 — A 단독으로는 VM 과 K8s 노드를 동시에 못 었는다
가상화	Proxmox VE (standalone)	Proxmox VE ×2 (클러스터 미구성 · 각각 standalone)	EC2 (Nitro)	2노드 Proxmox 클러스터는 quorum 함정으로 기각
배치 원칙		무흔적 급사 3회가 전부 호스트 A → master-quorum 다수-Prometheus-MinIO = B / PG-Redis primary = A	AZ-a · AZ-b 분산 (topologySpreadConstraints)	②의 배치 원칙은 실제 장애 이력에서 나왔다
노드 사양				
노드 목록	fb-data .8 (4c/8GB) fb-app-ai .9 (6c/7GB) fb-ci-harbor .10 (3c/5GB) fb-monitoring .11 (3c/6GB)	k8s-master .17 (2c/6GB) worker-a1 .20 (14GB) · worker-a2 .21 (11GB) — 호스트 A worker-b1 .18 · worker-b2 .19 (11GB) — 호스트 B	mp-mng-general (관리형 노드그룹 · ON_DEMAND) m7g.xlarge (4 vCPU / 16 GiB) × min 2 · desired 2 · max 4	③은 노드를 이름으로 부르지 않는다 — 그게 관리형 노드그룹의 요점
총 자원	16 vCPU / 26 GB	약 20 vCPU / 53 GB	8~16 vCPU / 32~64 GiB	②에서 자원이 2배로 늘었다 — 오퍼레이터-메시-관측이 클러스터 안으로 들어왔기 때문
OS · 런타임	Ubuntu 24.04 · Docker Engine	Ubuntu 24.04 · containerd 2.2.6	AL2023_ARM_64_STANDARD · containerd	③ 은 AMI 가 관리된다 — 노드 패치가 노드그룹 갱신으로 바뀐다
루트 볼륨	OS 80~150G + docker 전용 30~70G 분리	노드별 OS + VG openebs-vg 150G	60 GiB gp3 (암호화)	②의 전용 VG 가 LocalPV 의 물리 근거
컨트롤플레인	해당없음	master ×1 (VIP-HAProxy 없음 · controlPlaneEndpoint = .17:6443 직결) etcd = stacked 단일 · aescbc at-rest 암호화	EKS 관리형 (AWS 책임) · platform eks.31	②의 단일 컨트롤플레인은 알려진 한계다. 실측 = master 강제종료 3분 3초 동안 인그레스 중단 0(151/151 HTTP 200) · 복구 26초 — 변경 능력만 잃고 서빙은 유지된다
클러스터 밖		호스트 C .10 — Harbor · Jenkins · SonarQube (단일 디스크 98GB · 여유 27.9GB)	CI 서버 EC2 t4g.xlarge — GitLab · SonarQube · Runner (VPC-B) 대시보드 EC2 t3.medium (VPC-A 공개 서브넷)	호스트 C 는 디스크가 제약이다(RAM 아님) — 채우면 Harbor 가 죽고 배포가 전면 실패

③ 은 Spot 미체택(C-29). Blue-Green 은 promote 후 green 이 프로덕션 그 자체가 되어 파드가 이사하지 않고, 상시 Spot 도 메모리의 10%에만 걸린다(실측 CPU 요청/실사용 9.4배 vs 메모리 1.1배 — 실제 제약은 CPU 가 아니라 메모리다).

2-02 네트워크 · 진입

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
데이터플레인				
CNI	Docker bridge (docker0) + vbr1 host-only 10.10.10.0/24	Cilium 1.19.6 · kubeProxyReplacement · kube-proxy 미 설치	Cilium 1.19.6 · IPAM = ENI 모드	같은 CNI 를 유지한 것이 NetworkPolicy 자산을 그대로 옮긴 이유
라우팅 모드		VXLAN 확장력 (2026-07-27 실측 — 예상이던 native 를 뒤집음)	ENI — 파드가 노드 서브넷의 VPC IP 를 직접 받는다	②: CPU 천장 2.25Gbps > 물리 1GbE 라 선이 먼저 찬다 → 캡슐화 오버헤드가 무의미 ③: 파드가 VPC IP 라 ALB 가 파드를 직접 타깃한다
노드 간 암호화	없음	Cilium WireGuard (파드 간)	VPC 내부 · SG 로 통제	③은 계정 경계가 곧 신뢰 경계
파드 CIDR		10.0.0.0/8 계열 (클러스터 내부)	VPC 서브넷에서 직접 할당 (10.10.64.0/20 · 10.10.80.0/20)	ENI 모드의 대가 = 서브넷 IP 소모. /20 두 개로 여유 확보
진입 경로				
외부 진입	호스트 nginx 리버스 프록시 + 포트 직접 노출	MetalLB 0.16.1 L2 · 풀 .14~.16 type LoadBalancer 는 게이트웨이 전용 2개(공개 .14 · 내부 .15) — 개별 서비스 노출 금지	ALB (ACM TLS 중단 · AWS WAF 부착) mp-vpce 없이 공개 서브넷 2 AZ	③에서 NLB 패스스루(C-26)를 ALB(C-60)로 정정 — WAF 를 붙이려면 L7 이어야 한다
L7 게이트웨이	없음	Gateway API v1.6.1 · 구현체 Istio · Gateway mp-gw-public (ns mp-ingress)	동일 (Gateway API · Istio)	게이트웨이 리소스를 공유해 진입 실체만 교체했다
공개 도메인	없음 (LAN 전용)	app.mealbong.cloud → Cloudflare Tunnel(주황) → 게이트웨이	aws.mealbong.cloud → Cloudflare(회색) → ALB	프로덕션 진입점은 아직 ②를 향한다 — A3 DNS 컷오버 미완
내부 도구	VM 포트 직접 (Grafana :3000 등)	https://<이름>.mealbong.cloud 6중 (내부 게이트웨이 .15)	ArgoCD 만 노출(argocd-aws.mealbong.cloud) · 나머지는 port-forward	③은 WAF IP allowlist 가 문지기
경계 · 격리				
VPC · 세그먼트	단일 LAN 192.168.0.0/24 + host-only 10.10.10.0/24	단일 LAN (노드 .17~.21 · LB .14~.16)	mp-vpc-service 10.10.0.0/16 (공개2·노드2·데이터2 서브넷) mp-vpc-ci 10.11.0.0/16 (피어링 없음)	CI VPC 를 분리하고 피어링을 안 걸었다 — 빌드 서버가 프로덕션 대역을 못 본다
아웃바운드	VM 이 공유기 경유 직접	노드가 공유기 경유 직접	NAT Gateway 1대 (2 AZ 공유) + S3 Gateway Endpoint(무료) Interface Endpoint x3 (SQS · Secrets Manager · STS)	Interface Endpoint 3종의 근거는 비용이 아니라 NAT 단일 구성의 우회다
방화벽	호스트 ufw (실질 미운영)	NetworkPolicy + CiliumNetworkPolicy(FQDN egress)	Security Group 7중 + 동일 NetworkPolicy 자산	③은 SG(인프라) + netpol(파드) 2층
NACL		해당없음	미채택 — SG(상대유지) 로 충분하고 NACL(무상태)은 반환 트래픽 규칙을 사람이 관리해야 한다	의도적 미채택. 근거는 AWS 시방서 7번 시트

②→③ 에서 Cilium 마스크레이드 사고가 있었다 — egressMasqueradeInterfaces 를 eth0 로 뒀는데 Nitro 는 ens5 라 파드가 VPC 밖으로 못 나왔다. 최종값은 en+ (빈값 금지 · 콤마 목록 불가). 1차 ENI 파드만 우연히 정상이라 '일부분만 안 됨' 으로 오인하기 쉬웠다.

2-03 이미지 · 레지스트리

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
레지스트리	Harbor (fb-ci-harbor VM .10)	Harbor (호스트 C .10 · 프로젝트 mealplanning/)	Amazon ECR — 리포지토리 18개	②의 Harbor 는 ③ 이후에도 존치한다(C-89) — ECR 미러가 아니라 온프레미 amd64 의 자체 공급원
이미지 수	앱 7 + 파이프라인 일부	14종 (앱 11 + frontend + pgsync + data-pipeline 등)	18 리포 (arm64 16 + amd64 2)	③에서 operations-crawler-kurly 는 EKS 배포 대상이 아니다
아키텍처	linux/amd64	linux/amd64	linux/arm64 (Graviton) 예외: mp-crawler-kurly = amd64	멀티아치(manifest list) 미채택 — 사이트별로 자기 아키를 네이티브로 빌드한다. buildx-QEMU 불필요
베이스 이미지	python:3.12-slim · node · nginx	동일	python:3.12-slim / node:22-bookworm-slim → nginx:1.27-alpine (멀티스테이지) ES = docker.elastic.co + nori 커스텀	베이스는 3단계 내내 동일하게 유지
태그 정책	임의 (:latest 위주)	3태그 — :<sha> 불변 + :X.Y.Z 릴리스 불변 + :latest 가변	동일 3태그 + ECR imageTagMutability = IMMUTABLE_WITH_EXCLUSION	매니페스트 핀은 반드시 :<sha> — :latest 를 쓰면 ArgoCD 가 변경을 감지 못 하고 롤백이 불가능해진다
취약점 스캔	없음	Trivy 게이트 (CRITICAL · --ignore-unfixed · 차단)	동일 Trivy 게이트 + ECR 스캔	arm64 단일이라 docker build 가 이미지를 로컬에 남긴다 → Trivy 차단 게이트가 push 앞에 그대로 선다
코드 품질	없음	SonarQube (측정만 · 비차단)	동일 (CI EC2 에 동거)	의도적 비차단 — 게이트로 만들면 개발이 멈춘다
수명주기	수동	수동	ECR 수명주기 정책	③에서 처음 자동화

2-04 애플리케이션 워크로드

서비스	역할	포트	① Docker → ② 온프레임 K8s → ③ AWS EKS	비고
애플리케이션 12종 (ns app · Istio 사이드카 주입)				
account	계정·인증 (JWT · OAuth 카카오후글)	8001	compose → Deployment(HPA) → Blue-Green + HPA(min2/max4)	Blue-Green 대상 2종 중 하나. 승격 1초 · 롤백 2초
recipe	레시피 검색·상세 (Elasticsearch)	8002	compose → Deployment(카나리) → Blue-Green + HPA	부하시험에서 유일하게 포화 → 4 replica 로 수평 확장해 해결(p95 2.7s → 46ms)
price	가격 조회·최저가	8003	compose → Deployment → Deployment	캐시 스탬피드 해소(SWR + single-flight) — 실패 1,783 → 2건
chat	챗봇 (의도분류 + 템플릿 + RAG)	8004	compose → Deployment → Deployment	Gemini 다듬기는 opt-in
pantry	냉장고 재고	8005	compose → Deployment → Deployment	
recipebook	레시피북 · 공유	8006	compose → Deployment → Deployment	멘토 피드백의 다자간 트래픽 축
mealplan	식단 계획 · 예산	8007	compose → Deployment → Deployment	HPA 무용 — 다운스트림 대기형
notify	알림 fan-out	8008	compose → Deployment → Deployment	최저가 알림 fan-out
ocr	영수증 OCR (Gemini)	8009	compose → Deployment → Deployment	HITL 확정 저장
video	영상 URL 재료 추출 (YouTube + Gemini)	8010	→ Deployment → Deployment	외부 유료 API 예외 승인 건
ranking-serving	레시피 랭킹 서빙 (LightGBM)	8011	→ Deployment → Deployment	전부 CPU — GPU 학습 금지 제약
frontend	React / Vite / PWA	80	nginx 정적 → Deployment → Deployment	멀티스테이지 빌드
파이프라인 (ns pipeline · 메시 제외)				
crawler (CronJob 7종)	마켓컬리 · 오아시스 · 만개의레시피 수집		host cron → CronJob(KST) → 온프레임 잔류 (C-3)	③ 이후에도 온프레임에서 상시 돈다 — 크롤 실행 사이트가 AWS 가 아니다
consumer (Kafka)	수집 원본 정제·적재		compose → Deployment(KEDA scale-to-zero) → SQS 컨슈머(KEDA min0/max1)	SQS 에는 Kafka 의 product_id 파티션 친화성이 없어 PG 데드락 → maxReplica 1
relay_kafka_s3	온프레임 Kafka → S3 운반		→ — → 온프레임 릴레이 → S3 → SQS	3 스트림(retail · deal · recipe)
pgsync	PostgreSQL → Elasticsearch CDC		compose → Deployment(replicas 1 고정) → 동일	복제 슬롯이 단일 소비자라 replicas 를 못 늘린다
ranking-retrain	랭킹 모델 재학습		host cron → CronJob → CronJob	CPU 전용 (LightGBM)
AI 서버리스 (③ 에서 신설 · 독립 트랙)				

Lambda 10종	AI 배치 · 챗 API · RAG		→ — → AWS Lambda + Bedrock	EKS 앱 이전이 아니라 독립 프로젝트. IAM 그룹 mealplanning-ai 로 경계 분리 — 이름이 곧 경계 (mp-ai-* 밖은 거부)
------------	---------------------	--	----------------------------	--

operations(운영 API)는 EKS 배포 대상이 아니다 — 대시보드 EC2 의 docker compose 로 나갔다(C-84). 이미지는 레지스트리 정합을 위해서만 존재한다.

2-05 데이터 티어

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
PostgreSQL	postgres:16-alpine (compose · mem 1.5G) DB 2개 — terraform_state + foodbudget	CloudNativePG — primary(호스트A) + standby(호스트B) · 비동기 복제 + Pooler(PqBouncer · transaction 모드 · replica 2)	CloudNativePG — pg-1 · pg-2 + pgBouncer RDS 미채택	매니지드에서 사는 것은 백업·패일오버인데 CNPG 가 이미 준다. 스키 마-퍼-서비스 · role 격리 유지
Elasticsearch	ES 8.15.3 + nori 로컬빌드 (single-node · security off · heap 512m)	ECK — 3노드 · replicas 1 · 인증 컴 + HTTP TLS 끄(암호 화는 WireGuard 담당)	ECK — 동일 3노드 (mp-elasticsearch-nori arm64) OpenSearch Service 미채택	ES 는 파생 데이터다 — PG 에서 재색인 가능(실측 1.0초). 백업 대상 이 아니다
Redis	redis:7-alpine (캐시 LRU 256mb · 비영속)	OT-Container-Kit — primary + replica + Sentinel ×3 + 별도 redis-pqsync (통합 금지)	Amazon ElastiCache for Valkey — cache.t4g.micro × 2 (Multi-AZ)	판단 측은 비용이 아니라 Sentinel 운영 부담이었다 — 오퍼레이터 결 함 우회 코드가 프로덕션에 있었다(C-14)
Kafka	apache/kafka:3.9.0 — KRaft 단일 노드 (heap 512m)	Strimzi — KRaft combined 3 · RF=3 · min.insync.replicas=2 · auto.create.topics=false	없음 — S3 이벤트 알림 → SQS 로 대체 온프레미 Kafka 는 크롤용으로 존치	S3→SQS 는 EventBridge 가 아니라 버킷 이벤트 알림 직결이다 — 재 시도-리플레이가 없다
커넥션 풀	없음 (앱이 직접)	CNPG Pooler — 앱 9종 경유 · 풀 10→5	동일	실증 = account 4 replica 에서도 PG 커넥션 12/100
CDC	없음	PGSync (PG → ES · replicas 1 · pg-rw 직접 접속)	동일 (bootstrap 은 manual-only Job)	ArgoCD 혹 금지 — stock bootstrap 이 슬롯-트리거를 drop 한다
스키마	41 테이블 · 스키마-퍼-서비스 하이브리드	동일 (먹통 DDL · schema-production.sql)	동일 — 온프레미와 체크섬 일치(crawl_raw 의도적 제외)	ORM-Alembic 미사용 — psycopg3 + row_factory=dict_row
HA	없음 — 전부 단일 인스턴스	전 컴포넌트 HA (MinIO 만 문서화된 예외)	PG 2 · ES 3 · ElastiCache Multi-AZ	①에서 데이터 티어 VM 이 중단되면 서비스가 전면 정지했다

데이터 이관은 전부 논리 경로였다 — PG=barman/pg_dump · ES=재색인 · Kafka=드레인. 볼륨 레벨 복제는 원리상 불가(온프레미 LVM 21/21 이 thick, lvm-driver 는 thin 만 복원 지원).

2-06 스토리지

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
블록 볼륨	VM 로컬 디스크 · docker named volume (docker 전용 디스크 30~70G 분리)	OpenEBS LVM LocalPV 1.9.1 · CSI · RWO · WaitForFirstConsumer RWX 금지 · 노드 VG = openebs-vg(150G)	Amazon EBS gp3 CSI · PVC 총 230 GiB	LocalPV 는 정의상 노드를 못 벗어난다 — 그 노드가 unschedulable 이면 파드가 어디에도 못 뜬다(kubecost 실사고)
StorageClass	해당없음	openebs-lvm 2종	gp3 (Delete) · gp3-retain (Retain) 기본 SC 를 Retain 쪽에 뒀다	온프레미과 반대 선택. PVC 를 실수로 지웠을 때 데이터가 즉시하지 않게
PV 회수 정책	해당없음	라이브 PV 21/21 Retain 으로 전환 (파드 재시작 0)	기본 SC = Retain	②의 구멍 = 기본 SC 는 여전히 Delete 라 신규 PVC 는 보호 밖이었다 → ③에서 해소
볼륨 총량	VM 디스크 합계 약 430G	PVC 352 GiB → 125 GiB 로 정리	EBS 520 GiB (노드루트120 + PVC230 + CI140 + 대시보드30)	③ 산정 시 노드 루트 EBS 까지 비용에 편입
오브젝트 스토리지	없음	MinIO (인클러스터 · 단일 replica · 호스트 B 고정) + AWS S3 (백업 전용 · ap-northeast-2)	Amazon S3 — 버킷 11개 MinIO 삭제	MinIO 는 '전 컴포넌트 HA' 의 유일한 문서화된 예외였다
오브젝트 용도		Loki-Tempo 청크 · 모델 아티팩트	백업 · 관측 청크 · 크롤 원본 · Terraform state	Loki/Tempo 크래시루프 사고 — CNP toFQDNs 에 가상호스트 S3 이름 누락. 증상이 16시간 늦게 나온다(도는 파드는 살아 있고 재시작 때 죽음)
Terraform state	PG 원격 backend (fb-data DB)	S3 원격 backend (mp-backup-ap2 · S3 네이티브 락)	동일 S3 backend	①의 PG backend 는 순환 의존이었다 — 인프라를 만드는 도구의 상태가 그 인프라 안에 있었다

2-07 보안 · 접근통제

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
네트워크 계층				
파드 간 통제	없음 — 같은 호스트의 컨테이너가 서로 전부 보았다	NetworkPolicy 26종 — app ns 양방향 default-deny (무충단 적용 · 차단 실증)	동일 자산 이식 + Security Group 7종	주의 = 잠그기 전 실측에 :80 리다이렉트가 빠져 있었다. 트래픽을 직접 만들어 재측정해야 했다
egress 통제	없음	CiliumNetworkPolicy 29종 — FQDN egress (Gemini · chat-ocr)	동일 + VPC Endpoint	③에서 S3 가상호스트 이름 누락으로 Loki:Tempo 가 중단됐다(위 7번 시트)
서비스 간 암호화	평문	Istio PeerAuthentication STRICT (mTLS) · app ns	동일 STRICT	ambient 는 기각 — sidecar 유지
신원 · 권한				
사람 접근	공유 SSH 키 · 무제한	kubeadm admin.conf = cluster-admin (무기한 · revoke 불가) → RBAC 4티어(admin/app-dev/observability/data-dev)	IAM 그룹 → EKS Access Entry → K8s 그룹 mp:admin 권한 실체 = 정의한 ClusterRole	①②의 최대 리스크는 '권한이 넓다' 가 아니라 '취소 불가' 였다. ③에서 처음으로 IAM 으로 회수 가능해졌다
접근 3층	해당없음	SSH → admin.conf 단일 경로	① IAM 자격증명 ② eks:DescribeCluster ③ EKS Access Entry — 셋 중 하나만 빠져도 불지 않는다	세 층의 예러가 전부 '키가 잘못됐다' 로 읽혀서 진단이 어렵다(②가 없으면 kubeconfig 파일 자체가 안 생긴다)
워크로드 신원	해당없음	ServiceAccount + Role/RoleBinding	IRSA (OIDC provider) — 풀 13종 EKS Pod Identity 미채택	제한 조건을 IAM 신뢰정책 안에 직접 쓰려고 IRSA 를 골랐다
권한 분리	없음	ns 경계 = 메시 경계	IAM 그룹 3종 — mealplanning-dev(4) · -ai(1) · -dashboard(2)	mealplanning-ai 는 '이름이 곧 경계' — mp-ai-* 리소스 밖은 거부
파드 · 클러스터 하드닝				
PodSecurity	없음 (컨테이너 대부분 root)	PSA — app = restricted / data-pipeline-기타 = baseline (10 ns)	동일	data-pipeline 을 baseline 으로 둔 이유 = 오퍼레이터 생성 파드를 restricted 로 막으면 원인 찾기 어려운 실패가 난다
CIS 하드닝	없음	profiling 끄 · kubelet 권한 · 감사로그 · PSA 라이브	EKS 관리형 컨트롤플레인 + audit → CloudWatch Logs	②의 미해결 = 감사로그 보존창 13시간(portforward 폭주가 바이트의 90%)
etcd 암호화	해당없음	aescbc at-rest · 137/137 암호문 검증 키 = secrets.yml (없으면 스냅샷 복호 불가)	AWS 관리	③에서 이 책임이 사라진 것이 관리형의 실익
TLS 인증서	자체 로컬 CA (수동)	cert-manager v1.21.0 — 로컬 CA Issuer	ACM (ALB 종단) + cert-manager(내부)	외부는 관리형, 내부는 유지
위협 탐지	없음	없음	GuardDuty + CloudTrail → EventBridge → Lambda → Slack #mp-security (실도달 검증)	Alertmanager 경유 금지 — 관측 스택이 중단되면 보안 알림도 죽는 순환의존 GuardDuty→EventBridge 약 8분 지연
감사 로그	없음	K8s audit (보존 13h)	EKS audit → CloudWatch Logs (보존 30일) CloudTrail → S3 (SSE-S3 + Object Lock)	CloudTrail 은 의도적으로 SSE-S3 다 — KMS 면 키 삭제로 Object Lock 을 우회할 수 있다

2-08 Secret 관리

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
저장 위치	호스트의 .env 파일 · Ansible secrets.yml(gitignored)	전용 소스 ns 의 K8s Secret (적재는 Ansible ← secrets.yml)	AWS Secrets Manager — 스토어 mp-aws-secrets · prefix mp/prod/	Secret 은 Terraform 이 만들지 않는다 — tfstate 에 평문으로 남기 때문. 사람이 넣는다
주입 방식	compose env_file	External Secrets Operator 2.8.0 (Kubernetes provider)	ESO (동일) → 백엔드만 Secrets Manager · 인증은 IRSA	ESO 라는 껍데기를 유지해 앱 매니페스트를 한 줄도 안 고쳤다 — 이 관 비용을 가장 크게 줄인 결정
접근 통제	호스트 파일 권한	fb-secrets ns 는 admin 외 edit/view 금지	IAM 와일드카드 필수 (mp/prod/*) kms:Decrypt 불요	기본 KMS 키를 쓰면 별도 Decrypt 권한이 필요 없다
회전	수동 (사실상 안 함)	수동	수동 (Secrets Manager 회전 미채택)	사람 5명 규모에서 자동 회전은 과설계
커밋 방지	gitignore	gitignore + PR 리뷰	동일 + 레포 전수 스캔	이력 포함 전수 스캔 결과 실제 키 0건. AKIA 매치 13커밋은 전부 '커밋을 금지한다' 이라 적은 문장(오탐)

2-09 CI · CD

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비교
CI				
도구	GitHub Actions self-hosted 러너 (fb-ci)	Jenkins (호스트 C · Multibranch)	GitLab CI (EC2 1대 · VPC-B) + Jenkins 병행	GH Actions 3종은 중단돼 있다 — 러너 은퇴(07-27) · Ansible 롤 삭제 (07-31). 파일은 이관 레퍼런스일 뿐
트리거	push	GitHub 웹훅 → ci.mealpong.cloud (Cloudflare Tunnel · 이 경로만 노출)	GitLab push · MR	②의 pollSCM 1분은 단일 Pipeline 시절 서술 — Multibranch 는 웹훅 스캔
게이트	없음	pytest(카탈로그 10종) → SonarQube(측정) → docker build → Trivy CRITICAL 차단 → Harbor push	동일 게이트 (arm64 네이티브) → ECR push	MR 파이프라인 잡 38개가 전부 changes: 게이트 → docs:infra 만 바꾸는 MR 은 파이프라인이 아예 안 생긴다 → 풀백 mr-guard 신설
산출	Harbor :sha + :latest	동일 3태그	ECR arm64 3태그	릴리스 버전 태그는 릴리스 런에서만
CD				
도구	없음 — 수동 (ansible / compose up)	ArgoCD 10.2.1 (GitOps · 유일한 CD)	ArgoCD (동일) — Application 44개	CD 는 ② 에서 한 번 정하고 ③ 에서 안 바꿨다
배선	해당없음	Jenkins → config 레포 kustomize edit set image(:sha) 커밋 → ArgoCD 웹훅 → auto-sync (push→약 2초)	동일 · 오버레이만 overlays/onprem · overlays/eks 로 분기	한 PR 이 한쪽에만 닿는 것이 정상이다(C-83 덧셈 원칙) — 양쪽을 다 보고 판단해야 한다
레포 구조	단일 레포	앱 레포 + config 레포 2개 (매니페스트만)	동일 (코드 정본은 08-14 부터 GitLab)	config 레포는 직접 머지 허용 · 앱 레포는 PR 리뷰 필수
sync 정책	해당없음	4티어 — 오퍼레이터 완전자동 / 앱 auto-sync-selfHeal off / edge-policy 수동(16개)	동일	push 직후 sync patch 는 캐시된 옛 HEAD 를 본다 → annotate refresh=hard 후 sync (5분 → 20초)
백업	해당없음	Jenkins 백업 없음 (유예 확정 — 호스트 C 로컬 소재라 감수)	GitLab → S3 · 소스.이미지:etcd 7트랙	C-89 로 Jenkins 가 영구화되면서 이건 상시 리스크가 됐다(학습 목적 상 감수 · 사용자 확정)

2-10 배포 전략

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비교
방식	컨테이너 재기동 — 중단 발생	롤링 7종 + 카나리 2종 (Argo Rollouts · ADR-0001)	Blue-Green 2종 (account · recipe) + 롤링	'전 서비스 Blue-Green' 은 사실이 아니다 — 실제로는 2종. 전 서비스 전환은 확정 결정(C-27)이지 현황이 아니다
트래픽 전환	해당없음	HTTPRoute weight (Gateway API)	Service selector 교체 (activeService / previewService)	되돌림이 트래픽 전환 한 번
검증 게이트	없음	AnalysisRun (Prometheus 성공률)	동일 AnalysisRun + scaleDownDelaySeconds 300	사고 = BG 버스트가 Prometheus 를 선점해 게이트가 배포를 죽였다. 승격 게이트는 승격에 흔들리는 것에 의존은 금지한다
실측		카나리 전환 정상 · 분석 NaN 오탐 이슈	승격 1초 · 롤백 2초 · 전환 중 1,409 요청 5xx 0건	무중단이 실측으로 성립
함정		ignoreDifferences 가 미적용을 Synced 로 위장 · 구 Deployment 수동 삭제 필요	undo 는 실패 리비전으로 간다 · 리비전 번호 재배정 · GitOps 를 거스른다	롤백은 config 레포 되돌리기로 한다

2-11 오토스케일

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
파드 (HPA)	없음	account (CPU 기준)	account · recipe (min 2 / max 4 · CPU)	부하시험으로 확장 이득이 확인된 워크로드에만 건다
이벤트 기반	없음	KEDA — Kafka lag 기준 · 컨슈머 3종 scale-to-zero (min 0)	KEDA — SQS 큐 깊이 기준 · ScaledObject 3종 (min 0 / max 1)	KEDA roleArn 을 TriggerAuthentication 에 적으면 AccessDenied — keda-operator SA 에 IRSA 를 걸고 트리거엔 provider 만
노드	없음 (수동 VM 생성)	없음 (수동 노드 추가 · 램프 3→4→5)	Karpenter NodePool mp-burst (arm64 · on-demand)	노드 증 오토스케일은 ③에서 처음 생겼다
실측		account 4 replica 에서 PG 커넥션 12/100	Karpenter 노드 Running 51초 · 부하 1,500 VU 에서 노드 2→4→6	startupTaints 선언 전에는 파드 1개에 노드 3대를 과발주했다(지명 20초 < 노드 기동 45초). 위치는 EC2NodeClass 가 아니라 NodePool
HPA 무용 대상		mealplan · chat — 다운스트림 대기형이라 replica 를 늘려도 안 빨라진다	동일	부하시험 결론. 늘려서 이득 없는 것을 안 늘리는 것도 설계다
차단 요인		ResourceQuota mp-app-quota 6Gi 가 평시 84% → HPA 가 늘려도 파드 생성 0	동일 구조 · 쿼터 상향으로 해소	쿼터 초과는 Pending 이 아니라 FailedCreate(admission 거부)라 Karpenter 가 반응할 대상 자체가 없다

2-12 관측 · 알림

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
메트릭	Prometheus (VM .11 · 15d)	kube-prometheus-stack 87.20.0 · 로컬 PV · 호스트 B 고정 · metrics-server	동일 (replicas 1) Amazon Managed Prometheus 미채택	대시보드·알림 자산을 그대로 옮기는 쪽이 싸다. AZ 상실 시 시계열 약 12일치 손실은 감수(재파생 원본 없음)
로그	Loki (VM .11 · 168h)	Loki 7.1.0 SingleBinary · MiniIO 백엔드 · Alloy DaemonSet	Loki + S3 백엔드	CNP toFQDNs 에 가상호스트 S3 이름이 빠져 크래시루프 — 증상이 16시간 늦게 나온다
트레이스	Tempo (VM .11 · 168h)	Tempo 1.24.4 모놀리식 · MiniIO 백엔드	Tempo + S3	②에서 Tempo 만 무제한이라 336h 로 쌓여 OOM 사고가 났었다
대시보드	Grafana (VM .11 · :3000)	Grafana (내부 게이트웨이 .15 · https)	동일	③은 WAF IP allowlist 뒤
알림	Alertmanager → Slack	동일 · PrometheusRule 20종 · 물리계층 스크레이프 포함	동일 + GuardDuty/CloudTrail → EventBridge → Lambda → Slack #mp-security	보안 알림은 Alertmanager 를 안 거친다 — 관측 스택이 중단되면 보안 알림도 죽는 순환의존 회피
FinOps	없음	kubecost	kubecost (ns cost) + 대시보드 EC2	클러스터 밖 EC2 단독 실행은 미지원이라 인클러스터
우선순위	없음	PriorityClass 3종 (data-critical · app-normal · pipeline-low)	5티어 — + platform-critical · observability-high · mp-unclassified(globalDefault)	priority 0 파드 34 → 0. preemptionPolicy 는 불변 필드라 replace --force 필요
사각지대	전부	Application 이 Progressing 이면 Healthy 검사·알람 어디에도 안 걸린다(Alloy 204회 크래시를 16시간 못 봤다)	동일 구조 + 대시보드 생존감시 부재	조용한 시간대에는 신호 0 을 무한정 모른다(실측 3시간 52분)

2-13 백업 · DR

항목	① Docker (VM · 07.09~07.26)	② 온프레미 K8s (07.27~08.08)	③ AWS EKS (08.09~)	전환 근거 · 비고
PostgreSQL	없음	barman-cloud plugin → S3 연속 WAL(archive_timeout 302s) + 정기 base(매일 03:00 KST)	동일 + 온프레미로 크로스사이트 복제	① 단계에는 백업이 존재하지 않았다
RPO	해당없음	약 5분(전체 손실) / 약 0(파드·노드 장애)	5분 (S3 WAL 아카이브 경유 · C-40)	터널 직결이면 프로덕션 primary 에 복제 슬롯이 생겨 DR 사이트 단절이 프로덕션 디스크를 채운다 → S3 경유
RTO	해당없음	10분 미만(손상·논리오류) / 초~1분(파드·노드 = CNPG 자동 패일오버)	Warm Standby 전환 = 분 단위	등급 3 Warm Standby — 전체 스택이 실제로 돌고 트래픽만 안 받는다
Elasticsearch	없음	백업 안 함 — PG 에서 재색인(실측 1.0초)	동일	파생 데이터는 백업하지 않는다. 되살릴 수 없는 원본은 PostgreSQL 하나뿐
Redis	없음	백업 안 함 (비영속 캐시)	동일 — 비운 채로 시작	같은 원칙
Kafka	없음	백업 안 함 (드레인 · 재수집)	해당없음	RTO 2시간 유지
etcd	해당없음	스냅샷 매일 02:00 → S3 (RPO 24h · RTO 약 30분) 단일 멤버라 노드 상실 = 스냅샷 복원	AWS 관리	②의 구조적 단일 장애점. 스냅샷 주기를 줄히는 게 싸다(약 94MB · 수초)
그 외 트랙	없음	소스코드 · 릴리스 이미지 · Secret·PKI · Harbor · Jenkins config → S3 (systemd timer · 전용 IAM mp-backup)	동일 + GitLab	감사 결과 이미지 트랙 전량 실패(credential username 에 안내문구) · Secret 묶음 정지 · kubeadm PKI 백업 전무 → 롤 신설로 해소
DR 구성	없음	단일 사이트 (DR 없음)	AWS = Active / 온프레미 = Warm Standby (등급 3) 전환 2단 — ① 읽기 재개(가역) ② 쓰기 개방(편도)	온프레미는 DR 대기 + 크롤 상시 프로덕션의 이중역할(C-3) — 끄면 크롤이 전면 멈춘다

개선된 RPO/RTO 는 '최신 base + WAL 체인' 이 항상 있어야 성립한다. base 가 오래되면 복구 시 WAL 재생량이 늘어 RTO 가 다시 열화한다. 정기 base(ScheduledBackup)가 그 장치다.

심층 방어 (Defense-in-Depth) — 레이 어별 보안 오브젝트 역할 한눈 요약 (발표용). 상세는 각 Kind 시트. v1에서 '신규'였던 7-8번은 라이

#	레이어	막는 위협	K8s 오브젝트	보안 역할 (라이브 실측)	오퍼레이터·도구	상태
1	네트워크 격리	횡적 이동(lateral movement) · 유출 파드 egress	NetworkPolicy(23) · CiliumNetworkPolicy(13)	app-data-pipeline-observability 4개 ns 전부 default-deny + 화이트리스트. frontend egress = DNS-istiod 뿐(틸러도 나갈 곳 0). 외부 API 는 CNP toFQDNs 로만 열린다. 노드 CIDR 192.168.0.0/24 ingress 예외 = kubelet probe(빠뜨리면 전 파드 재시작 루프)	Cilium 1.19.6 (+Hubble relay-UI)	라이브
2	신원·권한 (RBAC)	토큰 탈취 · API 오남용	ServiceAccount · Role/RoleBinding · ClusterRole	앱은 default SA + automountServiceAccountToken=false(권한 0). ArgoCD 만 전역 쓰기. Jenkins 클러스터 권한 0(config 레포 커밋만). 사람 접근도 5명 분리(mp-users ns · Phase1)	내장 K8s · ansible k8s_team_rbac	라이브
3	파드 하드닝	컨테이너 탈출(escape) · 특권 상승(privesc)	SecurityContext · PodSecurity(PSS)	app ns = **enforce restricted**(위반 파드 배포 거부). 전 앱 파드가 runAsNonRoot-runAsUser 10001·seccomp RuntimeDefault-cap drop ALL-readOnlyRootFS-privEsc false. data-*-system 은 오퍼레이터 특권이 필요해 baseline(전역 restricted 면 인프라가 안 뜬다)	내장 admission	라이브 (10 ns 라벨 적용)
4	메시 (전송 암호화)	도청(평문 통신) · 서비스 위장	PeerAuthentication	app ns PeerAuthentication **STRICT** — 사이드카 없는 파드의 평문 호출은 거부된다. Istio native-sidecar(istio-proxy 가 initContainer). 워크로드 인증서 24h·12h 에 자동 갱신. data-pipeline-observability 는 메시 밖(평문, netpol 로 막는다)	Istio 1.30.3 + istio-cni	라이브
5	비밀·인증서	git 에 비밀 노출 · 인증서 만료	ExternalSecret/ClusterSecretStore · Certificate/Issuer	ESO 29개 ExternalSecret 이 fb-secrets ns → 각 ns 로 동기화(레포에 평문 0 · refresh 1h). cert-manager 가 Let's Encrypt prod 로 Gateway TLS 발급(90일 · 30일 전 자동 갱신)	External Secrets Operator · cert-manager	라이브
6	공급망 (이미지)	취약·위조 이미지 배포	CI 게이트 (Trivy) · imagePullSecret	Jenkins 파이프라인이 Trivy CRITICAL 게이트(--exit-code 1)로 빌드를 죽인다. Harbor 3태그(sha 불변 신원 + :X.Y.Z 릴리스 핀 + :latest). config 레포 핀은 sha 고정	Trivy · Harbor (호스트 C)	라이브
7	저장 암호화	etcd 스냅샷 탈취로 Secret 평문 노출	EncryptionConfiguration (kube-apiserver)	etcd at-rest 암호화 aescbc 적용 · 137/137 Secret 암호문 검증(평문 0). 키를 잃으면 스냅샷을 못 연다 → 주 1회 Secret-PKI 묶음으로 S3 분리 보관	kubeadm 직접 설정	라이브 (2026-08-01)
8	감사 로그	누가 무엇을 바꿨는지 모름	Audit Policy (kube-apiserver)	감사 로그 활성화 + PSA 라벨 10 ns + observability netpol. 미해결 = 보존창이 13시간뿐(원인은 특정 IP 의 portforward 폭주가 바이트의 90%)	kubeadm · CIS 하드닝	라이브·보존창 미결
9	거버넌스·정책강제	설정 드리프트 · 실수 배포	Kyverno ClusterPolicy	readOnlyRootFS 강제 · 새 ns 에 default-deny 자동 생성 · 레지스트리 Harbor 제한 · latest 차단	Kyverno	미도입 (실측: CRD-ns 없음)
10	감사 (CIS)	클러스터 설정 미스구성	kube-bench Job/CronJob	CIS 벤치마크 점검. 지적사항(profiling-kubelet 권한·감사로그·PSA)은 이미 반영했으나 **정기 스캔 잡은 없다** — 1회 스냅샷으로 처리했다	kube-bench	지적 반영 · 정기잡 없음

컨트롤플레인 자체 소유인 것은 온프레임이다 — EKS 는 AWS 가 관리한다(노드 목록에 안 보인다). 온프레임은 master x1 = 단일 장애점이며, 실측상 master 부재 3분 3초 동안 인그레스 중단은 0이었다(변경 능력만 잃는다). EKS 는 관리형 노드그룹 min2/desired2/max4 위에 Karpenter NodePool(mp-burst)이 버스트만 흡수한다 — 평소 Karpenter 노드는 0대다.

사이트	노드	역할	물리 호스트 / AZ	vCPU	RAM	IP	kubelet	런타임 / AMI	taint	비고
온프레임	k8s-master	control-plane	호스트 B (192.168.0.22)	2	6068104Ki (약 5.8Gi)	192.168.0.17	v1.34.10	containerd 2.2.6	node-role.kubernetes.io/control-plane:NoSchedule	단일 컨트롤플레인 · stacked etcd · apiserver LB 없음 (controlPlaneEndpoint=.17.6443 직결)
온프레임	k8s-worker-a1	worker	호스트 A (192.168.0.12)	6	12247544Ki (약 11.7Gi)	192.168.0.20	v1.34.10	containerd 2.2.6		PG primary · Redis primary 배치 — 급사 3회가 전부 호스트 A라 중재자는 B에 둔다
온프레임	k8s-worker-a2	worker	호스트 A (192.168.0.12)	6	11215364Ki (약 10.7Gi)	192.168.0.21	v1.34.10	containerd 2.2.6		P4(2026-07-31) 합류 — 5노드 완성
온프레임	k8s-worker-b1	worker	호스트 B (192.168.0.22)	6	11215360Ki (약 10.7Gi)	192.168.0.18	v1.34.10	containerd 2.2.6		2026-07-29 하드웨어 메모리 불량 + 디스크 데이터 오염 이력 — 램 교체 후 memtest PASS. bitrot-canary 감시 대상
온프레임	k8s-worker-b2	worker	호스트 B (192.168.0.22)	6	11215364Ki (약 10.7Gi)	192.168.0.19	v1.34.10	containerd 2.2.6		Prometheus · MinIO 고정 — 로컬 PV라 노드 결박. A 급사 시 관측 생존 경로
EKS	(컨트롤플레인)	control-plane	AWS 관리 (다중 AZ)				1.34 (eks.31)	AWS 관리		노드 목록에 나오지 않는다 — 관리 책임이 AWS 로 넘어간 것이 이관의 실익 중 하나
EKS	ip-10-10-64-103...	worker	ap-northeast-2a	4	16050128Ki (약 15.3Gi)	10.10.64.103	v1.34.9-eks-254016e	AL2023 (m7g.xlarge · arm64)		노드그룹 mp-mng-general · ON_DEMAND · 루트 60GiB gp3 암호화
EKS	ip-10-10-82-223...	worker	ap-northeast-2b	4	16050128Ki (약 15.3Gi)	10.10.82.223	v1.34.9-eks-254016e	AL2023 (m7g.xlarge · arm64)		노드그룹 mp-mng-general · 서브넷 mp-subnet-node-ap-northeast-2b
EKS	(Karpenter NodePool mp-burst)	worker (버스트)	AZ-a · AZ-b 자동	가변	가변		동일	AL2023 · arm64 · on-demand	startupTaints: node.cilium.io/agent-not-ready:NoExecute	평시 0대. startupTaints 없이는 파드 1개에 노드 3대를 과발주했다(지명 20초 < 기동 45초). 실측 Running 51초

네임스페이스 = 정책(주입 라벨-PSS-NetworkPolicy)이 붙는 경계. v1 설계도의 mp-*, ns 이름은 실물과 다르다 — 실제 ns 는 접두사 없는 app-data-pipeline-observability-argocd 다. k8s_cluster_base 롤이 앱 4개 ns 의 유일한 생산자다.

ns	용도	istio-injection	PodSecurity (enforce / audit-warn)	생성 출처	비고
app	FastAPI 11 + frontend + ranking-serving + gateway	enabled	restricted / restricted	ansible k8s_cluster_base	설계도의 mp-app-ns 아님. 메시 경계 · 카나리 대상 · tier=frontend backend 라벨
data	PG(CNPG) · ES(ECK) · Kafka(Strimzi) · Redis · PGSync	disabled	baseline / restricted	ansible k8s_cluster_base	메시 OFF(비-HTTP 스토어) · Quota 미부여(오퍼레이터 가정 보존)
pipeline	Kafka 컨슈머 4 + CronJob 17	disabled	baseline / restricted	ansible k8s_cluster_base	ResourceQuota 부여 · Job 사이드카 함정을 메시 OFF 로 구조적 회피
observability	kube-prometheus-stack · LGTM · MinIO · 내부 Gateway	disabled	baseline / restricted	ansible k8s_cluster_base	관측이 앱 자원을 고갈시키지 않도록 격리
argocd	ArgoCD (유일 CD)	disabled	baseline / restricted	helm/ansible	클러스터 전역 쓰기 → RBAC 격리
argo-rollouts	Argo Rollouts 컨트롤러 + 대시보드	disabled	baseline / restricted	ArgoCD	카나리 배포 컨트롤러(ADR-0001)
kube-system	Cilium(CNI) · CoreDNS · Alloy · node-exporter · Hubble		privileged	kubeadm/helm	코어 CNI · eBPF (ns 고정)
istio-system	istiod · Telemetry	disabled	privileged	helm	사이드카 주입 webhook · 메시 CA
metallb-system	MetalLB	disabled	privileged	helm	LoadBalancer IPAM (풀 .14-.16)
cert-manager	cert-manager (+cainjector-webhook)	disabled	baseline / restricted	helm	TLS 발급
external-secrets	External Secrets Operator	disabled	baseline / restricted	helm	비밀 동기화
cnpg-system	CloudNativePG + barman-cloud 플러그인	disabled	baseline / restricted	ArgoCD	PG 오퍼레이터
elastic-system	ECK Operator (StatefulSet)	disabled	baseline / restricted	ArgoCD	ES CR 감시→STS 생성
strimzi-system	Strimzi Cluster Operator	disabled	baseline / restricted	ArgoCD	Kafka 오퍼레이터
redis-operator-system	OT-Container-Kit redis-operator	disabled	baseline / restricted	ArgoCD	RedisReplication + Sentinel
keda	KEDA (operator-metrics-webhook)	disabled	baseline / restricted	ArgoCD	이벤트 기반 스케일 0↔N
openefs	OpenEBS LVM LocalPV CSI	disabled	privileged	helm	스토리지 프로비저너
cilium-secrets	Cilium TLS 자산		restricted	helm(cilium)	
fb-secrets	비밀 정본 ns (ESO 소스)	disabled	restricted	ansible k8s_eso	기존 실물명 — 리네임하지 않는다(참조가 깨진다)
mp-users	사람 접근 SA 전용 ns	disabled	restricted	ansible k8s_team_rbac	RBAC Phase1 (5명)
cost	Kubecost	disabled	baseline / restricted	ArgoCD	비용 가시화

클러스터 스코프(ns 없음). v1 의 TBD 가 전부 확정됐다. globalDefault 는 **아무것도 true 가 아니다** — priorityClassName 미지정 파드는 우선순위 0 이라 추출 1순위가 된다. QoS 실물은 app-data-pipeline 전부 Burstable 이고 cpu limit 은 어디에도 없다 — 티어 차등은 QoS 가 아니라 **PriorityClass + 분리 Quota**로 구현돼 있다.

이름	value	globalDefault	preemptionPolicy	대상 tier	비고
data-critical	1000000	false	PreemptLowerPriority	데이터 티어	eviction 최후 생존. PGSync 도 app 급이 아니라 이쪽을 참조
app-normal	100000	false	PreemptLowerPriority	앱	앱 기본 우선순위
pipeline-low	1000	false	PreemptLowerPriority	파이프라인	노드 압박 시 크롤러가 먼저 추출된다
system-cluster-critical	2000000000	false	PreemptLowerPriority	K8s 내장	kube-system 컴포넌트
system-node-critical	2000001000	false	PreemptLowerPriority	K8s 내장	노드 필수(CNI 등)
lvm-localpv-...-csi-controller-critical	900000000	false	PreemptLowerPriority	OpenEBS	차트 제공
lvm-localpv-...-csi-node-critical	900001000	false	PreemptLowerPriority	OpenEBS	차트 제공

ns 별 총량 상한.
v1 의 TBD 가 실
측으로 확정됐다
(부하테스트 결과
반영). **Quota 를
걸면 LimitRange
짜이 필수**다 —
requests 를 안 적
은 파드가 곧바로
거부되기 때문.

NS	이름	requests.cpu	requests.memory	limits.cpu	limits.memory	Pods	현재 사용	비고
app	mp-app-quota	6	6Gi				cpu 3090m/6 · mem 4128Mi/6Gi	배포 시점에 초과하면 **거부**된다. HPA-카나리가 같은 6Gi 예산을 나눠 쓴다
pipeline	mp-pipeline-quota	3	3Gi				cpu 50m/3 · mem 128Mi/3Gi	크롤러가 앱을 굶기지 못하게. KEDA scale-to-zero 라 평소 사용량이 낮다
data	(미부여)							의도적 미부여 — 오퍼레이터가 만드는 STS 가 quota 에 걸리면 무증상으로 안 뜬다
observability	(미부여)							위와 같은 이유

이름	NS	defaultRequest.cpu	defaultRequest.memory	default.memory	비고
mp-app-defaults	app	10m	128Mi	512Mi	Quota 짝 — requests 미기재 파드가 거부되는 것을 막는다. 없으면 BestEffort → eviction 1순위. `default` 에 **memory 만** 있고 cpu 가 없다 → 주입돼도 cpu limit 은 안 생긴다(전 파드 Burstable 의 이유)
mp-pipeline-defaults	pipeline	10m	128Mi	512Mi	app 과 동일한 값(실측). pipeline 은 limits 필수(Quota 조건)

분모는 requests(limits 아님). v1 대비 결정적 변화 — scaleTargetRef 가 Deployment 가 아니라 **Rollout** 이다(카나리 전환의 결과). HPA 는 안정(stable) ReplicaSet 만 조절하고 카나리 replica 는 Rollout 컨트롤러가 setWeight 로 계산한다.

이름	NS	scaleTargetRef	minReplicas	maxReplicas	cpu 목표(%)	현재	근거
mp-account	app	**Rollout**/mp-account	2	4	70	2 replica · cpu 0%	bcrypt 가 CPU 를 포화시킨다(k6 실측). Pooler 선행 필수 — 4 replica 에서도 PG 커넥션 12/100
mp-recipe	app	**Rollout**/mp-recipe	2	4	70	2 replica · cpu 0%	k6 Stage1 실증 — scale 1→4 에서 p95 2.7s→46ms. 병목이 account 로그인으로 옮겨간 것까지 확인
(미적용)	app	price-mealplan-chat-ranking-serving 등					mealplan-chat 은 HPA 무용 — 다운스트림(DB·외부 API) 대기가 병목이라 replica 를 늘려도 안 빨라진다. ranking-serving 은 실호출이 0이다(/health model_loaded=false + mealplan 플래그 OFF)
(미적용)	pipeline	컨슈머 4종					HPA 가 아니라 **KEDA ScaledObject** (Kafka lag) — min 0 scale-to-zero. HPA 는 0 으로 못 내린다
(미적용)	data	PG · ES · Kafka · Redis					quorum-nodeSet 수동 (상태 있는 워크로드)
(미적용)	observability	관측 워크로드					고정 replica

자발적 중단(drain-노드 정비)
만 막는다. 노드 급사는
topologySpread 담당. v1 의
mp-app-pdb(tier=backend
일괄)는 실물이 아니다 — 서
비스별로 4개만 건다.

이름	NS	minAvailable	대상 selector	허용 중단	비고
mp-account-pdb	app	1	app=account	1	카나리 대상 — drain 중에도 최소 1개 유지
mp-recipe-pdb	app	1	app=recipe	1	동일
mp-frontend-pdb	app	1	app=frontend	1	replica 2
mp-gw-public-pdb	app	1	gateway-name=mp-gw-public	1	유입 경로 — 이게 0 이 되면 전면 장애
(미적용)	app		replica 1 서비스 9종		replica 1 에 PDB minAvailable=1 을 걸면 **drain 이 영원히 막힌다**. 일부러 안 건다
(오퍼레이터 관리)	data		ES · PG · Kafka		ECK/CNPG/Strimzi 가 자체 maxUnavailable 로 관리

probe 는 kubelet 이 **파드 IP 로 직접** 호출한다(Service 미경유).
NetworkPolicy 에 노드 CIDR(192.168.0.0/24) ingress 예외가 반드시 있어야
한다 — 빠뜨리면 전 파드가 재시작 루프에 빠진다. 실제로 app-data ns 정
책에 그 예외가 들어 있다(NetworkPolicy 시트).

워크로드	httpGet	startupProbe (period/threshold)	readinessProbe	livenessProbe	비고
app 백엔드 10종 (account-recipe-price-pantry-recipebook-mealplan-notify-ocr-operations-ranking-serving)	/health (각 서비스 포트)	5s / 6 (=30s)	10s / 3	30s / 3	의존성이 중단돼도 200(degraded 도 200)
mp-chat	/health :8003	5s / 6 (=30s)	10s / 3	30s / 3	v1 은 60s 유예였으나 실물은 30s — gazetteer 로드가 그 안에 끝난다
mp-frontend	/healthz :8080	5s / 6 (=30s)	10s / 3	30s / 3	정적 서버. containerPort 8080(특권포트 회피 → NET_BIND_SERVICE 불요)
mp-video	/health :8011	(없음)	10s / 3 · initialDelay 10s	30s / 3 · initialDelay 20s	startupProbe 대신 initialDelay 를 쓴다
mp-cloudflared-app	/ready :2000	(없음)	10s / 3	30s / 3 · initialDelay 10s	틱널 커넥터
mp-gw-public / mp-gw-internal	(Istio 관리)				Gateway API 가 생성 — 직접 정의하지 않는다
Prometheus / Alertmanager / Grafana / Loki / Tempo / MinIO	차트 기본		차트 기본	차트 기본	관측 시트 참조
PG · ES · Kafka · Redis	(오퍼레이터 관리)				ES 는 health ≥ yellow 만 트래픽. CNPG 는 자체 probe

무중단 롤링의 게이트는 readinessProbe(Ready 판정)다. 카나리 주의 — ArgoCD 의 ignoreDifferences 가 미적용을 Synced 로 위장할 수 있다. 검증은 Application 상태가 아니라 **HTTPRoute 실물 backendRefs** 로 한다.

구분	필드 / 오브젝트	값 (라이브)	비고
배포전략	ADR-0001	B안 — account-recipe 만 카나리, 나머지는 RollingUpdate	전면 카나리는 6Gi quota 안에서 불가. 유입 비중이 큰 둘만 고른다
Rollout	Kind	argoproj.io/v1alpha1 Rollout (2종: mp-account · mp-recipe)	Deployment 를 대체한다. 구 Deployment 는 수동 삭제해야 했다
Rollout	strategy.canary.steps	setWeight 20 → pause 30s → analysis → setWeight 50 → pause 30s → analysis → setWeight 100	분석 통과 시 자동 승격. 실패하면 자동 롤백
Rollout	trafficRouting	plugins: argoproj-labs/gatewayAPI (httpRoute=mp-<svc>-route, ns=app)	플러그인은 **vendoring**(file:// 로드) — https 다운로드 의존 제거 + 릴리스 바이너리 Trivy CRITICAL 2 건 회피(소스 빌드)
AnalysisTemplate	account-canary-analysis · recipe-canary-analysis	Prometheus 질의로 5xx율-지연 판정	무트래픽 시 빈 벡터가 NaN 이 돼 오탐 → `(expr >= 0)` 로 방어
strategy (그 외 11종)	spec.strategy.type	RollingUpdate	
strategy	rollingUpdate.maxSurge / maxUnavailable	25% / 25% (7 분)	
strategy	spec.revisionHistoryLimit	10	rollout undo 용 구 ReplicaSet 보관
graceful	lifecycle.preStop.exec	sleep 5	EndpointSlice 제거 전파 대기 → 배포 중 5xx 방지
graceful	terminationGracePeriodSeconds	30	
QoS	requests vs limits	**전부 Burstable** — cpu limit 은 어디에도 없다	메모리만 limits 지정. 축출 보호는 QoS 가 아니라 PriorityClass + 분리 Quota 로 한다
resources	app 백엔드 requests	account 500m/256Mi · price 300m/128Mi · recipe 300m/128Mi · mealplan 150m/128Mi · 나머지 100m/128~192Mi	k6 부하테스트 결과 반영(config PR #82)
resources	app 백엔드 limits	memory 256~512Mi (cpu 없음)	
init pattern	ranking 모델 로드	MinIO 경유(블록 없음)	RWX 를 앱 코드 변경 0으로 제거
job pattern	DB 마이그레이션	Job / ArgoCD PreSync hook	init 아님(replica 마다 레이스)
anti-pattern	의존성 대기 init	미사용	앱이 이미 의존성 부재를 건딘다(재시작 수렴)
topologySpread (Deployment 11종)	labelSelector · whenUnsatisfiable	tier=backend · maxSkew 1 · **ScheduleAnyway** · hostname + zone 2축	완화 정책 — 분산이 안 되더라도 스케줄은 된다
topologySpread (Rollout 2종)	labelSelector · matchLabelKeys · whenUnsatisfiable	app=<svc> · **matchLabelKeys=[rollouts-pod-template-hash]** · **DoNotSchedule** · nodeTaintsPolicy=Honor	Deployment 와 값이 다르다. pod-template-hash 를 그대로 쓰면 카나리-안정 파드가 한 그룹으로 묶여 분산이 깨진다
SA · pull secret	serviceAccountName · imagePullSecrets	미지정(=default SA) · Deployment 예도 미지정	Harbor pull 자격은 **default SA 의 imagePullSecrets=[harbor]** 로 들어간다 — 워크로드 매니페스트에는 안 보이니 ns 를 새로 만들 때 이걸 빠뜨리면 ImagePullBackOff 가 난다
priorityClassName	spec.template.spec	app-normal (앱 전체)	PriorityClass 시트 참조
ES	워크로드 종류 · 파드 수	StatefulSet (ECK) · 3 (es-a 1 + es-b 2) · 8.19.19	Deploy 아님(quorum-PVC) · nori 분석기 bake 이미지
PG	워크로드 종류	CNPG Cluster (인스턴스 2) + Pooler (replica 2)	앱은 Pooler 를 분다 — 4 replica 에서도 커넥션 12/100

Service 는 bare 이름(accountrecipe...), 그 외 오브젝트만 mp- 접두, type LoadBalancer 는 게이트웨이 전용이며 개별 서비스는 노출하지 않는다 — 온프렘은 MetalB L2(14 공개 / .15 내부) 2개뿐, EKS 는 ALB 가 클러스터 밖에 있어 Gateway 서비스조차 ClusterIP 다. EKS 예만 있는 *-preview 는 Blue-Green 의 검증용 진입점이고, *-canary 는 온프렘 카나리 전체로 영역에 남아 있다.표기: '적용' = 해당 사이트에 존재 / '미적용' = 해당 사이트에

ns	Service	type	port : targetPort	온프렘	EKS	용도 / 비고
app	account	ClusterIP	8004 : 8004	적용	적용	계정 인증 (JWT : OAuth) — Blue-Green 대상
app	account-canary	ClusterIP	8004 : 8004	적용	적용	카나리 분기용 (온프렘 ADR-0001 잔재)
app	account-preview	ClusterIP	8004,15020	미적용	적용	Blue-Green previewService — 승격 전 검증 진입점
app	recipe	ClusterIP	8001 : 8001	적용	적용	레시피 검색 상세 (Elasticsearch) — Blue-Green 대상
app	recipe-canary	ClusterIP	8001 : 8001	적용	적용	카나리 분기용
app	recipe-preview	ClusterIP	8001,15020	미적용	적용	Blue-Green previewService
app	price	ClusterIP	8002 : 8002	적용	적용	가격 조회 최지가 (SWR 캐시)
app	chat	ClusterIP	8003 : 8003	적용	적용	챗봇 (의도분류 + 토큰링 + RAG)
app	pantry	ClusterIP	8005 : 8005	적용	적용	냉장고 채고
app	recipebook	ClusterIP	8006 : 8006	적용	적용	레시피북 : 공유
app	mealplan	ClusterIP	8007 : 8007	적용	적용	식단 계획 : 예산
app	notify	ClusterIP	8008 : 8008	적용	적용	알림 fan-out
app	ranking-serving	ClusterIP	8009 : 8009	적용	적용	레시피 랭킹 서빙 (LightGBM : CPU)
app	ocr	ClusterIP	8010 : 8010	적용	적용	영수증 OCR (Gemini)
app	video	ClusterIP	8011 : 8011	적용	적용	영상 URL 재로 추출
app	operations	ClusterIP	8011 : 8011	적용	미적용	EKS 배포 대상 아님 — 데시보드 EC2 의 compose 로 나갔다(C-84)
app	frontend	ClusterIP	80 : 8080	적용	적용	React/Vite PWA (nginx)
data	pg-rw	ClusterIP	5432	적용	적용	CNPG primary 쓰기 — PGSync 만 직접 접속(LISTEN/NOTIFY)
data	pg-ro / pg-r	ClusterIP	5432	적용	적용	읽기 전용 / 전체 read
data	pg-pooler	ClusterIP	5432 : pgbouncer	적용	적용	엠펙 전부 이 경유 — transaction 모드, account 4 replica 에서도 PG 커넥션 12/100
data	es-es-http	ClusterIP	9200	적용	적용	Elasticsearch 진입 (인증 토큰 : HTTP TLS 토큰)
data	es-es-a / es-es-b	Headless	9200	적용	적용	nodeSet 이름이 AZ 가 아니다 — es-es-b-0 이 AZ-a 에 있다(정확수 판단 주의)
data	es-es-transport	Headless	9300	적용	적용	ES 노드 간 전송
data	kafka-kafka-bootstrap	ClusterIP	9091,9092	적용	미적용	EKS 에 Kafka 없음 — S3 이벤트트 + SQS 로 대체(C-10)
data	kafka-kafka-brokers	Headless	9090,9091,8443,9092	적용	미적용	Strimzi KRaft combined 3 (RF=3)
data	mp-redis / -master / -replica	ClusterIP	6379, 9121	적용	미적용	EKS 는 ElasticCache for Valkey(클러스터 백) — Sentinel 제거(C-14)
data	mp-redis-s-hl	Headless	26379	적용	미적용	Redis Sentinel — 클라이언트가 Service 대신 이걸 본다
data	redis-pgsync	ClusterIP	6379	적용	적용	엠펙 Redis 와 통합 금지 — AOF 사고 격리 교훈
data	mp-elasticsearch-exporter	ClusterIP	9114	적용	적용	Prometheus 스케레이프
data	mp-pg-pooler-nodeport	NodePort	5432	미적용	적용	데시보드 EC2 전용 — ClusterIP 가 VPC 에 없는 주소라 NodePort 로 개방했다
data	mp-es-nodeport	NodePort	9200	미적용	적용	동일 사유
mp-ingress	mp-gw-public-istio	LB / ClusterIP	15021,80,443	적용 192.168.0.1 4	적용 (ClusterIP)	공개 진입, 온프렘=MetalB / EKS=ALB 가 밖에서 중단
observability	mp-gw-internal-istio	LoadBalancer	15021,80,443	적용 192.168.0.1 5	미적용	내부 도구 6종 (https://<이름>.mealbong.cloud)
cost	kubecost-frontend	NodePort	9090:30090	적용	적용	FinOps — 데시보드 EC2 가 조회

판정 신호 = AnalysisTemplate 의 Prometheus 질의.
Rollout 이 이 HTTPRoute 의 weight 를 직접 고쳐 트
래픽을 옮긴다. 검증은 ArgoCD Application 상태가
아니라 **HTTPRoute 실물 backendRefs** 로 해야 한
다 — ignoreDifferences 가 미적용을 Synced 로 위장
할 수 있다.

이름	NS	Gateway	path 매치	backendRef	weight	port	비고
mp-account-route	app	mp-gw-public	/api/auth · /api/users	account	**100**	8004	안정 버전
mp-account-route	app	mp-gw-public	"	account-canary	**0**	8004	평시 0. Rollout 이 setWeight 로 0→20→50→100 을 쓴다
mp-recipe-route	app	mp-gw-public	/api/recipes	recipe	**100**	8001	안정 버전
mp-recipe-route	app	mp-gw-public	"	recipe-canary	**0**	8001	동일
mp-frontend-route	app	mp-gw-public	/	frontend	1	80	SPA 폴백
mp-pantry-route	app	mp-gw-public	/api/pantry	pantry	1	8005	
mp-price-route	app	mp-gw-public	/api/prices	price	1	8002	
mp-mealplan-route	app	mp-gw-public	/api/mealplan · /api/expenses	mealplan	1	8007	
mp-chat-route	app	mp-gw-public	/api/mealplan/assistant	chat	1	8003	mealplan 보다 먼저 매치돼야 한다(더 긴 prefix)
mp-notify-route	app	mp-gw-public	/api/notifications	notify	1	8008	
mp-ocr-route	app	mp-gw-public	/api/pantry/ocr	ocr	1	8010	pantry 보다 긴 prefix
mp-recipebook-route	app	mp-gw-public	/api/recipes/book · /mine · /shared	recipebook	1	8006	recipe 보다 긴 prefix
mp-video-route	app	mp-gw-public	/api/recipes/extract	video	1	8011	
mp-https-redirect	app	mp-gw-public	/	(RequestRedirect)			80 → 443
mp-grafana-route · mp-argocd-ui-route · mp-harbor-route · mp-jenkins-route · mp-sonarqube-route · mp-minio-console-route · mp-loki-push-route	observability	mp-gw-internal	hostname 기반 (*.mealbong.cloud)	각 백엔드			내부 도구 7종 + https redirect

****envFrom.configMapRef 는 파드
가동 시점에 주입된다.****
ConfigMap 을 바꾸고 ArgoCD
sync 해도 도는 파드는 옛 값을 그
대로 쓴다 → 해당 워크로드
'rollout restart' 가 별도로 필요하
다. 체크섬 어노테이션이 없어 자
동으로 골라가지 않는다(개선 후
보).

이름	NS	키 수	주요 key	value (라이브)	비고
app-common	app	23	PGHOST	pg-pooler.data.svc	v1 의 mp-pg-rw 가 아니라 **Pooler** 다(P3 — 커넥션 폭증 방지)
app-common	app		PGPORT · PGDATABASE · PGUSER	5432 · foodbudget · fbapp	
app-common	app		ESHOST · ES_PORT · ES_USER	es-es-http.data.svc · 9200 · mp_recipe_reader	ES 인증 커짐 — 앱은 전용 읽기 계정(슈퍼유저 아님)
app-common	app		REDISHOST · REDIS_SENTINELS · REDIS_MASTER_GROUP	mp-redis-master.data.svc · mp-redis-s-0~2.mp-redis-s-hl.data.svc:26379 · mymaster	Sentinel-aware — master Service 는 노드 상실 시 갱신되지 않는다
app-common	app		KAFKA_BOOTSTRAP	kafka-kafka-bootstrap.data.svc:9092	
app-common	app		OTEL_EXPORTER_OTLP_ENDPOINT · _PROTOCOL	http://tempo.observability.svc:4317 · grpc	앱 SDK 가 Tempo 로 직접 push(Alloy 경유 아님)
app-common	app		OTEL_TRACES/METRICS/LOGS_EXPORTER	otlp / none / none	메트릭은 Prometheus 가 scrape
app-common	app		OTEL_PYTHON_EXCLUDED_URLS	/health\$,/metrics\$	probe 트레이스 노이즈 제거
app-common	app		ACCOUNT_BASE_URL · PANTRY_BASE_URL · RANKING_SERVING_URL	http://account:8004 · http://pantry:8005 · http://ranking-serving:8009	서비스 간 호출은 bare Service 이름
app-common	app		ENVIRONMENT · LOG_LEVEL	production · INFO	
chat-config	app	11	GENERATOR_BACKEND	template	LLM refine OFF — Bedrock/Gemini 코드는 있으나 운영은 템플릿 생성
chat-config	app		EXTRACTOR_BACKEND · GEMINI_MODEL	rule · gemini-3.5-flash-lite	챗 NER 은 rule 유지(실측 F1 0.990 > CRF 0.620)
chat-config	app		MONTHLY_CAP_ENABLED · RATE_LIMIT_ENABLED	false · false	유료 생성이 꺼져 있어 지금은 무해. LLM 을 켤 때 같이 켜야 한다
chat-config	app		MULTITURN-FOCUS-PANTRY-PERSIST-ACCOUNT_INTEGRATION	전부 true	기능 플래그
mealplan-config	app	3	RANKING_ML_ENABLED	false	개인화 랭킹 서빙과 앱이 끊겨 있다 — 규칙 순서만 나간다
mealplan-config	app		EVENT_PRODUCE_ENABLED · IMPRESSION_LOG_ENABLED	true · false	
mp-grafana-dashboards	app	13	dashboards	커스텀 대시보드 JSON 13장	observability 가 아니라 app ns 에 있다(sidecar 가 라벨로 수집)
mp-gw-public-params	app	1	(Istio Gateway 파라미터)		Gateway API infrastructure 참조
mp-app-tunnel-config	app	1	config.yaml	cloudflared 터널 라우팅	app.mealbong.cloud 유입

Secret 은 ESO 가 fb-secrets ns 에서 각 ns 로 동
기화한다 → **Git 에 평문 0** 라이브
ExternalSecret 29개 전부 SecretSynced. 회전 후
반영은 자동이 아니다 — env 주입이라 'rollout
restart' 가 필요하다.

이름	NS	type	공급 방식	용도	비고
mp- <서비스> -secrets (10종)	app	Opaque	ExternalSecret ← ClusterSecretStore fb-kubernetes	account-chat-mealplan-notify-ocr-operations-pantry-price-ranking-serving-recipe-recipebook-video	refresh 1h · 정본은 fb-secrets ns
harbor	app · data · pipeline · argo-rollouts	kubernetes.io/dockerconfigjson	ExternalSecret	Harbor 이미지 pull	플랫폼 ns 에 이 시크릿이 없어 카나리 컨트롤러 배포가 한 번 막혔다(rollouts-pullsecret App 신설)
mp-gw-public-tls	app	kubernetes.io/tls	cert-manager (Issuer mp-letsencrypt-prod)	외부 Gateway TLS	90일 · 만료 30일 전 자동 갱신
mp-gw-internal-tls	observability	kubernetes.io/tls	cert-manager	내부 Gateway TLS	동일
mp-letsencrypt-prod-key · -staging-key	app · observability	Opaque	cert-manager ACME 계정키	ACME 등록	staging 은 rate limit 회피용 리허설
mp-cloudflare-api-token	app · observability	Opaque	ExternalSecret	DNS-01 챌린지	
mp-app-tunnel-creds	app	Opaque	ExternalSecret	cloudflared 터널 자격증명	
mp-gcp-sa	app	Opaque	ExternalSecret	GCP 서비스 계정(Vertex/Gemini)	
mp-pg-backup-s3	data	Opaque	ExternalSecret	CNPG barman-cloud → S3	애플리케이션 mp-backup 과 **별개 IAM** — 백업 자격증명을 앱과 분리
mp-pg-onsite-minio	data	Opaque	ExternalSecret	PG 논리덤프 → 인클러스터 MinIO	7일 보존
mp-pg-replica-source	data	Opaque	ExternalSecret	복제 원본 접속	
mp-es-service-accounts	data	Opaque	ExternalSecret	ES 전용 계정(앱-pgsync-exporter)	슈퍼유저 elastic 대신 최소권한 계정으로 분리
mp-elasticsearch-exporter-auth	data	Opaque	ExternalSecret	ES exporter 인증	
mp-pgsync-secrets	data	Opaque	ExternalSecret	PGSync 접속	
mp-pipeline-secrets	pipeline	Opaque	ExternalSecret	크롤러 배치 자격증명(Bedrock-Gemini 등)	
mp-alertmanager-slack	observability	Opaque	ExternalSecret	Slack webhook 2계층	webhook_default · webhook_critical
lgtn-minio-creds · minio	observability	Opaque	**Ansible 직접 렌더**	Loki-Tempo 백엔드 자격증명	ESOArgoCD 미관리 — k8s_platform_apps 물이 유일 공급원이라 물을 지우면 불가하다
repo-food-budget-config	argocd	Opaque	ExternalSecret	config 레포 SSH 배포키	Jenkins 의 config-repo-deploy-key 와 **별개**
ClusterSecretStore fb-kubernetes	(cluster)		ESO	비밀 백엔드 = K8s provider(fb-secrets ns)	EKS 이식 시 provider 만 Secrets Manager 로 교체

원칙: 앱은 K8s API 를 안 써서 권한 0 이다.
automountServiceAccountToken=false 로 **쓰지 않는
토큰을 아예 마운트하지 않는다** — 파드가 킬렸을 때
API 서버로 가는 길을 미리 없앤 것.

이름	NS	automountToken	imagePullSecret	용도	비고
default	app	false	harbor	앱 파드 전부	v1 설계의 mp-app-sa 는 만들지 않았다 — 앱이 K8s API 를 안 쓰므로 default + 토큰 비마운트로 충분
mp-gw-public-istio	app	true		Gateway 파드	Gateway API 가 생성
mp-gw-internal-istio	observability	true		내부 Gateway 파드	동일
default	data · pipeline	false	harbor	데이터·파이프라인 워크로드	
kube-prometheus-stack-prometheus	observability	true		scrape target discovery	읽기 전용
kube-prometheus-stack-kube-state-metrics	observability	true		K8s Object 상태 조회	읽기 전용
kube-prometheus-stack-operator	observability	true		Prometheus/Alertmanager CR 관리	쓰기 필요
kube-prometheus-stack-grafana	observability	true		ConfigMap/Secret 조회	읽기 전용
alloy	kube-system	true		노드/파드 로그·디스커버리	읽기 전용
loki · tempo · minio-sa	observability	false		K8s API 미사용	권한 0
argocd-application-controller	argocd	true		GitOps 동기화	클러스터 전역 쓰기
bongsu · taehyun · geonu · jungeun · junghyun	mp-users	true		사람 접근(kubeconfig)	RBAC Phase1 — Role 시트 참조

원칙: 실제 권한이 필요한 곳은
ArgoCD(전역 쓰기)-오퍼레이터(자
기 CR)-관측(읽기)뿐이다. 2026-08-
01 사람 접근 RBAC Phase1 적용
(auth can-i 20/20 검증).
admin.conf 는 아직 살아 있다 —
Phase2 컷오버(개발 끝물)에서 회
수 예정.

주체	NS	Role / ClusterRole	Binding	권한 범위	비고
앱 파드 (default SA)	app	(권한 0)		없음	K8s API 미사용 → automountToken=false
argocd-application-controller	argocd	ClusterRole (argocd 차트)	ClusterRoleBinding	* / * / *	GitOps = 클러스터 전역 쓰기 → ns 격리로 폭발반경 제한
Jenkins (클러스터 밖)				없음	CI 는 클러스터에 접근하지 않는다. config 레포에 커밋만 한 다
오퍼레이터별 SA (12종)	*-system	ClusterRole: 자기 CR + 하위 리소스	ClusterRoleBinding	자기 CRD 관리	차트가 생성
prometheus / ksm / alloy	observability · kube-system	ClusterRole (차트)	ClusterRoleBinding	get·list·watch (읽기)	관측은 읽기 전용
mp-descheduler	(cluster)	ClusterRole mp-descheduler	ClusterRoleBinding mp-descheduler	파드 축출	노드 간 재배치
봉수	mp-users	cluster-admin (내장)	ClusterRoleBinding mp-bongsu-cluster-admin	전권	인프라 담당
태현	mp-users	cluster-admin (내장)	ClusterRoleBinding mp-taehyun-cluster-admin	전권	인프라 담당
건우	mp-users	view (전역) + edit (app · pipeline)	ClusterRoleBinding mp-geonu-view + RoleBinding mp-geonu-edit × 2	앱·파이프라인 편집 / 그 외 조회	AI 담당
정은	mp-users	view (전역) + edit (pipeline)	ClusterRoleBinding mp-jungeun-view + RoleBinding mp-jungeun-edit	파이프라인 편집 / 그 외 조회	데이터 담당
정현	mp-users	view (전역) + edit (observability)	ClusterRoleBinding mp-junghyun-view + RoleBinding mp-junghyun-edit	관측 편집 / 그 외 조회	모니터링 담당
mp-monitoring-edit	(cluster)	ClusterRole mp-monitoring-edit	(참조용)	관측 리소스 편집	역할 정의

파드를 선택하는 정책이 하나라도 생기면 **명시하지 않은 트래픽은 전부 차단**된다.
ingress/egress 스위치는 독립이다. 실측 총 23종(+ 차트-오퍼레이터 생성분). v1 설계 대비 실물은 ns 이름-selector가 다르다.

NS	name	policyTypes	대상(selector)	허용 규칙 (라이브)	비고
app	mp-frontend	Ingress, Egress	tier=frontend	in: gw-public:8080 · 노드CIDR / out: CoreDNS:53 · istiod:15012	백엔드 호출 0 — 프론트가 탈러도 나갈 곳이 없다
app	mp-backend	Ingress, Egress	tier=backend	in: gw-public · 같은 tier · 노드CIDR · prometheus:15020 / out: CoreDNS · istiod · data ns(5432-6379-26379-9200-9092) · 같은 tier · tempo:4317	data 접근은 포트 단위로만 열린다
app	mp-gw-public-egress	Egress	gateway-name=mp-gw-public	out: CoreDNS · istiod · backend:8001 · frontend:8080 · tempo:4317	게이트웨이가 나갈 곳을 한정
app	mp-operations	Ingress, Egress	app=operations	in: observability ns:8011 / out: observability:9090 · **<ACADEMY_PG_IP>/32:15432**	외부 IP egress 예외 1건 — 유일하게 CIDR 로 풀린 앱
data	mp-pg-instance	Ingress, Egress	cnpg.io/cluster=pg.podRole=instance	in: pooler · app/backend · pipeline · pgsync · 자기 클러스터 · cnpg-system · prometheus:9187 · 노드CIDR · mp-pg-on-site / out: CoreDNS	
data	mp-pg-pooler	Ingress, Egress	cnpg.io/podRole=pooler	in: app/backend:5432 · 노드CIDR / out: CoreDNS · instance:5432	앱의 실제 진입점
data	mp-es	Ingress, Egress	cluster-name=es	in: app/backend · pipeline · exporter · pgsync · 노드간 9300 · elastic-system · 노드CIDR / out: CoreDNS · 노드간 9300	
data	mp-redis	Ingress, Egress	app in (mp-redis, mp-redis-s)	in: app/backend · pipeline · 같은 ns · redis-operator-system · prometheus:9121 · 노드CIDR (6379-26379) / out: CoreDNS · 같은 ns	Sentinel 포트 26379 도 함께 연다
data	mp-pgsync-egress	Egress	app=pgsync	out: CoreDNS · pg:5432 · es:9200 · redis-pgsync:6379	
data	mp-pg-on-site	Egress	app=mp-pg-on-site	out: CoreDNS · pg:5432 · observability/minio:9000	논리덤프 → MinIO
data	mp-es-exporter · -bootstrap	Ingress, Egress	app=elasticsearch-exporter(-bootstrap)	in: prometheus:9114 / out: CoreDNS · es:9200	
pipeline	mp-pipeline-egress	Egress	전체	out: CoreDNS · kafka:9092 · pg:5432 · es:9200 · redis(6379-26379)	외부 API 는 CNP 가 따로 연다
pipeline	mp-pipeline-ingress	Ingress	전체	in: prometheus (metrics 포트)	스크레이프만 허용
observability	(8종)	Ingress/Egress		모니터링 시방서 참조	default-deny + baseline-allow + 워크로드별 허용 6종
argocd	(6종 · 차트 생성)	Ingress	argocd 컴포넌트별	차트 기본	직접 관리하지 않는다
data	kafka-network-policy-kafka 외 2 (Strimzi 생성)	Ingress	strimzi.io/*	오퍼레이터 기본	직접 관리하지 않는다

표준
NetworkPolicy 는
IP/CIDR 만 다뤄
서 CDN 뒤 외부
API 를 막을 수도,
열 수도 없다.
Cilium 이 DNS 응
답을 학습(DNS 프
록시)해 **FQDN
단위**로 허용한
다. 현재: 파드가
클러스터 DNS 를
써야 한다(직접 IP
를 밖으로 정책이
안 먹는다). 실측
13종.

NS	name	대상 파드	egress (toFQDNs) / 특수	비고
app	mp-account-egress-fqdn	app=account	kauth.kakao.com · kapi.kakao.com · oauth2.googleapis.com · openidconnect.googleapis.com	소셜 로그인. 표준 netpol 은 IP/CIDR 만 되는데 이들은 CDN 뒤라 FQDN 이 아니면 못 연다
app	mp-chat-egress-fqdn	app=chat	generativelanguage.googleapis.com	유료 API 도메인 통제
app	mp-ocr-egress-fqdn	(ns 전체)	generativelanguage.googleapis.com · aiplatform.googleapis.com · oauth2.googleapis.com	Gemini Vision
app	mp-video-egress-fqdn	app=video	aiplatform.googleapis.com · oauth2.googleapis.com · generativelanguage.googleapis.com · www.youtube.com	영상 추출
app	mp-cloudflared-egress	app=cloudflared-app	region1/2.v2.argotunnel.com · *.argotunnel.com · api.cloudflare.com	터널 아웃바운드
data	mp-pg-instance-egress	cnpg.io/cluster=pg.podRole=instance	s3.ap-northeast-2.amazonaws.com · toEntities: kube-apiserver	barman-cloud 백업 업로드
data	mp-pg-pooler-egress	cnpg.io/podRole=pooler	toEntities: kube-apiserver	
pipeline	mp-pipeline-fqdn-kurly	app=poller-kurly	kurly.com · *.kurly.com	크롤 대상 한정
pipeline	mp-pipeline-fqdn-oasis	(ns 전체)	www.oasis.co.kr	
pipeline	mp-pipeline-fqdn-recipe	(ns 전체)	www.10000recipe.com	
pipeline	mp-pipeline-fqdn-bedrock	(ns 전체)	bedrock-runtime.ap-northeast-2.amazonaws.com	리뷰 감정·요약 배치
pipeline	mp-pipeline-fqdn-gemini	app=chat-insights	generativelanguage.googleapis.com	
observability	mp-observability-infra-ingress	(ns 전체)	ingress fromEntities: host · remote-node · kube-apiserver	표준 netpol 로는 표현 못 하는 '노드-API서버' 개념을 Cilium entity 로 허용

구간별 암호화: 사용
자→GW =
HTTPS(Gateway 종
단) · app↔app =
mTLS(메시) · 노드간
= WireGuard. 갱신
주기가 셋 다 다르다
— Let's Encrypt 90
일(자동) / Istio 워크
로드 24시간(자동) /
kubeadm PKI 1년(**
수동**), 사람이 있을
수 있는 건 마지막 하
나뿐이다.

Kind	이름	NS	발급자 / 방식	대상	유효기간·갱신	비고
ClusterIssuer	fb-local-ca	(cluster)	로컬 CA (자체 서명)	내부 서비스		기존 실물명 — 리네임하지 않는다
Issuer	mp-letsencrypt-prod	app · observability	ACME (Let's Encrypt) · DNS-01(Cloudflare)	*.mealbong.cloud		실사용 발급자
Issuer	mp-letsencrypt-staging	app · observability	ACME staging	리허설		rate limit 회피용 — 설정 검증은 여기서
Certificate	mp-gw-public-cert	app	mp-letsencrypt-prod	app.mealbong.cloud (외부 Gateway)	notAfter 2026-10-27 · renewalTime 2026-09-27 (≈90일/30일 전)	spec.duration-renewBefore 를 **지정하지 않았다** — 90일/30일은 Let's Encrypt 발급 기간과 cert-manager 기본(1/3 지점) 계산 결과다. Secret mp-gw-public-tls
Certificate	mp-gw-internal-cert	observability	mp-letsencrypt-prod	*.mealbong.cloud (내부 도구 7종)	notAfter 2026-10-28 · renewalTime 2026-09-28	동일 — 명시 설정 없음. Secret mp-gw-internal-tls
Certificate	barman-cloud-dient · -server	cnpg-system	self-signed Issuer	CNPG ↔ barman 플러그인 gRPC	**duration 2160h(90일) · renewBefore 360h(15일)** — 명시 설정	플러그인 내부 통신 · 게이트웨이 인증서와 달리 여기는 값을 봐야 됐다
(Istio 내장)	워크로드 인증서	app	istiod (Citadel)	app ↔ app mTLS	**24시간 · 12시간에 자동 갱신**	cert-manager 관여 없음 — 메시가 자체 발급
(Istio 내장)	istiod CA 루트	istio-system	self-signed	메시 신뢰 루트	10년	ConfigMap 으로 각 ns 배포
(kubeadm PKI)	ca.crt · apiserver · etcd 등	(호스트)	kubeadm 자체 CA	control plane	**1년 · 수동 갱신**	만료 알림 없음 — 358일 남음(2026-08 기준). 갱신 누락 시 클러스터가 멈춘다
(Cilium)	WireGuard 노드 키	(노드)	Cilium	노드 간 전 트래픽	cert 관리 0	투명 암호화 — DB 홀 포함

메시 경계는 app ns 하나뿐이다. mTLS STRICT + netpol 이 겹쳐 있어, 시연에서 'netpol 이 허용한 경로(Hubble FORWARDED)인데도 mTLS 가 막아 wget 이 타임아웃되는' 두 증을 한 화면에서 볼 수 있다.

Kind	이름	NS	필드	값	비고
Namespace label	app	app	istio-injection	enabled	주입 = MutatingWebhook. 기존 파드는 rollout restart 해야 붙는다
(주입 방식)	Istio CNI plugin	istio-system	가로채기	CNI (initContainer 아님)	PSS restricted 가 NET_ADMIN 을 금지해서 CNI 방식이 **필수**다
(사이드카 형태)	istio-proxy	app	컨테이너 종류	**native sidecar (initContainer · restartPolicy=Always)**	실측 initContainers = [istio-validation, istio-proxy(Always)] / containers = [<앱> 1개]. K8s 1.29+ native sidecar 라 'kubect! get pod' 의 READY 1/1 에도 사이드카가 붙어 있다
PeerAuthentication	mp-app-strict-mtls	app	mtls.mode	**STRICT**	v1 의 PERMISSIVE→STRICT 계획이 완료됐다. 사이드카 없는 평문 호출은 거부된다
Telemetry	mp-mesh-tracing	istio-system	tracing	OTLP → tempo.observability:4317	메시 전역 트레이싱
Sidecar	(미적용)	app	egress.hosts		v1 의 mp-app-sidecar 는 만들지 않았다 — 워크로드 12개 규모에선 Envoy 메모리 절감 효과가 작다
AuthorizationPolicy	(미적용)				L7 인가는 DROP 결정 — netpol(L3/4) + mTLS 로 충분하다고 판단
Namespace label	data · pipeline · observability	각 ns	istio-injection	disabled	비-HTTP 스토어-배치는 메시 밖. Job 은 Envoy 가 완료를 막는 함정을 구조적으로 회피
(전제)	Cilium socketLB	kube-system	hostNamespaceOnly	true	없으면 소켓 LB 가 ClusterIP 를 파드 IP 로 이미 치환해 사이드카가 가로챌 대상이 사라진다 → mTLS 가 무증상으로 깨진다

온프레미 PVC 21개 / EKS 15개 — 자이는 Kafka 3(EKS 미채택) · MinIO(S3 로 대체) · mp-ranking-model(이미지에 빌드로 소거) · mp-recipe-crawl-state(크롤 전류), accessMode 는 전부 ReadWriteOnce 다 — RWX 는 금지다(온프레미 LocalPV 는 원리상 불가, EKS 도 정합을 위해 동일 규칙을 유지한다), 온프레미는 StorageClass 가 openebs-lvm(기본Delete) 하나뿐이라 신규 PVC 가 보호 밖이었고, EKS 는 기본을 gp3-retain(Retain)으로 뒤집었

사이트	ns	PVC	용량	StorageClass	accessMode	소비자 (워크로드)	배치 (노드 / AZ)	비고
온프레미	data	pg-1		20Gi	openebs-lvm	ReadWriteOnce	CNPG pg-1 (primary)	k8s-worker-a1
온프레미	data	pg-1-wal	10Gi	openebs-lvm	ReadWriteOnce	CNPG pg-1 WAL		k8s-worker-a1
온프레미	data	pg-2	20Gi	openebs-lvm	ReadWriteOnce	CNPG pg-2 (standby)		k8s-worker-b1
온프레미	data	pg-2-wal	10Gi	openebs-lvm	ReadWriteOnce	CNPG pg-2 WAL		k8s-worker-b1
온프레미	data	elasticsearch-data-es-es-a-0	10Gi	openebs-lvm	ReadWriteOnce	ECK es-es-a-0		k8s-worker-a1
온프레미	data	elasticsearch-data-es-es-b-0	10Gi	openebs-lvm	ReadWriteOnce	ECK es-es-b-0		k8s-worker-b2
온프레미	data	elasticsearch-data-es-es-b-1	10Gi	openebs-lvm	ReadWriteOnce	ECK es-es-b-1		k8s-worker-b1
온프레미	data	data-kafka-combined-0	20Gi	openebs-lvm	ReadWriteOnce	Strimzi KRaft combined		k8s-worker-b1
온프레미	data	data-kafka-combined-1	20Gi	openebs-lvm	ReadWriteOnce	Strimzi KRaft combined		k8s-worker-a1
온프레미	data	data-kafka-combined-2	20Gi	openebs-lvm	ReadWriteOnce	Strimzi KRaft combined		k8s-worker-b2
온프레미	observability	minio	50Gi	openebs-lvm	ReadWriteOnce	MinIO (단일 replica)		k8s-worker-b2
온프레미	observability	prometheus-...-prometheus-0	30Gi	openebs-lvm	ReadWriteOnce	Prometheus		k8s-worker-b2
온프레미	observability	alertmanager-...-alertmanager-0	2Gi	openebs-lvm	ReadWriteOnce	Alertmanager		k8s-worker-b2
온프레미	observability	storage-loki-0	10Gi	openebs-lvm	ReadWriteOnce	Loki SingleBinary		k8s-worker-b1
온프레미	observability	storage-tempo-0	10Gi	openebs-lvm	ReadWriteOnce	Tempo 모놀리식		k8s-worker-b1
온프레미	app	mp-ranking-model	1Gi	openebs-lvm	ReadWriteOnce	ranking-serving		k8s-worker-b2
온프레미	pipeline	mp-recipe-crawl-state	1Gi	openebs-lvm	ReadWriteOnce	레시피 크롤 CronJob		k8s-worker-a1
온프레미	pipeline	mp-chat-reports	1Gi	openebs-lvm	ReadWriteOnce	mp-chat-insights CronJob		k8s-worker-a1
온프레미	cost	aggregator-db-storage-...-0	64Gi	openebs-lvm	ReadWriteOnce	kubecost aggregator		k8s-worker-a2
온프레미	cost	kubecost-local-store	32Gi	openebs-lvm	ReadWriteOnce	kubecost		k8s-worker-a2
온프레미	cost	persistant-configs-...-0	1Gi	openebs-lvm	ReadWriteOnce	kubecost aggregator		k8s-worker-a2
EKS	data	pg-1	20Gi	gp3-retain	ReadWriteOnce	CNPG pg-1 (primary)		ap-northeast-2a
EKS	data	pg-1-wal	10Gi	gp3-retain	ReadWriteOnce	CNPG pg-1 WAL		ap-northeast-2a
EKS	data	pg-2	20Gi	gp3-retain	ReadWriteOnce	CNPG pg-2 (standby)		ap-northeast-2b
EKS	data	pg-2-wal	10Gi	gp3-retain	ReadWriteOnce	CNPG pg-2 WAL		ap-northeast-2b
EKS	data	elasticsearch-data-es-es-a-0	10Gi	gp3-retain	ReadWriteOnce	ECK es-es-a-0		ap-northeast-2a
EKS	data	elasticsearch-data-es-es-b-0	10Gi	gp3-retain	ReadWriteOnce	ECK es-es-b-0		ap-northeast-2a
EKS	data	elasticsearch-data-es-es-b-1	10Gi	gp3-retain	ReadWriteOnce	ECK es-es-b-1		ap-northeast-2b
EKS	observability	prometheus-...-prometheus-0	20Gi	gp3-retain	ReadWriteOnce	Prometheus (replicas 1)		ap-northeast-2b
EKS	observability	alertmanager-...-alertmanager-0	2Gi	gp3	ReadWriteOnce	Alertmanager		ap-northeast-2a
EKS	observability	storage-loki-0	10Gi	gp3	ReadWriteOnce	Loki		ap-northeast-2a
EKS	observability	storage-tempo-0	10Gi	gp3	ReadWriteOnce	Tempo		ap-northeast-2b
EKS	pipeline	mp-chat-reports	1Gi	gp3-retain	ReadWriteOnce	mp-chat-insights CronJob		ap-northeast-2b
EKS	cost	aggregator-db-storage-...-0	64Gi	gp3	ReadWriteOnce	kubecost aggregator		ap-northeast-2a
EKS	cost	kubecost-local-store	32Gi	gp3	ReadWriteOnce	kubecost		ap-northeast-2a
EKS	cost	persistant-configs-...-0	1Gi	gp3	ReadWriteOnce	kubecost aggregator		ap-northeast-2a

PV 는 PVC 와 1:1 로 동작 프로버지닝되며 이름은 pvc-<uid> 다 — 직접 만들지 않는다. 이 시트는 '회수 정책과 결박 위치' 를 본다. 온프렘은 라이브 PV 21/21 을 Retain 으로 일괄 patch 했으나(파드 재시작 0으로 무중단) 기본 StorageClass 는 Delete 로 남아 신규 PVC 가 보조 막이었다. EKS 는 기본 SC 자체를 gp3-retain 으로 두어 그 구멍을 처음부터 막았다. 결박의 성격이 다르다 — 온프렘 LocalPV 는 nodeAffinity 로 '노드' 에 묶이고(그 노드가 unschedulable 이면 어디에도 못 댄다), EBS 는 'AZ' 에 묶인다.

사이트	PV 수	용량 합계	reclaimPolicy	결박 단위	StorageClass	status	비고
온프렘	21	약 322Gi	Retain (21/21)	노드 (pv.spec.nodeAffinity)	openebs-ilm	Bound	라이브 patch 로 전환 — 파드 재시작 0 성공. 단, 기본 SC 는 Delete 라 신규 PVC 는 보조 땀(EKS 에서 해소)
온프렘 (내역)		a1 6 · a2 3 · b1 6 · b2 6					LocalPV 는 정의상 노드를 못 벗어난다 — kubecost PV 3종이 a2 에 결박돼 cordon 중 어디에도 못 댔다(실사)
EKS	15	약 230Gi	Retain 8 / Delete 7	AZ (topology.ebs.csi.aws.com/zone)	gp3-retain 8 · gp3 7	Bound	기본 SC = gp3-retain — 데이터 원본(PG · ES · Prometheus · chat-reports)은 Retain, 재파생 가능한 것(Loki · Tempo · Alertmanager · kubecost)은 Delete
EKS (내역)		AZ-a 10 · AZ-b 5					AZ 분포가 치우쳐 있다 — AZ-a 상설 시 ES 2/3 를 잃어 정확수가 깨진다(es-es-b-0 이 AZ-a 에 있기 때문)
공통			allowVolumeExpansion: true		영속 모두		확장은 가능, 축소는 불가
공통			volumeBindingMode		WaitForFirstConsumer		파드가 스케줄된 뒤에 볼륨을 만든다 — 결박 위치를 스케줄러가 정하게 한다
공통			accessMode		ReadWriteOnce		RWX 금지 — 온프렘은 원리상 불가하고, EKS 도 정합을 위해 같은 규칙을 유지한다

****RWX 금지**** — LVM LocalPV
는 노드 로컬이라
ReadWriteOnce 만 된다. 파드
가 노드에 묶이고, 그 노드가 중
단되면 같은 PVC 를 쓰는 파드
는 다른 노드에서 못 뜬다.
WaitForFirstConsumer = 파드
가 스케줄된 뒤에 볼륨을 만든
다(노드 불일치 방지). SC 이름
은 base 에 하드코딩하지 않고
overlay 로 교체한다.

이름	scope	provisioner	volumeBindingMode	allowExpansion	reclaimPolicy	소비자	EKS overlay	비고
openebs-lvm (default)	Cluster	local.csi.openebs.io	WaitForFirstConsumer	true	**Delete**	관측-데이터 티어 전 PVC	gp3	PVC 삭제 시 볼륨도 삭제
openebs-lvm-retain	Cluster	local.csi.openebs.io	WaitForFirstConsumer	true	**Retain**	(현재 소비자 없음)	gp3	PVC 를 지워도 볼륨이 남는다 — 실수 복구용

본 시스템 Job 은 대부분 Cronjob 이 만든 것이다 — 직접 선언한 Job 은 없다. 이 시트는 그 산출 Job 의 공통 규칙을 본다. 실행 전에 계수는 successfuljobsHistoryLimit 이 정한다(온프레임 S2 · EKS 26) — 온프레임이 많은 것은 Cronjob 자체가 10개 더 있기 때문이다. backoffLimit 이 오버액트하다 다르다 — 재시도가 한 것(수집 집계 0)과 비한 것(descheduler 6 · pg-dump 2)을 구분했다. Job 파드는 메시에서 제외한다(ns pipeline · data 는 istio-injection disabled) — 사이드카가 살아 있으면 Job 이 Complete 로 끝나지 않는다.

사이트	ns	Job (상징 출처)	completions	parallelism	backoffLimit	suspend	비고
공통	pipeline	mp-poller-*/mp-*/pruner / 집계 (Cronjob 산출)	1	1	0	false	재시도하지 않는다 — 다음 주기가 온 재시도다. 크롤은 실패해도 다음 실행이 따라잡는다
공통	data	mp-pg-onsite-dump-epoch> (Cronjob 산출)	1	1	2	false	유일하게 재시도를 2회 허용 — 덤프는 다음 주기가 24시간 뒤라 놓치면 하류가 빈다
공통	kube-system	mp-descheduler-epoch> (Cronjob 산출)	1	1	6	false	재시도 6회 — 축출은 PDW 때문에 일시적으로 막히는 것이 정상이라 여러 번 시도해야 한다
공통	app	mp-oc<config>canary-epoch> (Cronjob 산출)	1	1	1	false	외부 API 계약 검사 — 1회 재시도
온프레임	kube-system	mp-bitrot-canary-b1/b2-epoch> (Cronjob 산출)	1	1	0	false	온프레임 전용 — 로컬 디스크 오일 감시 EKS 는 EBS 라 해당 없음
온프레임	pipeline	mp-crawl-s3-relay-epoch> (Cronjob 산출)	1	1	0	false	실전 결함 — 밀레이 개체 키가 초 단위라 같은 초에 flush 되면 45.6% 가 무중상으로 떨어졌다(로그는 전부 success · ETag 도 통과)
공통	(전 ns)	successfuljobsHistoryLimit / failedJobsHistoryLimit					기본값 유지 — 실행 이력이 남아야 실패를 사후에 볼 수 있다. 현재 온프레임 S2 · EKS 26 게 보존
공통	(정책)	restartPolicy					OnFailure — Job 파드는 Never/OnFailure 만 허용된다
공통	(정책)	PriorityClass					pipeline-low (1000) · preemptionPolicy: Never — 수집은 지연해도 되지만 서비스를 밀어내선 불가하다
공통	(정책)	메시 제외					ns pipeline · data 는 istio-injection disabled — 사이드카가 종료되지 않으면 Job 이 영원히 Running 으로 남는다
EKS	data	mp-pgsync-bootstrap (수동 전용)	1	1	0	true	suspend: true 로 두고 사람이 켜 때만 돈다. ArgoCD 쪽으로 결면 불가하다 — stock bootstrap 이 복제 슬롯과 트리거를 drop 해 CDC 가 끌린다

온프렘 23개 - EKS 13개 — 자이 10개가 전부 수집(크롤) 계열이다. 이것이 확성 클장 C-3 의 실물이다. 온프렘은 이온 후 DR 대기 사이트 안 동시에 크롤 상시 프로덕션 이라는 이중역할을 가진다. 온프렘을 끄면 DR 놓리면 되는 게 아니라 데이터 수집이 전면 멈춘다. 한 Cronjob 이 spectimeZone Asia/Seoul 이다 — vlsn-com 의 CRON TZ 미지름 때문에 UTC 로 환산해 두었던 것을 Kds 로 오면서 KST 로 복원했다. concurrencyPolicy 는 전부 forbid — 알 실행이 안 끝났으면 건너뛰는(크롤)같은 중복 실행이 데이터를 망가뜨린다.표기: '적용' = 해당 사이트에 존재 / '미적용' = 해당 사이트에 없음

이름	CronJob	스케줄 (KST)	온도역	EKS	하는 일	비고
pipeline	mp-poller-kurly	30 3 * * *	적용	미적용	마케팅의 신선.가공 가격 수집	크롤 실행 사이트가 AWS 가 아니다 — 온프렘 전용(C-3)
pipeline	mp-poller-oasis-dawn	10 4 * * *	적용	미적용	오아시스다워 새벽 카레고라 물량	온프렘 전용
pipeline	mp-poller-oasis-moon	10 13 * * *	적용	미적용	오아시스다워 밤오 물량	온프렘 전용
pipeline	mp-poller-deal-timeline	5 15 * * *	적용	미적용	오아시스 마감세팅 (15시)	온프렘 전용 - 달은 시간이 정해져 있다
pipeline	mp-poller-deal-closecall	5 17 * * *	적용	미적용	오아시스 마감세팅 (17시)	온프렘 전용
pipeline	mp-poller-recipe	0 5 * * 0,3	적용	미적용	만계외제식재 크롤 (주 2회)	온프렘 전용
pipeline	mp-poller-recipe-review	0 6 * * 0,3	미적용	미적용	레시피 리뷰 크롤	온프렘 전용 - PG 직접 직접
pipeline	mp-crawl-s3-rtty	*/30 * * * * *	적용	미적용	kafka 도둑 → 달 쿨방 (retail deal recipe 3.0스드럼)	이온 후 신상 — 온프렘 수집 결과를 AWS 로 넘기는 다리, S3 이벤트 알림 → SQS → KEDA 전수어
kube-system	mp-bittot-canary-b1	*/30 * * * * *	적용	미적용	디스크 데이터 오동 감시 (worker-b1)	2026-07-29 b1 임기 데이터 오동 시고 후속, EKS 는 EBS 라 불필요
kube-system	mp-bittot-canary-b2	*/30 * * * * *	적용	미적용	디스크 데이터 오동 감시 (worker-b2)	동일
pipeline	mp-poller-price-matview	20 * * * * *	적용	적용	가격 matview 갱신 (매시 20분)	hotdesk 는 matview 가 없어 개시 만료 시 PG 를 직접 직접 조항한 디(opcode+5867) — 스텝워드 시고의 중복 요망이었다
pipeline	mp-poller-price-anomaly	40 4 * * * *	적용	적용	가격 이상탐지 (20일 윈도우 - 2 < 2.0 - 정속표본 20)	최저가 알림과 행사지
pipeline	mp-poller-es-recipes	30 6 * * 0,3	적용	적용	레시피 Elasticsearch 예면	PGsync CDC 와 별개의 전방 제작인 경로
pipeline	mp-deal-pruner	*/10 * * * * *	적용	적용	만료된 딜 정리	10분마다 — 달은 수명이 짧다
pipeline	mp-pantry-expire-recompute	30 5 * * 0	적용	적용	남았고 소비기한 재계산 (주 1회)	용어는 유통기한이 아니라 소비기한(2023 개월)
pipeline	mp-user-data-pruner	30 6 * * * *	적용	적용	사용자 데이터 보존기간 정리	
pipeline	mp-chat-insights	0 6 * * *	적용	적용	대화 로그 인사이트 집계	PVC mp-chat-reports 에 출력
pipeline	mp-score-review-sentiment	0 7 * * 0,3	적용	적용	리뷰 감성 분석 (CPU 오름)	
pipeline	mp-summernote-reviews	0 8 * * 0,3	적용	적용	리뷰 요약	
pipeline	mp-data-invariants	0 6 * * 1	적용	적용	데이터 불변식 점검 (주 1회 필요함)	스키마 참조 통합 자동 검사
data	mp-pg-on-site-dump	0 4 * * *	적용	적용	PostgreSQL 논리 덤프	backoffLimit 2 — 다른 것보다 재시도를 허용한다
app	mp-oci-config-canary	15 5 * * 1	적용	적용	OCI 설정 카니리 검사 (주 1회)	외부 API 계약 변화 조기 감지
kube-system	mp-deadscheduler	*/30 * * * * *	적용	적용	nfsd 죽으로 죽은 리드 분산 복구	TSC 가 ScheduleAnyway 라 강제되어 없다 — 균형은 보장이 아니라 결과다. 이 Cronjob 이 그럴 되들된다

설치-CRD 업그레이드는 각 오퍼레이터를 실
제로 쓰는 도메인 소관이다. CRD 는 클러스
터 전역이라 ns 를 자동 생성하지 않는다 —
ns 는 k8s_cluster_base 룰이 만든다.
'CloudNative PG · 'E'CK 의 Cloud 는 cloud-
native 룰 뜻한다(매니지드 서비스가 아니다).

오퍼레이터	NS	설치	관리 CR	라이브 상태	비고
Cilium 1.19.6	kube-system	ansible k8s_cilium (helm)	CiliumNetworkPolicy · IPPool	DS 5/5	CNI · eBPF · kube-proxy 대체 · VXLAN · WireGuard · Hubble
Istio 1.30.3	istio-system	helm	PeerAuthentication · Telemetry · Gateway	istiod 1/1	메시 + Gateway API 구현체
MetalLB	metallb-system	helm	IPAddressPool · L2Advertisement		풀 .14-.16 · 게이트웨이 전용 2개
cert-manager	cert-manager	helm	Issuer · ClusterIssuer · Certificate	3 파드	TLS 자동 발급·갱신
External Secrets	external-secrets	helm	ClusterSecretStore · ExternalSecret	3 파드	K8s provider(fb-secrets ns)
ArgoCD	argocd	(부트스트랩)	Application · AppProject	44 Application	GitOps CD
Argo Rollouts	argo-rollouts	ArgoCD	Rollout · AnalysisTemplate	2/2 + 대시보드	카나리(ADR-0001) · gatewayAPI 플러그인 vendoring
CloudNativePG	cnpg-system	ArgoCD	Cluster · Pooler · ScheduledBackup · ObjectStore		+ barman-cloud 플러그인 별도 파드
ECK (Elastic)	elastic-system	ArgoCD	Elasticsearch	STS 1/1	nori bake 이미지 · 인증 컴 · HTTP TLS 끄
Strimzi	strimzi-system	ArgoCD	Kafka · KafkaTopic		Kafka 4.3.0 · RF=3
redis-operator (OT-Container-Kit)	redis-operator-system	ArgoCD	RedisReplication · RedisSentinel		v0.26.0 · Sentinel 은 spec.sentinel 인라인
KEDA	keda	ArgoCD	ScaledObject	3 파드	Kafka lag 기반 0↔N
OpenEBS LVM LocalPV	openebs	helm	StorageClass · 동적 PV		CSI controller + node
Descheduler	kube-system	ArgoCD			노드 간 재배치
Kubecost	cost	ArgoCD		3 파드	비용 가시화
Kyverno			ClusterPolicy	미도입	정책 강제 — 실측상 CRD-ns 없음

흐름: GitHub push → 웹훅 → Jenkins(빌드-게이트) → **config 레포 sha 커밋** → ArgoCD → 클러스터. CI 는 Jenkins 다(GH Actions 아님). config 레포에 .github/ 를 만들지 말 것 — 돌지 않는 컨테기가 CI 정본이 될로 워린다(2026-08-02 실수 기록).

Kind / 구성	이름	source / 설정	sync 정책	비고
Application (root)	mealplanning-root	config 레포 argocd/applications/	automated (prune-selfHeal)	앱 트랙 app-of-apps
Application (root)	platform-root	config 레포 platform/argocd/	automated (selfHeal, prune off)	플랫폼 트랙 뿌리를 돌로 나눠 한쪽 실수가 다른 트랙으로 안 번진다
Application (앱 13)	mp-account · mp-recipe · mp-price · mp-chat · mp-pantry · mp-recipebook · mp-mealplan · mp-notify · mp-ocr · mp-operations · mp-video · mp-ranking-serving · mp-frontend	config 레포 services/<svc>/overlays/onprem	automated (prune off · **selfHeal off**)	selfHeal 을 끈 이유 = HPA 가 조정현 replica 를 ArgoCD 가 되돌리면 싸운다
Application (오피레이티)	cnpg-operator · cnpg-plugin-barmen-cloud · eck-operator · strimzi-operator · redis-operator · keda · rollouts · rollouts-pullsecret · descheduler · kubecost · alloy · loki · tempo	platform/	automated (prune-selfHeal)	완전 자동 — 드리프트를 즉시 되돌린다
Application (수동 sync)	app-common · gateway · gateway-internal · monitoring · pipelines · mp-policies(-data,-observability,-pipeline) · pg · pooler · es · kafka · redis · pgsync	각 경로	**automated 없음**	머지만으로는 안 나간다. 'kubecti patch application -n argocd <앱> --type merge -p '{"operation":{"sync":{"revision":"","HEAD"}}}')
Application (자동 승격)	mp-cloudflared · pipelines		automated (selfHeal)	config#105 로 2026-08-03 승격
AppProject	default · platform			프로젝트 격리
자격증명	repo-food-budget-config	ExternalSecret → argocd ns		Jenkins 의 config-repo-deploy-key 와 **완전히 별개**
CI (클러스터 밖)	Jenkins (호스트 C.10)	GitHub 웹훅 → Cloudflare Tunnel (ci.mealbong.cloud)		인바운드 0 · 클러스터 권한 0. Multibranch Pipeline
CI 게이트	pytest → SonarQube → Trivy	pytest(DB-free 7종) · Sonar(축정만 비자단) · Trivy CRITICAL(자단)		pytest 는 카탈로그 18종 중 8종만 커버 — data-pipeline 은 테스트 15개가 있는데 testtrue 플러그가 없다
배치 안정성	pipeline CronJob 17종	최근 실행 이력 실측		실패 1건 = mp-poller-kurly (2026-08-03 18:30) · 다음 회차(08-04)는 성공 = 간헐 실패. 나머지 16종 전부 성공
CD 인계 지점	config 레포 커밋	kustomize edit set image → overlays/onprem newTag=sha		'branch 'main' 일 때만. PR 빌드는 CD 스킵
이미지 태깅	3태그	<sha>(풀변 신원) + X.Y.Z(릴리스 판) + :latest(가변)		config 레포 편은 **sha** — latest 면 ArgoCD 가 변화를 감지 못 하고 롤백도 불가하다
롤백	config 레포 git revert			배포 이력을 앱 히스토리와 분리
PreSync hook	DB 마이그레이션 Job			마이그레이션 1회 실행 보장
GH Actions	(비활성)	github/workflows/ 3종		중단돼 있다 — runs-on: [self-hosted, fb-ci] 인데 러너가 은퇴(2026-07-27)를 식재(2026-07-31). 파일은 Jenkins 이관 레퍼런스일 뿐

폐기는 무료 측시가 아니라 '만료 후 첫 실행 때' 일어난다. 실제 최대 보관은 etcd 15일 · 비밀 127일 · 소스 431일이다. 타이머가 멈추면 삭제도 멈춘다 — 신선도 알림 9층(MpBackup*)이 이걸 지켜본다. AWS S3 = 오프사이트, MiniIO/OpenEBS = 운영 저장소. Standard→IA 라이프사이클 전환은 **실행하지 않는다.

대상	실행 주체/방식	저장소	주기	보존	지우는 주체·시점	복구	비고
PostgreSQL WAL	CNPG barman-cloud 플러그인	S3 mp-backup-ap2/pg/	상시(~5분)	**30일**	barman 플러그인 · 백업 실행 시	PITR	ObjectStore.spec.retentionPolicy
PostgreSQL base	ScheduledBackup mp-pg-daily	S3 mp-backup-ap2/pg/	매일 03:00 KST	**30일**	동일	PITR	자격증명은 mp-pg-backup-s3(앱과 분리)
PostgreSQL 논리 덤프	CronJob mp-pg-onsite-dump	**인클러스터 MiniIO**	매일 04:00 KST	**7일**	CronJob 자신 (mc rm --older-than 7d) · 매 실행 끝	논리 복원	온사이트 빠른 복구용
etcd 스냅샷	호스트 systemd 타이머	S3 mp-etcd-backup-ap2/etcd/	매일 02:00 KST	**14일** (로컬 3개)	백업 스크립트 자신 · 매 실행 끝	클러스터 재구축	aescbc 암호화라 **암호화 키 없이는 못 연다**
비밀 kubeadm PKI	ansible secrets_backup 롤 (systemd 타이머)	S3 mp-backup-ap2/secrets/	주 1회 일 02:30 KST	**120일** (로컬 3개)	백업 스크립트 자신 · 매 실행 끝	복호 후 복원	etcd 암호화 키 + ca.key.sa.key. AES-256 · 업로드 전 복호 왕복 검증
소스 미리	호스트 systemd 타이머	S3 mp-source-backup-ap2/source/	월 1회 1일 03:30 KST	**400일** (로컬 repo당 2개)	백업 스크립트 자신 · 매 실행 끝	레포 복원	
릴리스 이미지	Jenkins (릴리스 런)	S3 mp-image-backup-ap2/	릴리스마다	**정책 없음 = 무기한**	**없음**	docker load	미결 — 보존값을 정해야 한다
Terraform state	Terraform S3 backend	S3 mp-backup-ap2/tfstate/	상시	무기한	없음	state 복원	참금 = S3 네이티브 락파일
Elasticsearch	(미구상)					**PG 재색인**	_snapshot_slm 톨 다 비어 있다 — 스냅샷 대신 재색인 전략
Kafka	RF=3 + min.insync.replicas=2 + PV	OpenEBS LocalPV		본 토픽 **7일** / DLQ **30일** (retention.ms 실측)	Kafka 자체	재크롤·재처리	DLQ 롤 4배 길게 잡았다 — 실패 메시지를 재처리할 시간을 벌기 위해
Redis	재생성					PG-파이프라인 기준 재생성	일반 캐시는 백업하지 않는다
Harbor · JENKINS_HOME	(보류 확정)					호스트 C 재구축	2026-08-03 보류 결정
관측 설정(대시보드·규칙)	Git / ArgoCD	Git	변경 시			ArgoCD 재동기화	관측 설정 정본
Prometheus-Loki-Tempo 원본	별도 백업 안 함	PV / MiniIO		retention 적용		운영 저장소 조회	고비용·재생성 가능

이름	NS	replicas	image	ports	ConfigMap	Secret	Service	비고
kube-prometheus-stack-grafana	observability	1	grafana/grafana:13.1.1	3000	grafana-datasource - lgtm-grafana-datasources	kube-prometheus-stack-grafana	kube-prometheus-stack-grafana	사이드카 2개(x8s-sidecar:2.9.0)로 대시보드 datasource 자동 로드
kube-prometheus-stack-kube-state-metrics	observability	1	registry.k8s.io/kube-state-metrics:v2.19.1	8080, 8081			kube-prometheus-stack-kube-state-metrics	SA + ClusterRole read-only
kube-prometheus-stack-operator	observability	1	prometheus-operator:v0.92.1	10250			kube-prometheus-stack-operator	PrometheusAlertmanager CR → STS
minio	observability	1	quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z	9000, 9001	minio	minio	minio - minio-console - mp-minio-console	**Deployment** 다(설계도의 StatefulSet 아님) — 단일 replica 호스트 B 고정, 문서화된 예외
mp-gw-internal-istio	observability	1	proxyv2:1.30.3	15021, 80, 443		mp-gw-internal-tls	mp-gw-internal-istio (LB :15)	내부 도구 유입 단일점
hubble-relay - hubble-ui	kube-system	1 - 1	(cilium 1.19.6)	4245 - 8081	hubble-* - config		hubble-relay - hubble-ui	노드별 flow 집계 + 서비스 맵. 링버퍼 4095 flow ≈ 50~80초 보존

Prometheus-Alertmanager 는 매니페스트가 아니라
prometheus-operator 가 CR 을 보고 만든다 — STS
를 직접 고치면 오퍼레이터가 되돌린다.

이름	NS	replicas	mode	image	ports	ConfigMap	Secret	PVC	비고
prometheus-kube-prometheus-stack-prometheus	observability	1	single	prometheus:v3.13.1-distroless	9090	...-rulefiles-0 (44 개)	...-scrape-config	prometheus-db (30Gi)	retention 15d · 호스트 B · req 200m/1Gi · lim 2500Mi
alertmanager-kube-prometheus-stack-alertmanager	observability	1	single	alertmanagerv0.33.1	9093, 9094		mp-alertmanager-slack	alertmanager-db (2Gi)	Slack 2계층(default-critical)
loki	observability	1	single-binary	loki:3.6.8	3100, 9095	loki · loki-runtime	lgtn-minio-creds	storage-loki-0 (10Gi)	청크-인덱스 = MinIO(bucket=loki) · retention 168h
tempo	observability	1	monolithic	tempo:2.9.0	3200, 4317, 4318	tempo	lgtn-minio-creds	storage-tempo-0 (10Gi)	영구 block = MinIO · retention 168h
elastic-operator	elastic-system	1		(ECK)					ECK 오퍼레이터는 STS 다

DaemonSet→Pod (ReplicaSet 없음).
node_textfile_scrape_error 지표가 존재한다고 textfile collector 가 켜진 건 아니다 — 차트 기본값에 디렉터리가 없어서 오래 꺼져 있었다.

이름	NS	배치	image	ports	ConfigMap	ServiceAccount	host mount	비고
alloy	kube-system	전 노드 5	grafana/alloyv1.18.0	12345	alloy	alloy	/var/log/pods (readOnly)	파드 로그 → Loki. 트레이스는 앱이 Tempo 로 직접 push
node-exporter-prometheus-node-exporter	kube-system	전 노드 5	node-exporterv1.12.1	9100		node-exporter	호스트 / → /host/root	textfile collector 활성화 — '--collector.textfile.directory=/host/root/var/lib/node_exporter/textfile_collector'. 백업 신선도 지표가 여러기로 들어온다
cilium · cilium-envoy	kube-system	전 노드 5	cilium 1.19.6		cilium-config	cilium	BPF fs · cgroup	CNI 본체(관측 아님 · Hubble agent 포함)

구 NodePort 노출(Grafana :30300 · Loki :31100)은
2026-07-30 회수됐다. 내부 도구는 전부 `https://<이름>.mealbong.cloud` 로 내부 Gateway(15)를 거친다.

이름	NS	type	port	targetPort	선택 대상	용도
kube-prometheus-stack-grafana	observability	ClusterIP	80	3000	grafana Pod	grafana.mealbong.cloud
kube-prometheus-stack-prometheus	observability	ClusterIP	9090, 8080	9090, 8080	prometheus Pod	Grafana query · 카나리 분석 · operations
kube-prometheus-stack-alertmanager	observability	ClusterIP	9093, 8080	9093, 8080	alertmanager Pod	Prometheus alert
kube-prometheus-stack-kube-state-metrics	observability	ClusterIP	8080	8080	ksm Pod	메트릭
loki	observability	ClusterIP	3100, 9095	3100, 9095	loki Pod	Alloy push · Grafana query
tempo	observability	ClusterIP	3200, 4317, 4318 (+레거시 8)	동일	tempo Pod	OTLP 수신
minio · minio-console · mp-minio-console	observability	ClusterIP	9000 · 9001 · 9001	동일	minio Pod	S3 API · 콘솔
mp-gw-internal-istio	observability	**LoadBalancer**	15021, 80, 443	동일	gateway Pod	EXTERNAL-IP **192.168.0.15** (MetalLB L2)
alloy · node-exporter-...	kube-system	ClusterIP	12345 · 9100	동일	각 DS Pod	메트릭
hubble-relay · hubble-ui	kube-system	ClusterIP	80	4245 · 8081	각 Pod	flow 집계 · UI

PVC 가 인터페이스,
PV 는 OpenEBS LVM
CSI 가 동적 생성한
다. MinIO 는 **운영
저장소이지 백업이
아니다** — 오프사
이트 백업은 AWS S3
쪽이다(Backup-DR
시트).

소비자	NS	PVC	AccessMode	용량	StorageClass	PV	비고
prometheus	observability	prometheus-...-db-...-0	RWO	30Gi	openebs-lvm	동적 생성	호스트 B 고정 · retention 15d
alertmanager	observability	alertmanager-...-db-...-0	RWO	2Gi	openebs-lvm	동적 생성	silence-nflog
minio	observability	minio	RWO	50Gi	openebs-lvm	동적 생성	loki-tempo-mp-pg-onsite 버킷
loki	observability	storage-loki-0	RWO	10Gi	openebs-lvm	동적 생성	v1 '영구 PVC 없음' → 실제로는 있다(WAL-캐시)
tempo	observability	storage-tempo-0	RWO	10Gi	openebs-lvm	동적 생성	동일(WAL)

mealplanning(mp) — AWS
Cloud 설계 시방서 및 AWS 비용
(단가)

항목	내용
문서 버전	v1.0 (확정본) · 2026-08-17
문서 성격	확정 시방서. 이 문서의 모든 행은 ① 확정된 설계 결정이거나 ② 2026-08-17 실측으로 확인된 구성이다. 계획·검토중·미확정 항목은 실지 않는다.
대상 시스템	월 식비 예산 기반 밀플래닝 서비스 — 애플리케이션 12종 · 데이터 티어(PostgreSQL·Elasticsearch·Valkey) · 데이터 파이프라인(정제 3종 + 배치 10종)
문서 범위	AWS 형상 한정. 계정 1개(ap-northeast-2) 안에 구성된 리소스와 그 설계 근거만 기술한다. 온프레미 형상은 이 문서의 범위가 아니다.
아키텍처 요약	Amazon EKS(1.34, Graviton/arm64) 위에 애플리케이션·데이터 티어·관측 스택을 자체 운영한다. 외부 진입은 ALB(+WAF) 단일 엣지, 파드 신원은 IRSA, Secret 은 Secrets Manager, 배포는 GitLab CI → ECR → ArgoCD 다.
설계 원칙	① 매니지드 서비스는 운영 부담이 실제로 줄 때만 채택한다 — RDS·OpenSearch·MSK 는 미채택, ElastiCache 는 채택. ② 통제는 이름과 신원으로 건다 — 네트워크 계층(NACL)이 아니라 SG·NetworkPolicy·IAM 신뢰정책. ③ 미채택도 산출물이다 — 검토 후 쓰지 않기로 한 것은 근거와 함께 남긴다. ④ 전 리소스 IaC — Terraform(AWS) + Ansible(클러스터 부트스트랩) + Kustomize(워크로드).
상태 표기	적용 = 실제 배포·동작 중 (2026-08-17 실측 확인) 미채택 = 검토 후 쓰지 않기로 확정된 것 (사유 기재)
마스킹	AWS 계정 ID 는 <ACCOUNT_ID> 로, 공인 IP·엔드포인트 호스트명은 생략했다. 이 레포는 공개(public)다.
실측 기준	AWS API(eks-ec2·iam-s3-ecr-sqs-elasticache-elbv2-wafv2-guardduty-cloudtrail-logs-cloudwatch-pricing) 및 클러스터 kubectl 조회 — 2026-08-17

시트	답는 것
1. Docker Image	ECR 이미지 18종 — 베이스 이미지 · 아키텍처 · 태그 정책
2. DEPARTMENT_Role	역할분담과 대응하는 AWS IAM 그룹 · Kubernetes 그룹
3. Amazon EKS	클러스터 형상 — 버전 · 노드그룹 · 네트워킹 · 애드온 · 오토스케일 · 데이터 티어
4. EKS Object	네임스페이스별 워크로드 · 컨테이너 · 이미지 · 포트
5. IAM	사람 그룹/사용자 · 머신 사용자 · IRSA 룰 13종 · 서비스 룰 6종
6. VPC	VPC 2개 · 서브넷 7개 · 게이트웨이 · 엔드포인트 · 라우팅
7. NACL	미채택 — 근거와 대체 통제
8. Security Group	SG 7종의 인바운드/아웃바운드 (실측 규칙)
9. RBAC	EKS 사람 접근 — Access Entry 와 ClusterRole
10. NetworkPolicy	NetworkPolicy 26종 + CiliumNetworkPolicy 29종
11. Amazon S3	버킷 11개 — 용도 · 암호화 · 버전관리 · Object Lock · 수명주기 · 실측 용량
12. Amazon SQS	크롤 운반 큐 3종 + DLQ 3종 + Karpenter 중단 알림 큐
13. Amazon ECR	리포지토리 18개 — 태그 불변성 · 스캔 · 암호화 · 수명주기

14. Amazon Bedrock	AI 배치가 쓰는 모델과 권한 경계
15. Amazon Comprehend	미채택 — 사유
16. Amazon Security	WAF · GuardDuty · CloudTrail · 감사로그 · 보안 알림
17. DNS·인증서	Route 53 미채택 근거 · AWS 측 진입 엔드포인트 · ACM
18. 비용 산정	실측 구성 × AWS 공개 단가 — 고정비 · 사용량 과금

ECR 리포지토리 (mealplanning/)	베이스 이미지	아키텍처	태그 정책	용도	상태
mp-account-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	계정-인증 (JWT · OAuth)	적용
mp-recipe-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	레시피 검색-상세 (Elasticsearch)	적용
mp-price-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	가격 조회-최저가	적용
mp-chat-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	챗봇 (의도분류 + 템플릿)	적용
mp-pantry-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	냉장고 재고	적용
mp-recipebook-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	레시피북-공유	적용
mp-mealplan-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	식단 계획-예산	적용
mp-notify-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	알림 fan-out	적용
mp-ocr-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	영수증 OCR	적용
mp-video-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	영상 URL 재료 추출	적용
mp-ranking-serving	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	레시피 랭킹 서빙 (LightGBM)	적용
mp-frontend	node:22-bookworm-slim → nginx:1.27-alpine	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	React/Vite PWA (멀티스테이지)	적용
mp-pgsync	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	PostgreSQL → Elasticsearch CDC	적용
mp-data-pipeline	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	정제 워커-배치 (파이프라인 트랙)	적용
mp-elasticsearch-nori	docker.elastic.co/elasticsearch/elasticsearch	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	한국어 형태소 분석기(nori) 포함 ES	적용
mp-rollouts-gatewayapi-plugin	golang:1.25-alpine → busybox:1.37.0	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	Argo Rollouts Gateway API 플러그인	적용
mp-operations-service	python:3.12-slim	linux/arm64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	운영 API — EKS 배포 대상 아님 (범위 결정). 이미지는 레지스트리 정합을 위해 보관한다.	미채택(EKS)
mp-crawler-kurly	playwright/python (Chromium 포함)	linux/amd64	`<sha>` 불변 + `X.Y.Z` 릴리스-불변 + `latest` 가변	크롤 수집기 — 실행 사이트가 AWS 가 아니어서 arm64 이미지를 만들지 않는다. AWS 는 수집 결과를 S3-SQS 로 받아 정제하는 구간만 담당한다(12번 시트).	미채택(EKS)

아키텍처 정책	사이트별 단일 아키텍처를 쓴다. 멀티아지(manifest list)는 미채택 — 빌드 시간이 배로 늘고 무태그 부산물이 리포에 쌓이는데, AWS 는 Graviton 전용이라 얻는 것이 없다. AWS(EKS) = arm64 단일. GitLab CI(EC2 · arm64)가 빌드해 ECR 로 push 한다.				
태그 정책 강제	ECR imageTagMutability = IMMUTABLE_WITH_EXCLUSION, 예외 필터 = `latest` (와일드카드). `<sha>`·`X.Y.Z` 는 레지스트리가 덮어쓰기를 거부하고 `latest` 만 갱신된다 — 정책이 문서가 아니라 레지스트리에 박혀 있다.				
매니페스트 핀	Kubernetes 매니페스트는 반드시 `<sha>` 로 핀한다. `latest` 로 두면 ArgoCD 가 변경을 감지하지 못하고 되돌릴 태그도 남지 않는다.				

팀 / 역할	팀원	주요 업무	AWS IAM · Kubernetes 권한	상태
Project Leader / Infra	봉수	프로젝트 총괄 · AWS 이관 설계 · Terraform/Ansible IaC · 클러스터 운영	IAM: 머신 사용자 mp-platform 으로 Terraform apply K8s: mp:admin	적용
Infra / Platform	태현	Kubernetes 플랫폼 · 배포전략(Blue-Green) · 노드/스토리지	IAM 그룹: mealplanning-dev K8s: mp:admin	적용
Dev (App)	건우	애플리케이션 개발 · CI/CD 파이프라인 · AI 서버리스 트랙	IAM 그룹: mealplanning-dev, mealplanning-ai K8s: mp:admin	적용
Observability	정현	관측(Prometheus-Loki-Tempo-Grafana) · 알림 · FinOps 대시보드	IAM 그룹: mealplanning-dev, mealplanning-dashboard K8s: mp:admin	적용
Data	정은	데이터 파이프라인 · 수집 · PostgreSQL/Elasticsearch 스키마	IAM 그룹: mealplanning-dev, mealplanning-dashboard K8s: mp:admin	적용

IAM 그룹 (실측)	멤버 수	용도	부여 권한	상태
mealplanning-dev	4	EKS 접근 · SSM 세션 · S3/Bedrock 조회	mp-team-dev(커스텀): eks:DescribeCluster · eks:ListClusters · ssm:StartSession(대상 인스턴스 태그로 한정) · 본인 세션 한정 Terminate/Resume	적용
mealplanning-ai	1	AI 독립 트랙 (서버리스)	mp-ai-dev + 전용 정책 3종. 이름이 곧 경계 — `mp-ai-*` 리소스 밖은 거부하고, iam:CreateRole 은 권한 경계(mp-ai-boundary) 부착을 조건으로만 허용한다.	적용
mealplanning-dashboard	2	FinOps:운영 대시보드 접근	대시보드 EC2 와 그 조회 대상(비용·관측 지표)에 한정	적용

설계 근거				
Kubernetes 권한 등급 = mp:admin 단일		사람 5명이 전원 같은 팀이고 온콜을 나눠 지지 않는다. 등급을 나누면 장애 국면에서 '권한이 없어 손을 못 대는 사람'이 생기는데, 그 손실이 오조작 위험보다 크다고 판단했다. 사후 통제는 EKS 감사로그(16번 시트)가 맡는다.	K8s: ClusterRole mp-admin (Ansible 이 생성) 실체는 Terraform 이 아니라 클러스터 안에 있다	적용
Access Entry 는 IAM 그룹에서 파생		IAM 그룹 mealplanning-dev 멤버십이 EKS Access Entry 로 이어진다. 그룹에 넣는 것 자체가 클러스터 관리자 부여다 — 사람 추가는 이 한 곳에서만 일어난다.	EKS Access Entry → kubernetesGroups: mp:admin	적용

IAM Identity Center · SCP 미 채택		계정이 1개이고 사람이 5명이며 전원 같은 팀이다. 권한을 주 는 손과 상한을 정하는 손이 같으면 경계가 성립하지 않는다 — 도구만 늘고 통제는 늘지 않는다. 대신 IAM 권한 경계 + 예 산 알림 + CloudTrail 로 사후 탐지한다.		미채택
-----------------------------------	--	---	--	-----

항목	구성 (실측)	설계 근거 / 비고	상태
클러스터 형상			
Cluster name	mp-eks	Terraform var.cluster_name	적용
Kubernetes version	1.34 (platform version eks.31)	클러스터 상태 ACTIVE	적용
Region / AZ	ap-northeast-2 / ap-northeast-2a · 2b (2개)	AZ 3개는 정족수가 필요한 컴포넌트가 있을 때 의미가 있는데, 본 시스템 데이터 티어는 CloudNativePG(2 인스턴스)·ElastiCache(2 노드)라 정족수를 쓰지 않는다. 3번째 AZ 는 비용만 늘린다.	적용
VPC / Subnet	mp-vpc-service (10.10.0.0/16) / mp-subnet-node-ap-northeast-2a · 2b	6번 시트	적용
노드그룹	mp-mng-general (관리형 노드그룹 · ON_DEMAND)	관리형 노드그룹을 고정 용량으로 두고, 그 위에 Karpenter 를 버스트 전용으로 없는 2층 구조	적용
인스턴스	m7g.xlarge (Graviton · arm64 · 4 vCPU / 16 GiB) × 2대 — AZ 당 1대	Spot 미채택. Blue-Green 은 promote 후 green 이 프로덕션 그 자체가 되므로 '배포할 때만 Spot' 이 성립하지 않고, 상시 Spot 은 중단 시 재기동 비용이 절감분을 넘는다.	적용
노드 스케일링	min 2 · desired 2 · max 4 (Terraform 은 desired 변경을 무시)	정상 상태를 코드가 되돌리지 않게 desired 를 ignore 한다	적용
AMI / 루트 볼륨	AL2023_ARM_64_STANDARD / 60 GiB gp3 (암호화)	노드 루트 EBS 도 비용 산정에 편입했다(18번 시트)	적용
오토스케일 (층2)	Karpenter NodePool `mp-burst` — arm64 · on-demand · m7g.large xlarge 2xlarge · AZ 2a·2b · 상한 cpu 16 / memory 64Gi	Blue-Green 배포 시 순간적으로 파드가 2배가 된다. 그 버스트만 흡수하고 평시에는 0대다. Cluster Autoscaler 미채택 — 노드그룹 단위로만 움직여 버스트에 느리다.	적용

네트워킹			
CNI	Cilium 1.19.6 · kube-proxy 대체 · IPAM = ENI 모드	파드가 노드 서브넷의 VPC IP 를 직접 받는다. ALB 가 파드를 직접 타깃으로 잡고 (target-type ip), SG 가 파드에도 걸린다. 노드 서브넷을 /20 으로 잡은 이유이기도 하다.	적용
애드온	aws-ebs-csi-driver · coredns (2종)	vpc-cni-kube-proxy 는 설치하지 않는다 — Cilium 이 둘 다 대체한다	적용
서비스 메시	Istio 1.30.3 (sidecar 모드)	mTLS STRICT 를 app 네임스페이스에 적용한다(10번 시트)	적용
진입	Gateway API — Gateway `mp-gw-public` (namespace mp-ingress · replicas 2) 유입 = ALB → 게이트웨이 파드	진입점 네임스페이스를 분리해 앱 네트워크 정책과 진입 정책이 서로를 넘지 않게 했다	적용
엔드포인트 접근	public + private (public CIDR 0.0.0.0/0)	팀원이 각자 노트북에서 직접 kubectl 을 쓴다. 이 클러스터에는 상시 접속 가능한 경유 호스트가 없어 public 을 달으면 접근 수단 자체가 사라진다. 인가는 IAM + Access Entry 2단으로 건다.	적용

스토리지 · 상태			
스토리지	EBS CSI · StorageClass `gp3` (Delete) / `gp3-retain` (Retain · 기본값)	기본 StorageClass 를 Retain 쪽에 뒀다. PVC 를 지웠을 때 데이터가 같이 사라지는 것이 기본 동작이면 불가하다고 봤다. EFS 미채택 — RWX 를 요구하는 워크로드가 없다.	적용

데이터 티어	PostgreSQL = CloudNativePG (pg-1 · pg-2 · pgBouncer Pooler 2) Elasticsearch = ECK (노드 3) Redis 호환 = ElastiCache for Valkey (cache.t4g.micro × 2 · Multi-AZ · 자동 페일 오버 · 저장 시 암호화)	RDS-OpenSearch 미채택 — 매니지드에서 사는 것은 백업·페일오버인데 그 둘은 이미 오퍼레이터가 하고 있고, 확장 파라미터와 형태소 분석기 플러그인의 통제권을 잃는 대가가 더 크다. Valkey 만 매니지드로 간 이유는 반대다 — 자체 운영의 부담이 Sentinel 이라는 별도 합의 계층이었고, 그건 오퍼레이터 결함 우회 코드를 프로덕션에 남겼다.	적용
메시지 큐 (클러스터 내)	없음 — Kafka 미채택	수집 결과 운반은 S3 이벤트 + SQS 로 대체했다(12번 시트). 브로커 3대와 그 스토리지를 상시 띄울 만큼의 유량이 아니다.	미채택
이벤트 기반 스케일	KEDA — ScaledObject 3종 (SQS 큐 깊이 기준 · min 0 / max 1)	평시 0 파드. 큐에 메시지가 들어올 때만 뜬다.	적용
HPA	mp-account · mp-recipe (min 2 / max 4 · CPU 기준) · istiod (min 1 / max 5)	부하 시험으로 확장 이득이 확인된 워크로드에만 건다	적용
PriorityClass	data-critical / platform-critical / observability-high / app-normal / pipeline-low(선점 안 함) / mp-unclassified(기본값)	노드가 찼을 때 무엇을 먼저 살릴지 5티어로 못박고, 그 아래 기본값(mp-unclassified)을 뒤 등급 없는 파드가 생기지 않게 한다. pipeline-low 는 선점하지 않는다 — 배치가 서비스 파드를 밀어내면 불가하다.	적용

신원 · 배포 · 관측			
파드 신원	IRSA (OIDC provider) — 롤 13종	EKS Pod Identity 미채택 — 제한 조건을 IAM 신뢰정책 안에 직접 쓸 수 있어야 했다(5번 시트)	적용
사람 신원	EKS Access Entry → kubernetesGroups (mp:admin)	관리형 access policy 미채택 — 권한 실체는 정의한 ClusterRole 이다(9번 시트)	적용
비밀	External Secrets Operator → AWS Secrets Manager ClusterSecretStore `mp-aws-secrets` · prefix `mp/prod/` · 시크릿 10건	SSM Parameter Store 미채택 — 4KB 한도가 인증서·연결정보 묶음에 걸린다	적용
배포 (CD)	ArgoCD 인클러스터 — Application 44개 (전부 Synced/Healthy)	CI 는 클러스터 밖(GitLab EC2), CD 는 클러스터 안. 빌드가 중단돼도 배포 상태는 유지된다.	적용
배포 전략	Argo Rollouts Blue-Green — mp-account · mp-recipe	새 버전을 미리 띄워 검증하고 서비스만 갈아끼운다. 되돌림이 트래픽 전환 한 번이다.	적용
관측	kube-prometheus-stack + Loki + Tempo + Alloy (자체 운영)	Amazon Managed Prometheus/Grafana 미채택 — 대시보드와 알림 규칙을 이미 코드로 갖고 있고, 옮기면 그 자산을 다시 만들어야 한다.	적용
FinOps	kubecost (namespace `cost`)	네임스페이스·워크로드 단위 비용 귀속. 클러스터 밖 EC2 단독 실행은 지원되지 않아 인클러스터로 둔다.	적용
컨트롤플레인 로깅	audit → CloudWatch Logs (`/aws/eks/mp-eks/cluster` · 보존 30일)	api-authenticator-controllerManager-scheduler 는 끈다. 사후에 답해야 하는 질문은 '누가 무엇을 했나' 하나이고 그건 audit 에만 있다. 나머지는 같은 질문에 답하지 않으면서 수집 비용만 만든다. CloudTrail 은 AWS API 감사이지 Kubernetes API 감사가 아니다 — kubectl get secret 은 CloudTrail 에 남지 않는다.	적용

Namespace	Object	종류	label(app)	container	image	containerPort	상태
app	mp-account	Rollout (Blue-Green)	account	account	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-account-service:<sha>	8004	적용
app	mp-recipe	Rollout (Blue-Green)	recipe	recipe	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-recipe-service:<sha>	8001	적용
app	mp-price	Deployment	price	price	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-price-service:<sha>	8002	적용
app	mp-chat	Deployment	chat	chat	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-chat-service:<sha>	8003	적용
app	mp-pantry	Deployment	pantry	pantry	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-pantry-service:<sha>	8005	적용
app	mp-recipebook	Deployment	recipebook	recipebook	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-recipebook-service:<sha>	8006	적용
app	mp-mealplan	Deployment	mealplan	mealplan	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-mealplan-service:<sha>	8007	적용
app	mp-notify	Deployment	notify	notify	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-notify-service:<sha>	8008	적용
app	mp-ranking-serving	Deployment	ranking-serving	ranking-serving	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-ranking-serving:<sha>	8009	적용
app	mp-ocr	Deployment	ocr	ocr	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-ocr-service:<sha>	8010	적용
app	mp-video	Deployment	video	video	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-video-service:<sha>	8011	적용
app	mp-frontend	Deployment	frontend	frontend	<ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com/mealplanning/mp-frontend:<sha>	8080	적용
data	pg	CNPG Cluster (인스턴스 2)	cnpg.io/cluster=pg	postgres	오퍼레이터 산출	5432	적용
data	pg-pooler	CNPG Pooler (replicas 2)	cnpg.io/podRole=pooler	pgbouncer	오퍼레이터 산출	5432	적용
data	es	ECK Elasticsearch (노드 3)	elasticsearch.k8s.elastic.co/cluster-name=es	elasticsearch	mealplanning/mp-elasticsearch-nori:<tag>	9200	적용
data	mp-pgsync	Deployment	app=pgsync	pgsync	mealplanning/mp-pgsync:<sha>		적용
data	mp-redis-pgsync	Deployment	app=redis-pgsync	redis	redis:<tag>	6379	적용
data	mp-elasticsearch-exporter	Deployment	app=elasticsearch-exporter	exporter	prometheus-community/elasticsearch-exporter	9114	적용
data	mp-pg-onsite-dump	CronJob	app=pg-onsite-dump	dump	mealplanning/mp-data-pipeline:<sha>		적용
mp-ingress	mp-gw-public-istio	Gateway API (replicas 2)	gateway.networking.k8s.io/gateway-name=mp-gw-public	istio-proxy	istio 산출	443 / 15021 / 80	적용
pipeline	mp-retail-refiner	Deployment (KEDA · min 0 / max 1)	app=retail-refiner	retail-refiner	mealplanning/mp-data-pipeline:<sha>	9401	적용
pipeline	mp-recipe-refiner	Deployment (KEDA · min 0 / max 1)	app=recipe-refiner	recipe-refiner	mealplanning/mp-data-pipeline:<sha>	9403	적용
pipeline	mp-deal-notifier	Deployment (KEDA · min 0 / max 1)	app=deal-notifier	deal-notifier	mealplanning/mp-data-pipeline:<sha>	9402	적용
pipeline	CronJob 10종	CronJob (timeZone Asia/Seoul)		batch	mealplanning/mp-data-pipeline:<sha>		적용
cost	kubecost (aggregator · frontend · local-store · finopsagent)	StatefulSet/Deployment			kubecost 배포본	9090 / 9004 / 9006	적용

pipeline CronJob 10종 상세	mp-poller-price-matview(매시 20분) · mp-poller-price-anomaly(04:40) · mp-poller-es-recipes(일·수 06:30) · mp-deal-pruner(10분마다) · mp-user-data-pruner(04:30) · mp-pantry-expire-recompute(일 05:30) · mp-chat-insights(06:00) · mp-data-invariants(월 06:00) · mp-score-review-sentiment(일·수 07:00) · mp-summarize-reviews(일·수 08:00) 전반 VCS 규칙						
플랫폼 네임스페이스	kube-system(cilium · karpenter · ebs-csi · aws-load-balancer-controller · coredns · metrics-server · alloy) · istio-system · argocd · argo-rollouts · cert-manager · external-secrets · cnpg-system · elastic-system · keda · observability(prometheus · grafana · loki · tempo · alertmanager) · cost · mp-ingress						
명명 규칙	Service 이름은 접두사 없이 도메인 그대로(account · recipe · price ...) 두고, 그 외 오브젝트에는 mp- 를 붙인다. 서비스 이름이 곧 클러스터 DNS 이름이라 접두사가 붙으면 애플리케이션 설정에 인프라 명명 규칙이 새어 들어간다						

구분	이름	주체 / ServiceAccount	용도	권한	상태
사람 그룹					
IAM 그룹	mealplanning-dev	사람 4명	EKS 접근 · SSM 세션	mp-team-dev — eks:DescribeCluster · ListClusters · ssm:StartSession(대상 태그 한정)	적용
IAM 그룹	mealplanning-ai	사람 1명	AI 독립 트랙	mp-ai-dev + 정책 3종 — `mp-ai-*` 밖 거부 · iam:CreateRole 은 권한 경계 조건부	적용
IAM 그룹	mealplanning-dashboard	사람 2명	FinOps-운영 대시보드	대시보드 EC2 및 비용지표 조회 한정	적용
머신 사용자					
IAM 사용자	mp-platform	(머신)	Terraform apply — AWS 플랫폼 스택	플랫폼 리소스 관리 권한	적용
IAM 사용자	mp-backup	(머신)	백업 업로드 · Terraform state backend	지정 버킷 한정	적용
IAM 사용자	mp-bedrock	(머신)	Bedrock 호출 검증	Bedrock 한정	적용
IAM 사용자	mp-crawl-uploader	(머신)	수집 결과 업로드	s3:PutObject on mp-crawl-ap2 의 incoming/ 접두사	적용
IAM 사용자	mp-crawl-refiner-onprem	(머신)	클러스터 밖 정제 워커	sqs 수신 · s3:GetObject 한정	적용
파드 신원 — IRSA (신뢰정책 sub = system:serviceaccount:<ns>:<sa>)					
IRSA 롤	mp-cilium-operator	kube-system : cilium-operator	ENI 생성·부착 (IPAM ENI 모드)	ec2:CreateNetworkInterface · AttachNetworkInterface · AssignPrivateIpAddresses 등	적용
IRSA 롤	mp-ebs-csi	kube-system : ebs-csi-controller-sa	EBS 볼륨 프로비저닝·부착	AmazonEBSCSIDriverPolicy 상당	적용
IRSA 롤	mp-karpenter	kube-system : karpenter	충2 노드 프로비저닝	ec2:RunInstances · iam:PassRole(mp-eks-node) · SQS(중단 알림 큐) 수신	적용
IRSA 롤	mp-lb-controller	kube-system : aws-load-balancer-controller	ALB 타킷 등록	elasticloadbalancing:RegisterTargets 등 — TargetGroupBinding 경로로 좁힘	적용
IRSA 롤	mp-external-secrets	external-secrets : external-secrets	Secrets Manager 읽기	secretsmanager:GetSecretValue on mp/prod/*	적용
IRSA 롤	mp-pg-barman	data : pg	PostgreSQL 물리 백업 (barman-cloud → S3)	s3 read/write on mp-backup-ap2 의 지정 경로	적용
IRSA 롤	mp-pg-dump	data : mp-pg-onsite-dump	PostgreSQL 논리 덤프	s3:PutObject on mp-pg-dump-ap2	적용
IRSA 롤	mp-loki-s3	observability : loki	로그 청크 저장	s3 read/write on mp-observability-eks	적용
IRSA 롤	mp-tempo-s3	observability : tempo	트레이스 블록 저장	s3 read/write on mp-observability-eks	적용
IRSA 롤	mp-backup-probe	observability : mp-backup-probe	백업 산출물 존재·신선도 점검	s3:ListBucket · GetObject on 백업 버킷	적용
IRSA 롤	mp-crawl-refiner	pipeline : mp-retail-refiner	수집 결과 수신·정제	sqs:ReceiveMessage/DeleteMessage · s3:GetObject on mp-crawl-ap2	적용
IRSA 롤	mp-crawl-scaler	keda : keda-operator	SQS 큐 깊이 조회 (KEDA 스케일 트리거)	sqs:GetQueueAttributes	적용
IRSA 롤	mp-pipeline-bedrock	pipeline : mp-score-review-sentiment · mp-summarize-reviews	리뷰 감성분석·요약	bedrock:InvokeModel on 지정 모델 ARN	적용
서비스 롤					
서비스 롤	mp-eks-cluster	eks.amazonaws.com	EKS 컨트롤플레인	AmazonEKSClusterPolicy	적용
서비스 롤	mp-eks-node	ec2.amazonaws.com	워커 노드 (관리형 노드그룹 · Karpenter 공용)	AmazonEKSWorkerNodePolicy · AmazonEC2ContainerRegistryReadOnly · AmazonEKS_CNI_Policy	적용
서비스 롤	mp-ci-server	ec2.amazonaws.com	CI 서버 EC2	AmazonSSMManagedInstanceCore + 전송 버킷 한정	적용
서비스 롤	mp-dashboard-ec2	ec2.amazonaws.com	FinOps-운영 대시보드 EC2	비용·지표 조회 한정	적용
서비스 롤	mp-security-notifier	lambda.amazonaws.com	보안 이벤트 → Slack 전달	로그 기록 + 웹훅 시크릿 조회	적용
서비스 롤	mp-ci-job	GitLab OIDC (sts:AssumeRoleWithWebIdentity)	CI 잡이 ECR push	ECR push, 신뢰정책 sub 에 프로젝트 경로를 리터럴로 박고 ref 를 main 으로 한정 — 와일드카드 금지	적용

설계 근거	① 노드 풀에 SSM 을 붙이지 않는다 — 붙는 순간 노드 셀을 떠 사람이 그 노드의 모든 파드 Secret 을 읽게 되고, 그건 Kubernetes RBAC 을 전면 우회 한다. ② IRSA 를 택하고 Pod Identity 를 쓰지 않은 이유 — 제한 조건(어느 네임스페이스의 어느 ServiceAccount 인지)이 IAM 신뢰정책 안에 있어야 IAM 만 읽고도 경계를 알 수 있다. ③ Bedrock 풀은 모델 ARN 단위로 고정한다 — 모델을 바꾸면 배포 후 AccessDenied 로 즉시 드러난다(무증				
-------	--	--	--	--	--

mp-vpc-service :
10.10.0.0/16 (서비스 VPC · DNS support/hostnames 활성화)

Subnet 티어	Name	CIDR	입주자	Security Group	라우팅	상태
공개 (AZ-a)	mp-subnet-public-ap-northeast-2a	10.10.0.0/24	ALB ENI · NAT Gateway · 대시보드 EC2	mp-sg-alb-public / mp-sg-dashboard	mp-rt-public → IGW	적용
공개 (AZ-b)	mp-subnet-public-ap-northeast-2b	10.10.1.0/24	ALB ENI	mp-sg-alb-public	mp-rt-public → IGW	적용
노드 (AZ-a)	mp-subnet-node-ap-northeast-2a	10.10.64.0/20	EKS 노드 + 파드(ENI 모드) + Interface 엔드포인트	mp-sg-eks-node / mp-sg-vpce	mp-rt-node → NAT	적용
노드 (AZ-b)	mp-subnet-node-ap-northeast-2b	10.10.80.0/20	EKS 노드 + 파드 + Interface 엔드포인트	mp-sg-eks-node / mp-sg-vpce	mp-rt-node → NAT(AZ-a)	적용
데이터 (AZ-a)	mp-subnet-data-ap-northeast-2a	10.10.32.0/24	ElastiCache (mp-cache)	mp-sg-elasticache	mp-rt-data — 기본 경로 없음	적용
데이터 (AZ-b)	mp-subnet-data-ap-northeast-2b	10.10.33.0/24	ElastiCache 복제본	mp-sg-elasticache	mp-rt-data — 기본 경로 없음	적용

게이트웨이 / 엔드포인트	이름	위치	설명			상태
Internet Gateway	mp-igw-service	서비스 VPC	공개 서브넷의 0.0.0.0/0 대상			적용
NAT Gateway	mp-nat (EIP mp-eip-nat)	mp-subnet-public-ap-northeast-2a	1대만 둔다. 두 AZ 가 공유한다. 포기한 것 = AZ-a 단절 시 두 AZ 모두 아웃바운드 불가. 실측 처리량이 월 30.9 GB 라 이중화 비용이 그 위험의 값보다 크다고 판단했다.			적용
S3 Gateway Endpoint	mp-vpce-s3	mp-rt-node 연결	무료. ECR 이미지 레이어의 실체가 S3 라 이 경로가 없으면 이미지 pull 이 전부 NAT 데이터 처리 요금을 탄다.			적용
Interface Endpoint x 3	mp-vpce-sqs / -secretsmanager / -sts	노드 서브넷 2개 · private DNS 활성화	근거는 비용 절감이 아니라 NAT 단일 구성의 우회다 — 이 세 호출은 NAT 가 중단돼도 살아 있어야 한다. private DNS 를 켜기 때문에 애플리케이션 코드 변경이 0 이다.			적용

Route Table	경로	대상	연결 서브넷			상태
mp-rt-public	10.10.0.0/16 / 0.0.0.0/0	local / mp-igw-service	public a-b			적용
mp-rt-node	10.10.0.0/16 / 0.0.0.0/0	local / mp-nat	node a-b			적용
mp-rt-data	10.10.0.0/16 만	local	data a-b — 기본 경로를 넣지 않는 것이 이 라우팅 테이블의 목적이다			적용

mp-vpc-ci :
10.11.0.0/16 (CI
VPC · 서비스 VPC
와 피어링 없음)

Subnet 티어	Name	CIDR	입주자	Security Group	라우팅	상태
공개 (AZ-a)	mp-subnet-ci-public	10.11.0.0/24	CI 서버 EC2 (GitLab · SonarQube · Runner)	mp-sg-ci	mp-rt-ci-public → mp-igw-ci	적용
S3 Gateway Endpoint	mp-vpce-ci-s3		NAT 가 없으므로 ECR 레이어가 이 경로로 나 간다(빌드마다 수 GB)		mp-rt-ci-public	적용

설계 근거	① CIDR 선정 — 10.10/16 · 10.11/16 은 클러스터 파드/서비스 대역 및 사내 대역과 겹치지 않게 골랐 다. 겹치면 상호 접속 시 라우팅이 무증상으로 어긋 난다. ② 노드 서브넷이 /20 인 이유 — Cilium ENI 모드 라 파드가 이 서브넷의 VPC IP 를 직접 쓴다. 별도 파드 CIDR 이 없어서 좁으면 파드가 뜨지 않는다. /20 = 사용가능 4,091 vs 현재 소요 노드 2대 × max-pods 58 = 116 (약 35배 여유 — Karpenter 확 장과 Blue-Green 2배 버스트를 함께 흡수한다). ③ VPC 를 2개로 가른 이유 — CI 가 서비스 VPC 안 에 있으면 빌드 서버 침해가 곧 서비스 네트워크 침 해가 된다. 둘은 서로를 몰라도 되므로 피어링하지 않는다. ④ CI VPC 에 NAT 를 두지 않는다 — 공개 서브넷 + 공인 IP 로 직접 아웃바운드하고, 그 대가로 인바 운드 SG 규칙을 0개로 유지한다(유입은 아웃바운드 터널로만). IMDSv2 필수 · hop limit 1. ⑤ 서브넷 태그가 곧 기능이다 — kubernetes.io/role/elb(ALB) · Cilium ENI IPAM · karpenter.sh/discovery. 지우면 프로비저닝이 무증					
-------	---	--	--	--	--	--

Network ACL — 미채택			
대상	결정	근거	대체 통제
mp-vpc-service	미채택 (기본 NACL 유지)	NACL 이 Security Group 보다 나은 점은 명시적 DENY 하나인데, 본 시스템 위협 모델에 서 그 하나가 작동하지 않는다. ① 막고 싶은 것은 '어떤 파드가 어떤 파드에 못 가게' 이고, 그 경계는 서브넷이 아니다. ② 서브넷에서 인터넷으로 나가는 것은 이미 라우팅이 막는다 — 데이터 티어 라우팅 테이블에 기본 경로가 없다. ③ NACL 은 stateless 라 응답 트래픽용 고포트(1024-65535)를 열어야 하는데, 그 순간 '명시적 DENY' 의 실효가 사라진다.	① Security Group (SG 참조 방식 · 8번 시트) ② NetworkPolicy / CiliumNetworkPolicy — FQDN 단위 egress 포함 (10번 시트) ③ 라우팅 — 데이터 티어에 외부로 나가는 경로 없음 ④ Istio mTLS STRICT
mp-vpc-ci	미채택	인바운드 SG 규칙이 0개라 NACL 이 추가로 막을 것이 없다. 유입 경로는 아웃바운드 터널 하나뿐이다.	SG 인바운드 0 + IMDSv2 hop limit 1 + 옛지 IP 허용목록

VPC	Security Group	적용 대상	Inbound (실측)	Outbound	설계 의도	상태
mp-vpc-service	mp-sg-alb-public	공개 ALB (mp-alb-public) Subnet: public a-b	TCP 443 ← 0.0.0.0/0 TCP 80 ← 0.0.0.0/0	ALL → mp-sg-eks-node (SG 참조)	인터넷에 열리는 유일한 지점. TLS 는 여기서 종단한다(ACM). 80 은 443 리다이렉트 전용이다.	적용
mp-vpc-service	mp-sg-eks-node	EKS 워커 노드 + 파드 (ENI 모드) Subnet: node a-b	ALL ← self (노드·파드 상호) TCP 1025-65535 ← EKS 클러스터 SG TCP 443 ← mp-sg-alb-public TCP 15021 ← mp-sg-alb-public (헬스체크) TCP 30000-32767 ← mp-sg-dashboard (NodePort) TCP 30091 · 30092-30093 ← mp-sg-dashboard	ALL → 0.0.0.0/0	ENI 모드라 파드 IP 가 VPC 주소다. 이 SG 가 파드에도 걸린다 — ALB 가 파드를 직접 타깃(target-type ip)으로 잡을 수 있는 근거이자, 파드 수준 방화벽이 SG 로도 성립하는 이유다.	적용
mp-vpc-service	eks-cluster-sg-mp-eks-* (AWS 생성)	컨트롤플레인 ENI	TCP 443 ← mp-sg-eks-node	ALL	kubelet ↔ apiserver	적용
mp-vpc-service	mp-sg-vpce	Interface 엔드포인트 3종 Subnet: node a-b	TCP 443 ← mp-sg-eks-node TCP 443 ← mp-sg-dashboard		SQS · Secrets Manager · STS. 클라이언트를 SG 참조로 한정해 엔드포인트가 VPC 전체에 열리지 않게 한다.	적용
mp-vpc-service	mp-sg-elasticache	ElastiCache Valkey (mp-cache) Subnet: data a-b	TCP 6379 ← mp-sg-eks-node TCP 6379 ← self (복제)	TCP 6379 → self	클라이언트는 EKS 노드/파드뿐이다. 데이터 서브넷에는 밖으 로 나가는 경로 자체가 없다.	적용
mp-vpc-service	mp-sg-dashboard	FinOps-운영 대시보드 EC2 Subnet: public a	TCP 443 ← 0.0.0.0/0 TCP 80 ← 0.0.0.0/0 TCP 8011 ← mp-sg-eks-node	ALL → 0.0.0.0/0	인증은 오리진의 oauth2-proxy(Google)가 맡는다. 대시보드가 클러스터를 보는 벽은 서브넷이 아니라 'ClusterIP 는 VPC 에 없는 주소' 라는 사실이다 — NodePort 로 붙는다.	적용
mp-vpc-ci	mp-sg-ci	CI 서버 EC2 (GitLab · SonarQube · Runner)	없음 — 인바운드 규칙 0개	ALL → 0.0.0.0/0	유입은 아웃바운드 터널로만 들어온다. CI VPC 에 NAT 를 두 지 않은 것의 상책 조건이다 — 공인 IP 를 직접 쓰는 대신 열 린 포트를 0으로 유지한다.	적용

주체	Kubernetes 그룹 / ClusterRole	Resources	Verbs	설명	상태
IAM 사용자 4명 + Terraform 머신 사용자 1 (IAM 그룹 mealplanning-dev 에서 파생된 EKS Access Entry)	mp:admin / ClusterRole `mp-admin`	*	*	클러스터 전체 관리. 권한 실체는 Terraform 이 아니라 Ansible 이 생성하는 ClusterRole 이다 — Terraform 만 적용하고 클러스터 부트스트랩을 돌리지 않으면 그룹만 생기고 권한은 0 이다.	적용
읽기 전용 주체	mp:viewer / ClusterRole `mp-viewer`	대부분 리소스 (secrets 제외)	get · list · watch	감사·조회 전용 등급. ClusterRole 은 상시 준비돼 있고, 현재 바인딩된 주체는 없다.	적용
노드	system:nodes (Access Entry: mp-eks-node 를)	노드 자신에 필요한 범위		관리형 노드그룹과 Karpenter 노드가 공용한다	적용

설계 근거					
사람이 클러스터에 붙는 데 필요한 층은 3개다				① IAM 자격증명 → sts:GetCallerIdentity 통과 ② IAM 권한 eks:DescribeCluster → aws eks update-kubeconfig 통과 (없으면 kubeconfig 파일 자체가 생성되지 않는다) ③ EKS Access Entry → kubectl 통과 (없으면 인증 실패) 하나만 빠져도 여러 메시지가 원인을 가리키지 않으므로 세 층을 함께 관리한다.	적용
RBAC 은 IRSA 의 상위 통제다				`pods create` 또는 `serviceaccounts/token create` 권한이 있으면 그 ServiceAccount 의 AWS 자격증명을 얻을 수 있다. Kubernetes 권한을 넓히면 AWS 권한도 함께 넓어진다. 권한 검증 시 서브리소스까지 확인해야 이 경로가 검사된다.	적용
등급을 하나로 둔 판단				사람 5명이 전원 같은 팀이고 온콜을 분리하지 않는다. 등급을 나누면 장애 국면에서 '권한이 없어 손을 못 대는 사람' 이 생기는데, 그 손실이 오조작 위험보다 크다고 봤다. 사후 통제는 EKS audit 로그가 말한다(보존 30일).	적용

Name	Namespace	Pod selector	Policy types	Ingress (허용)	Egress (허용)	종류	상태
mp-default-deny-ingress	app	{ } (전체)	Ingress			NetworkPolicy	적용
mp-default-deny-egress	app	{ } (전체)	Egress			NetworkPolicy	적용
mp-frontend	app	tier=frontend	Ingress, Egress	게이트웨이(mp-ingress)	백엔드 서비스 · DNS	NetworkPolicy	적용
mp-backend	app	tier=backend	Ingress, Egress	frontend · 게이트웨이 · Prometheus	data(pg-pooler · es) · DNS · 서비스 간 API	NetworkPolicy	적용
mp-operations	app	app=operations	Ingress, Egress	내부 게이트웨이		NetworkPolicy	적용
mp-account-egress-fqdn	app	app=account	Egress(FQDN)		OAuth 공급자 도메인 한정	CiliumNetworkPolicy	적용
mp-ocr-egress-fqdn	app	app in (ocr, ocr-config-canary)	Egress(FQDN)		생성형 API 도메인 한정	CiliumNetworkPolicy	적용
mp-chat-egress-fqdn	app	app=chat	Egress(FQDN)		생성형 API 도메인 한정	CiliumNetworkPolicy	적용
mp-video-egress-fqdn	app	app=video	Egress(FQDN)		영상 API · 생성형 API 도메인 한정	CiliumNetworkPolicy	적용
mp-app-node-probe	app	{ }	Ingress	노드(kubelet) 프로브		CiliumNetworkPolicy	적용
mp-pg-instance	data	cnpg.io/cluster=pg	Ingress, Egress	app · pooler · 백업	S3(barman) · DNS	NetworkPolicy	적용
mp-pg-pooler	data	cnpg.io/podRole=pooler	Ingress, Egress	app · pipeline	pg 인스턴스	NetworkPolicy	적용
mp-es	data	ECK 클러스터	Ingress, Egress	app(recipe · price) · pgsync · exporter	ES 노드 간	NetworkPolicy	적용
mp-pgsync-egress	data	app=pgsync	Egress		pg · es · redis-pgsync	NetworkPolicy	적용
mp-redis-pgsync	data	app=redis-pgsync	Ingress	pgsync		NetworkPolicy	적용
mp-pg-onsite / -egress-fqdn	data	app=pg-onsite-dump	Ingress, Egress		pg · S3(덤프 업로드)	NetworkPolicy + Cilium	적용
mp-es-exporter / -bootstrap	data	app=elasticsearch-exporter	Ingress, Egress	Prometheus	es	NetworkPolicy	적용
mp-gw-public-ingress	mp-ingress	gateway-name=mp-gw-public	Ingress	ALB (SG 경유 유입)		CiliumNetworkPolicy	적용
mp-gw-public-egress	mp-ingress	gateway-name=mp-gw-public	Egress		app 백엔드	NetworkPolicy	적용
mp-default-deny-ingress / -egress	mp-ingress	{ }	Ingress, Egress			NetworkPolicy	적용
mp-pipeline-ingress / -egress	pipeline	{ }	Ingress, Egress	Prometheus	pg · S3 · SQS · DNS	NetworkPolicy	적용
mp-pipeline-fqdn-bedrock	pipeline	AI 배치	Egress(FQDN)		Bedrock 엔드포인트	CiliumNetworkPolicy	적용
mp-pipeline-fqdn-crawl-transport	pipeline	정제 워커	Egress(FQDN)		S3 · SQS 엔드포인트	CiliumNetworkPolicy	적용
mp-pipeline-fqdn-gemini	pipeline	생성형 배치	Egress(FQDN)		생성형 API 도메인	CiliumNetworkPolicy	적용
mp-observability-default-deny	observability	{ }	Ingress, Egress			NetworkPolicy	적용
mp-observability-* (스크레이프·질의·푸시 7종)	observability	prometheus / loki / tempo / 게이트웨이	Ingress, Egress	클러스터 내부 스크레이프·푸시 경로	S3(청크·블록) · 알림 발신	NetworkPolicy + Cilium	적용
mp-deny-imds	app · data · pipeline · observability · mp-ingress	{ }	Egress		링크로컬 메타데이터 주소 차단	CiliumNetworkPolicy	적용
mTLS	app	PeerAuthentication `mp-app-strict-mtls`	STRICT			Istio	적용
실측 요약	NetworkPolicy 26건 + CiliumNetworkPolicy 29건 (ArgoCD 차트 기본 제공 6건 별도) · app 네임스페이스 mTLS STRICT — 2026-08-17 확인						

설계 근거	① app - mp-ingress - observability 는 ingress/egress 양방향 default-deny 로 시작하고 필요한 경로만 되연다. '무엇을 막을까' 가 아니라 '무엇을 허용할까' 로 뒤집어야 빠뜨림이 드러난다. ② 외부 API 로 나가는 트래픽은 IP 가 아니라 FQDN 으로 건다 — 대상이 CDN 뒤에 있어 IP 가 수시로 바뀐다. Cilium 이 DNS 응답을 보고 정책을 적용한다. ③ mp-deny-imds 를 전 네임스페이스에 건다 — 파드가 인스턴스 메타데이터로 노드 를 자격증명을 훔치는 경로를 끊는다. IRSA 로 신원을 나눠 놓아도 이 구멍이 열려 있으면 무의미하다. ④ NetworkPolicy 의 podSelector 는 같은 네임스페이스만 본다 — 네임스페이스를 넘는 허용은						
-------	--	--	--	--	--	--	--

Region	Bucket	Public Access	암호화	버전관리	Object Lock	수명주기 규칙	실측 용량(GB)	용도	상태
ap-northeast-2	mp-backup-ap2	4중 전부 차단	SSE-S3	미사용		backstop-expire-barman-eks	5.38	PostgreSQL 물리 백업(barman-cloud) · Terraform state backend(S3 네이티브 잠금)	적용
ap-northeast-2	mp-observability-eks	4중 전부 차단	SSE-S3	미사용		backstop-expire-orphans	0.07	Loki 로그 청크 · Tempo 트레이스 블록 · ALB 액세스 로그. 보존은 Loki/Tempo 가 관리하고 수명주기는 고아 객체 백스톱이다.	적용
ap-northeast-2	mp-crawl-ap2	4중 전부 차단	SSE-S3	미사용		incoming-expire · failed-expire · abort-incomplete-multipart	0.00	수집 결과 운반 — 업로드 → S3 이벤트 → SQS → 정제 워커	적용
ap-northeast-2	mp-pg-dump-ap2	4중 전부 차단	SSE-S3	미사용		c79-tier-and-expire	0.04	PostgreSQL 논리 덤프. 물리 백업 버킷과 분리해 장애 도메인을 가른다.	적용
ap-northeast-2	mp-cloudtrail-ap2	4중 전부 차단	SSE-S3	활성	COMPLIANCE 90일	audit-tiering	0.00	CloudTrail 감사 로그	적용
ap-northeast-2	mp-source-backup-ap2	4중 전부 차단	SSE-S3	미사용		source-tier-and-expire · gitlab-expire · abort-incomplete-multipart	0.17	소스 저장소 백업 (오프사이트)	적용
ap-northeast-2	mp-etcd-backup-ap2	4중 전부 차단	SSE-S3	미사용		없음	0.39	클러스터 상태 스냅샷 보관	적용
ap-northeast-2	mp-image-backup-ap2	4중 전부 차단	SSE-S3	미사용		없음	0.03	컨테이너 이미지 백업	적용
ap-northeast-2	mp-ssm-transfer-ap2	4중 전부 차단	SSE-S3	미사용		expire-transfer-objects	0.00	SSM Session Manager 경유 파일 전송 — SSH 키페어를 쓰지 않기 위한 경로	적용
ap-northeast-2	mp-ai-model-ap2	4중 전부 차단	SSE-S3	활성		없음	0.00	AI 트랙 모델 아티팩트	적용
ap-northeast-2	mp-dashboard-tfstate-ap2	4중 전부 차단	SSE-S3	활성		없음	0.00	대시보드 스택 Terraform state (플랫폼 state 와 분리)	적용
	합계 11개 버킷						6.08		

설계 근거	① 전 버킷 공통 — Block Public Access 4중 전부 활성 · ACL 미사용(버킷 소유자 강제) · SSE-S3 기본 암호화. ② CloudTrail 버킷만 Object Lock COMPLIANCE 를 건다 — 계정을 나눌 수 없는 조건에서 '관리자도 감사 로그를 지울 수 없다' 를 사는 유일한 방법이다. 이 버킷을 KMS 가 아니라 SSE-S3 로 둔 것도 같은 이유다 — KMS 키를 지울 수 있으면 Object Lock 을 우회할 수 있다. ③ 수명주기의 판단 측은 절감이 아니다 — 현재 총량이 6 GB 라 절감액은 무의미하다. '무엇을 언제까지 보관하고 언제 지우는가' 를 정책으로 못박아 두는 것이 목적이다. ④ AWS Backup 미체택 — 애플리케이션 백업(barman · 저장소 미라)은 정합성 시점이 정확하고, 인프라 스냅샷은 거칠다. 레이어가 달라 대체 관계가 아							
-------	---	--	--	--	--	--	--	--

Queue	종류	생산자	소비자	용도 / 비고	상태
mp-crawl-retail	표준	S3 이벤트 (mp-crawl-ap2)	pipeline : mp-retail-refiner	소매 가격 수집 결과 정제	적용
mp-crawl-recipe	표준	S3 이벤트 (mp-crawl-ap2)	pipeline : mp-recipe-refiner	레시피 수집 결과 정제	적용
mp-crawl-deal	표준	S3 이벤트 (mp-crawl-ap2)	pipeline : mp-deal-notifier	특가 감지·알림	적용
mp-crawl-retail-dlq	DLQ	mp-crawl-retail 재시도 초과	(수동 조사)	실패 격리	적용
mp-crawl-recipe-dlq	DLQ	mp-crawl-recipe 재시도 초과	(수동 조사)	실패 격리	적용
mp-crawl-deal-dlq	DLQ	mp-crawl-deal 재시도 초과	(수동 조사)	실패 격리	적용
mp-karpenter-interruption	표준	EventBridge (EC2 중단·상태 변경)	kube-system : karpenter	노드 회수 사전 감지 → 파드 사전 이동	적용

설계 근거	① Kafka 를 건너내고 S3 + SQS 로 간 이유 — 필요한 것은 '수집 결과 한 덩어리를 잃지 않고 한 번 처리'였다. 브로커 3대와 그 스토리지·운영을 상시 지고 갈 유량이 아니다. 객체 저장소가 원본을 갖고 큐는 포인트만 나른다. ② 큐를 3개로 가른 이유 — 정제 로직과 실패 성격이 서로 다르다. 한 종류가 밀려도 다른 종류가 멈추지 않는다. ③ DLQ 를 큐마다 둔다 — 실패 메시지가 큐에 남아 계속 재처리되면 정상 처리량을 잠식한다. ④ 소비자 확장은 최대 1 파드로 제한한다 — 같은 대상을 여러 워커가 동시에 갱신하면 데이터베이스 잠금이 서로를 기다린다. 메시지 순서 보장이 없는 표준 큐를 쓰는 대신 동시성 쪽을 막았다. ⑤ KEDA 는 큐 깊이를 보고 확장을 시킨다. 1 큐를 1000개로 늘리면...				
-------	--	--	--	--	--

Repository	Region	Name	태그 불변성	암호화	취약점 스캔	수명주기 정책	상태
Private	ap-northeast-2	mealplanning/mp-account-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-chat-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-crawler-kurly	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-data-pipeline	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-elasticsearch-nori	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-frontend	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-mealplan-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-notify-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-ocr-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-operations-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-pantry-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-pgsync	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-price-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용

Private	ap-northeast-2	mealplanning/mp-ranking-serving	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-recipe-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-recipebook-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-rollouts-gatewayapi-plugin	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
Private	ap-northeast-2	mealplanning/mp-video-service	IMMUTABLE_WITH_EXCLUSION (예외: latest)	AES-256	Scan on push	① 무태그 14일 만료 ② 최근 20개 초과분 만료	적용
		합계 18개 리포지토리 · 실측 저장량 7.32 GB					

설계 근거	① 태그 불변성을 레지스트리가 강제한다 — '<sha>' 와 'X.Y.Z' 는 덮어쓸 수 없고 'latest' 만 예외다. 정책을 문서 가 아니라 레지스트리 에 박아 두면 사람이 지 키지 않아도 지켜진다. ② 무태그 만료를 0일 로 두지 않는다 — 빌드 부산물이 무태그로 잠 시 남는 구간이 있어 즉 시 만료하면 진행 중인 push 가 깨진다. ③ 보존 20개는 되돌릴 수 있는 폭이다. 작게 잡으면 롤백할 태그가 남지 않는다. ④ 리포지토리는 자동 생성되지 않는다 — Terraform 이 18개를 명시적으로 만든다. 이 름이 어긋나면 배포 시 점의 pull 실패로 가장 늦게 드러나므로 코드 에 고정한다. ⑤ 레지스트리 주소 = <ACCOUNT_ID>.dkr.ec r.ap-northeast-						
-------	--	--	--	--	--	--	--

Model	Region	호출 주체 (배치)	용도	신원 · 권한 경계	상태
amazon.nova-micro-v1:0 (+ 교차리전 추론 프로파일)	ap-northeast-2	pipeline : mp-score-review-sentiment	상품 리뷰 감성 점수화	IRSA `mp-pipeline-bedrock` — bedrock:InvokeModel 을 모델 ARN 단위로 고정	적용
amazon.nova-micro-v1:0	ap-northeast-2	pipeline : mp-summarize-reviews	리뷰 장단점 요약	동일	적용

설계 근거					
권한을 모델 ARN 으로 고정한다				교차리전 추론 프로파일을 쓰면 프로파일 ARN 과 실제 추론이 일어나는 리전의 파운데 이션 모델 ARN 이 둘 다 필요하다. 덕분에 모 델을 바꾸면 배포 직후 AccessDenied 로 즉시 드러난다 — 무증상으로 다른 모델로 넘어가 지 않는다.	적용
AI 의 본체는 Bedrock 이 아니다				재료 개체명 인식(CRF) · 신신도 예측 (XGBoost) · 레시피 랭킹(LightGBM) 은 전부 CPU 자체 모델이고 클러스터 안에서 돈다. Bedrock 은 리뷰 텍스트 처리 2건에만 쓴다 — 자체 모델로 만들 이유가 없는 범용 언어 작업이라서다.	적용
OpenSearch Serverless (벡터 저장소) 미 채택				최소 과금 단위(OCU)가 본 시스템 전체 인프 라 비용에 맞먹는다. 검색 대상 규모가 그 단 위를 채우지 못하므로 유사도 검색은 PostgreSQL 일반 테이블로 처리한다.	미채택
Amazon SageMaker 미채택				학습은 GPU 없이 CPU 로 끝나고, 서빙은 이 미 클러스터 안 파드(mp-ranking-serving)로 돌고 있다. 관리형 학습·서빙 인프라를 없으면 비용만 늘고 대체되는 것이 없다.	미채택

Amazon Comprehend — 미
채택

API 작업	Region	근거	상태
(전체)		① 리뷰 감성분석은 이미 Bedrock 배치로 돌고 있다 — 같은 일을 하는 부품을 둘 두지 않는다. ② 본 시스템 텍스트 처리의 핵심은 감성이 아니라 한국어 식재료 개체명 인식이고, 그것은 도메인 사전 기반 CRF 라 범용 관리형 NER 로 대체되지 않는다. 표준 품목명으로 정규화하는 부분이 실제 난이도인데 그 사전은 본 문서 대상이다. ③ 호출당 과금이 붙는 부품은 대체 수단이 있으면 붙이지 않는다.	미채택

AWS WAF					
구분	이름 / 구성	보호 대상	동작	설명	상태
Web ACL	mp-waf-public (scope REGIONAL)	mp-alb-public		ALB 에 부착. L7 계층에 검사점을 두려면 부착 가능한 로드밸런서가 있어야 한다 — ALB 를 택한 이유 중 하나다(NLB 에는 WAF 를 붙일 수 없다).	적용
Rule 1	rate-limit-per-ip (Rate-based)	전체 경로	Block	IP 당 5분 2,000 요청 초과 시 차단. CDN 을 두지 않으므로 L7 폭주 방어는 이 룰이 담당한다.	적용
Rule 2	aws-known-bad-inputs (AWS 관리형)	전체 경로	Block	알려진 악성 입력 패턴 차단	적용
Rule 3	aws-common-ruleset-count-only (AWS 관리형)	전체 경로	Count	의도적으로 Count 로 둔다. 이 룰셋의 본문 크기 제한이 영수증 이미지 업로드와 영상 URL 처리를 정상 요청인데도 끊는다. 차단으로 올리려면 그 두 경로를 먼저 예외 처리해야 한다.	적용

탐지 · 감사					
서비스	구성	대상	보존	설계 근거	상태
Amazon GuardDuty	계정 전체 활성화	계정 전체		탐지 계층이라 IAM:IRSA-RBAC(인가 계층)이 구조적으로 대체할 수 없다. VPC Flow Logs 를 따로 켜지 않아도 독립적으로 관찰한다 — Flow Logs 는 미채택이다.	적용
AWS CloudTrail	trail `mp-trail` — 다중 리전 · 로그 파일 검증 활성화	계정 전체 관리 이벤트	Object Lock COMPLIANCE 90일	계정을 나눌 수 없는 조건에서 감사 로그의 무결성을 Object Lock 으로 산다. 버킷은 버전관리 활성화 + SSE-S3.	적용
EKS 컨트롤플레인 로깅	audit → CloudWatch Logs `aws/eks/mp-eks/cluster`	mp-eks	30일	Kubernetes API 감사의 유일한 원본이다. CloudTrail 은 AWS API 감사이지 Kubernetes 감사가 아니다 — 클러스터 안에서 일어난 Secret 열람은 CloudTrail 에 남지 않는다. 실측 수집량 약 3.1 GB/일. api-authenticator 등 나머지 로그 타입은 같은 질문에 답하지 않으면서 수집 요금만 늘리므로 끈다.	적용
보안 알림	GuardDuty · CloudTrail · WAF → EventBridge → Lambda `mp-security-notifier` → Slack			클러스터 안의 알림 체계(Alertmanager)를 경유하지 않는다. 보안 알림이 필요한 국면은 클러스터가 이미 이상한 국면이라, 경유시키면 알림 자체가 순환 의존이 된다. Slack 웹훅은 Secrets Manager 에 두고 Lambda 가 실행 시점에 읽는다 — Terraform state 에 평문으로 남기지 않는다.	적용
저장 시 암호화	EBS(노드 루트:PVC) · S3(SSE-S3) · ElastiCache · EKS 관리형 컨트롤플레인			고객 관리형 KMS 키는 쓰지 않는다 — 키 관리 주체와 리소스 관리 주체가 같은 손이라 분리에서 오는 통제 이득이 없고, 키 소실이 곧 데이터 소실이라는 위험만 추가된다.	적용

미채택					
서비스	근거				상태
AWS Security Hub	여러 보안 서비스의 집계기인데 컨 것이 GuardDuty 하나뿐이라 집계할 대상이 없다.				미채택
AWS Config	리소스가 Terraform 으로 전량 코드화돼 있어 드리프트 탐지가 중복된다. 이미 plan 으로 같은 질문에 답하고 있다.				미채택

VPC Flow Logs	GuardDuty 가 독립적으로 관찰하므로 탐지 능력이 늘지 않고 로그 저장 비용만 남는다.				미채택
Amazon CloudFront	정적 자산 비중이 낮고 사용자가 단일 리전에 몰려 있어 CDN 이득이 작다. 대가로 L7 폭주 방어가 WAF 레이트 룰 단독이 된다는 점을 인지하고 택했다.				미채택
IAM Identity Center · SCP	계정 1개 · 사람 5명 · 전원 같은 팀. 권한을 주는 손과 상한을 정하는 손이 같으면 경계가 성립하지 않는다.				미채택
고객 관리형 KMS 키	위 '저장 시 암호화' 행 참조 — 관리 주체가 분리되지 않아 통제 이득이 없다.				미채택

Amazon Route 53 — 미채택. DNS 는 기존 등록기관(Cloudflare)의 관리형 DNS 를 유지한다.

항목	유형	대상	설명	상태
Route 53 Hosted Zone			서비스 도메인은 이미 외부 DNS 에서 운영 중이고, 옮겨도 얻는 것이 이관 작업뿐이다. AWS 리소스를 가리키는 데 Route 53 이 필수인 구간이 없다(ALB 는 CNAME 으로 충분하다).	미채택

AWS 측 진입 엔드포인트 · 인증서

항목	유형	대상	설명	상태
mp-alb-public	Application Load Balancer	internet-facing · public 서브넷 a-b	AWS 측 유일한 인터넷 진입점. 리스너 443(HTTPS · ACM 종단) + 80(301 리다이렉트). 기본 동작은 고정 응답이고, 호스트 헤더 규칙에 맞는 요청만 타깃 그룹으로 간다.	적용
타깃 그룹	target-type ip	mp-gw-public-istio 파드 2개 (노드 서브넷 ENI IP)	Cilium ENI 모드라 파드 IP 가 VPC 주소다 — ALB 가 노드를 거치지 않고 게이트웨이 파드를 직접 타깃으로 잡는다. 실측 상태 healthy 2/2.	적용
ACM 인증서	public 인증서	app.mealbong.cloud	상태 ISSUED. ALB 443 리스너에 부착돼 있다. 검증은 DNS(CNAME) 방식.	적용
ALB 액세스 로그	S3	mp-observability-eks	진입 계층의 요청 기록. 애플리케이션 로그(Loki)와 분리해 둔다 — 앱이 중단돼도 남아야 하는 기록이다.	적용

설계 근거	① 엣지를 하나로 만든다 — TLS 중단·WAF·레이트 제한·액세스 로그를 전부 ALB 한 곳에 모은다. 엣지가 둘이면 '이 트래픽이 어느 엣지를 거쳤는가'를 증명하는 장치(IP 허용목록 상시 갱신 · mTLS · 헤더 검증)가 계속 따라붙는데, 그 장치가 새 고장 원인이 된다. ② NLB 가 아니라 ALB 인 이유 — WAF 부착점과 호스트/경로 기반 라우팅이 필요했다. L4로는 둘 다 얻지 못한다. ③ TLS 는 ALB 에서 끝난다(ACM). 인증서 갱신이 자동이고 만료 사고의 여지가 사라진다.			
-------	--	--	--	--

5-1 미채택 결정

검토 대상	판정	근거	대체한 것
AWS 매니지드 서비스			
Amazon RDS	미채택	매니지드에서 사는 것은 백업·페일오버인데 CloudNativePG 가 이미 준다. 스키마-퍼-서비스·role 격리·확장 CR 을 그대로 유지하는 값이 더 크다	CloudNativePG 자체운영 (C-15)
Amazon OpenSearch Service	미채택	nori 형태소 분석기 커스텀 이미지를 쓴다. 매니지드로 가면 그 이미지를 못 넣는다	ECK 자체운영 (C-15)
Amazon MSK	미채택	브로커 3대와 그 운영비를 사는 대신, 수집 결과 운반이라는 실제 용도에는 S3 이벤트 + SQS 로 충분하다	S3 이벤트 알림 → SQS (C-10)
EC2 Spot	미채택	Blue-Green 은 promote 후 green 이 프로덕션 그 자체가 되어 파드가 이사하지 않는다 → '배포할 때만 Spot' 이 성립하지 않는다. 상시 Spot 도 메모리의 10%에만 걸린다(실측 CPU 요청/실사용 9.4배 vs 메모리 1.1배)	ON_DEMAND + Karpenter 버스트 (C-29)
AWS Backup	미채택	복구 절차가 운영 주체에 없다. barman + S3 라이프사이클로 같은 보호를 더 싸게 얻는다	barman-cloud → S3 (C-67)
Amazon Managed Prometheus / Grafana	미채택	대시보드와 알림 자산(PrometheusRule 20종)을 그대로 옮기는 쪽이 싸고 빠르다	kube-prometheus-stack 자체운영
Amazon Route 53	미채택	도메인이 이미 Cloudflare 에 있고 DR 전환을 CF 레코드 하나로 한다. 권한을 둘로 쪼갤 이유가 없다	Cloudflare DNS
Amazon Cognito	미채택	자체 JWT + OAuth(카카오·구글)가 이미 동작한다. 카카오는 Cognito 기본 제공 IdP 가 아니다	자체 account 서비스
Amazon Comprehend	미채택	한국어 식품 도메인 개체명에 맞지 않는다. 자체 CRF 가 더 정확하다	자체 CRF NER
EKS Pod Identity	미채택	제한 조건을 IAM 신뢰정책 안에 직접 쓰고 싶었다	IRSA (OIDC provider)
SSM Parameter Store	미채택	4KB 한도가 인증서·연결정보 묶음에 걸린다	AWS Secrets Manager
Network ACL	미채택	SG(상태유지)로 충분하고 NACL(무상태)은 반한 트래픽 규칙을 사람이 관리해야 한다 — 실수 표면만 넓어진다	Security Group 7종
IAM Identity Center · SCP	미채택	계정 1개 · 사람 5명 · 전원 같은 팀. 권한을 주는 손과 상한을 정하는 손이 같으면 SCP 는 통제가 아니라 번거로움이다	IAM 그룹 3종
AZ 3개	미채택	AZ 3개는 정족수가 필요한 컴포넌트가 있을 때 의미가 있는데, 데이터 티어 정족수를 클러스터 안에서 이미 만족한다	AZ 2개 (a · b)
온프레임 · 플랫폼			
Kubespray	미채택	kubeadm 을 직접 다루는 것이 학습 목적에 맞고, 추상화가 1계층 줄면 장애 시 원인이 보인다	kubeadm 직접
Cilium LB IPAM	미채택	MetalLB L2 가 이미 검증돼 있고 게이트웨이 2개만 노출하면 된다	MetalLB 0.16.1 L2
Istio ambient	미채택	sidecar 가 이미 동작하고 있고 ambient 로 얻는 것이 본 시스템 규모에서 크지 않다	Istio sidecar
Grafana Mimir	미채택	규모가 Prometheus 단독의 1~15% 수준이고 알림 경로가 길어진다	Prometheus 유지
ClickHouse	드롭	고불륨 시계열이 아직 없다. 승격 시 재도입	PostgreSQL 경량 가격 이력
GPU 학습	금지(제약)	GTX 1060 3GB — 학생 예산. 전 AI 를 CPU 로 설계	CRF · XGBoost · LightGBM

데이터 소스			
쿠팡 크롤	보류	robots + Akamai 블로커	마켓컬리로 대체
지마켓 타임딜	드롭	Cloudflare 차단	오아시스 타임/마감세일
냉장고를부탁해	드롭	공식 구조화 데이터 0건	만개의레시피 단일
할인 주기 예측 (LightGBM)	드롭	8주로는 할인 사이클이 부족해 예측이 성립하지 않는다	최저가 알림(통계 이상탐지)

5-2 비용

항목	구성 (실측)	수량	단가(USD)	단위	월(USD)
단계별 비용 성격					
① Docker (VM)	자체 하드웨어 (Proxmox 호스트 A) · 전기료만				0 (외부 지출)
② 온프레미 K8s	자체 하드웨어 2대 + S3 백업 버킷				약 1 (S3)
③ AWS EKS	아래 A + B				약 818
A. 고정비 — 구성 × AWS 공개 단가					
EC2 — EKS 워커	m7g.xlarge (Graviton) × 2대 상시	2 × 730h	0.2006	시간	292.88
EKS — 컨트롤플레인	클러스터 1개	730h	0.10	시간	73.00
EC2 — CI 서버	t4g.xlarge × 1대	730h	0.1664	시간	121.47
EC2 — 대시보드	t3.medium × 1대	730h	0.052	시간	37.96
VPC Interface Endpoint	SQS · Secrets Manager · STS × 2 AZ = ENI 6개	6 × 730h	0.013	시간	56.94
NAT Gateway — 시간	1대 (2 AZ 공유)	730h	0.059	시간	43.07
EBS (gp3)	노드루트120 + PVC230 + CI140 + 대시보드30	520 GiB	0.0912	GB-월	47.42
ElastiCache for Valkey	cache.t4g.micro × 2노드 (Multi-AZ)	2 × 730h	0.0192	시간	28.03
Application Load Balancer	ALB 1개 (시간 요금)	730h	0.0225	시간	16.43
AWS WAF	Web ACL 1개 + 규칙 3개	1 + 3	5 + 1×3	월정액	8.00
공인 IPv4	NAT 1 · ALB 2 · 대시보드 1 · CI 1	5 × 730h	0.005	시간	18.25
A 소계					743.45
B. 사용량 과금 — 실측 사용량 × 공개 단가					
CloudWatch Logs — 수집	EKS audit · 실측 3.1 GB/일	94.2 GB/월	0.76	GB	71.59
CloudWatch Logs — 보관	보존 30일 · 실측 1.36 GB	1.36 GB	0.0314	GB-월	0.04
NAT Gateway — 데이터 처리	실측 30.9 GB/월	30.9 GB	0.059	GB	1.82
B 소계 (주요 항목)	S3-ECR-SQS-Bedrock 등 소액 항목 포함				약 75
합계					약 818

비용 절감의 최대 후보는 CloudWatch Logs 수집(월 \$71.59) 이다 — EKS 감사로그 3.1GB/일. 로그 타입을 audit 만 남긴 것이 이미 1차 절감이다. Cost Explorer 는 SCP 로 차단돼 있어 단가는 Price List API 로 확인했다.

서비스	구성 (실측)	수량	단가(USD)	단위	월 비용(USD)
A. 고정비 — 구성 × 공개 단가					
Amazon EC2 — EKS 워커	m7g.xlarge (Graviton) × 2대 상시	2 × 730시간	0.2006	시간	292.88
Amazon EKS — 컨트롤플레인	클러스터 1개	730시간	0.1000	시간	73.00
Amazon EC2 — CI 서버	t4g.xlarge × 1대	730시간	0.1664	시간	121.47
Amazon EC2 — 대시보드	t3.medium × 1대	730시간	0.0520	시간	37.96
VPC Interface Endpoint	SQS · Secrets Manager · STS × 2 AZ = ENI 6개	6 × 730시간	0.0130	시간	56.94
NAT Gateway — 시간	1대 (2 AZ 공유)	730시간	0.0590	시간	43.07
Amazon EBS (gp3)	노드 루트 120 + EKS PVC 230 + CI 140 + 대시보드 30 = 520 GiB	520 GiB	0.0912	GB-월	47.42
ElastiCache for Valkey	cache.t4g.micro × 2노드 (Multi-AZ)	2 × 730시간	0.0192	시간	28.03
Application Load Balancer	ALB 1개 (시간 요금)	730시간	0.0225	시간	16.43
AWS WAF	Web ACL 1개 + 규칙 3개	1 + 3	\$5 + \$1×3	월정액	8.00
공인 IPv4 주소	NAT 1 · ALB 2 · 대시보드 1 · CI 1 = 5개	5 × 730시간	0.0050	시간	18.25
A 소계					743.45
B. 사용량 과금 — 실측 사용량 × 공개 단가					
CloudWatch Logs — 수집	EKS audit · 실측 3.1 GB/일	94.2 GB/월	0.7600	GB	71.59
CloudWatch Logs — 보관	보존 30일 · 실측 저장 1.36 GB	1.36 GB	0.0314	GB-월	0.04
NAT Gateway — 데이터 처리	실측 30.9 GB/월	30.9 GB	0.0590	GB	1.82
AZ 간 데이터 전송	노드 총 송신 실측 732 GB/월 — 전량이 AZ 를 넘는다고 본 상한값	732 GB × 양방향	0.0100	GB·방향	14.64
Amazon ECR — 저장	18개 리포 · 실측 7.32 GB	7.32 GB	0.1000	GB-월	0.73
Amazon S3 — 저장	11개 버킷 · 실측 6.08 GB	6.08 GB	0.0250	GB-월	0.15
Application Load Balancer — LCU	실측 평균 0.0006 LCU	0.0006 × 730시간	0.0080	LCU-시간	0.00
Amazon GuardDuty	실측 사용량 기준			사용량	0.02
Amazon SQS	월 100만 요청 미만 — 프리티어 내			요청	0.00
B 소계					89.01
합계 (A + B)					832.45

산정 기준	① 단가는 AWS Price List API(ap-northeast-2, 온디맨드) 2026-08-17 조회값이다. 할인 약정은 반영하지 않았다. ② 월 = 730시간으로 계산한다. ③ 수량은 실제 배포된 리소스를 API 로 센 값이다 — 계획 수량이 아니다. ④ AZ 간 데이터 전송은 노드 송신 총량 전부가 AZ 를 넘는다고 가정한 상한이다. 실제 과금분은 이보다 작다(상당량이 같은 AZ 안에서 끝난다)				
비용 구조 해석	① 상위 3개 항목(EKS 워커 \$292.88 · EKS 컨트롤플레인 \$73.00 · CI 서버 \$121.47)이 전체의 약 59% 다. 이 인프라의 비용은 저장소나 트래픽이 아니라 '상시 켜 두는 컴퓨터' 다. ② 노드를 3대에서 2대로 줄인 것이 단일 최대 절감이었다(\$146.44/월). 근거는 실측 — CPU 요청량이 실사용의 약 9배였고 메모리는 1.1배였다. 실제로 사는 것은 CPU 가 아니라 메모리다. ③ CloudWatch Logs 수집(\$71.59)이 4번째로 큰 항목이다 — 감사 로그를 남기기로 한 결정의 실제 가격이며, 로그 타입을 audit 하나로 줄여 이 값을 유지한다. ④ VPC Interface Endpoint(\$56.94)는 절감 항목이 아니라 순증이다. NAT 단일 구성에서 SQS-Secrets Manager-STs 호출이 NAT 와 함께 죽지 않게 하려고 지불한다. ⑤ Spot 인스턴스 미채택 — Blue-Green 배포에서 promote 후 green 이 프로덕션 그 자체가 되므로 '배포할 때만 Spot' 이 성립하지 않고, 상시 Spot 은 중단 시 재기동 비용이 절감분을 상쇄한다.				

5-4 검토 · 승인

구분	일자	담당	내용
작성	2026-08-24	봉수 (PL · 인프라)	3단계 대조 시방서 v1.0 신규 작성
원천 ①	2026-07-31 (SUPERSEDED)		docs/docker-infra-status.md — Docker(VM) 단계 형상. 폐기됐으나 이력 원문으로 참조
원천 ②	2026-08-05	봉수 · 태현	mp_k8s-오브젝트-시방서_(통합).xlsx (27시트) · docs/mp_k8s_infra_status.md
원천 ③	2026-08-17	봉수	mp_AWS-Cloud-설계시방서_AWS비용(단가).xlsx (19시트) · docs/mp_aws_prep_checklist.md (C-1~C-89)
검토 대기		태현 (플랫폼)	2·3·7·11·12번 시트 — 노드·네트워크·스토리지·배포·오토스케일
검토 대기		정현 (관측)	13번 시트 — 관측·알림
검토 대기		정은 (데이터)	6번 시트 — 데이터 티어
검토 대기		건우 (앱·AI)	5번 시트 — 애플리케이션 워크로드 · 서버리스

이 문서는 확정 형상만 신는다. 값이 바뀌면 원천 문서를 먼저 고치고 여기로 반영한다 — 반대 방향으로 하면 정보가 돌이 된다.